Bachelorarbeit

KI-gestütztes Liveticker-Redaktionssystem für den Profifußball

Automatisierte Textgenerierung für Echtzeit-Sportjournalismus mit Human-in-the-Loop-Kontrolle

Autor

Jonas Kimmer

Hochschule

Hochschule RheinMain

Fachbereich Wirtschaftsinformatik

Betreuung

Prof. Dr. [Name]

Firmenbetreuung

Stackwork GmbH, Eintracht Frankfurt

April 2026

Abstract

Die manuelle Erstellung von Liveticker-Einträgen im Profifußball ist ein kognitiv anspruchsvoller, zeitkritischer Prozess. Diese Arbeit konzipiert und implementiert ein hybrides KI-gestütztes Redaktionssystem auf Basis von Large Language Models: eine FastAPI/PostgreSQL-Backend-Architektur mit mehrsprachiger LLM-Pipeline und React-Frontend. Drei Betriebsmodi (*auto*, *coop*, *manual*) ermöglichen einen graduellen Übergang zwischen autonomer KI-Generierung und redaktioneller Kontrolle.

Die Evaluation umfasst 352 automatisierte Tests, eine TypeScript-Typenabdeckung von 95,84% sowie eine qualitative Bewertung von 16 KI-generierten Ticker-Einträgen aus 9 Bundesliga-Spielen. Die KI-generierten Texte erzielen einen Gesamtdurchschnitt von 4,3/5 Punkten (Korrektheit 4,6, Tonalität 4,1, Verständlichkeit 4,3). Auf Basis gemessener LLM-Latenzen ergibt sich ein geschätztes TTP-Potenzial von 3,4–5,9 Sekunden im *auto*-Modus gegenüber 30–120 Sekunden im manuellen Status quo.

Schlüsselwörter: Large Language Models, Liveticker, Natural Language Generation, Human-in-the-Loop, Few-Shot Prompting, Sportjournalismus, FastAPI, React, TypeScript

Inhaltsverzeichnis

1 Einleitung **3**

1.1 Problemstellung und Zielsetzung . . . 3

1.2 Forschungsfrage . . . 4

1.3 Methodisches Vorgehen . . . 5

1.4 Abgrenzung . . . 5

1.5 Aufbau der Arbeit . . . 6

2 Motivation und Anforderungen **7**

2.1 Redaktioneller Aufwand bei Livetickern . . . 7

2.2 Mehrsprachigkeit als Herausforderung . . . 9

2.3 White-Label-Bedarf im Profisport . . . 10

2.4 Anforderungsanalyse am Beispiel Eintracht Frankfurt . . . 11

2.5 Linguistische Anforderungen an die Textgenerierung . . . 12

2.6 Zusammenfassung der Systemanforderungen . . . 12

2.6.1 Funktionale Anforderungen . . . 12

2.6.2 Nicht-funktionale Anforderungen . . . 13

2.6.3 Architektur-Anforderungen . . . 14

3 Stand der Technik **15**

3.1 Large Language Models . . . 15

3.2 Prompt Engineering . . . 16

3.3 Natural Language Generation im Sportjournalismus . . . 17

3.4 Echtzeit-Kommunikation im Web . . . 17

3.5 ETL-Prozesse und Workflow-Automatisierung . . . 18

3.6 Backend-Technologien . . . 19

3.7 Frontend-Technologien . . . 20

3.8 Deployment und Containerisierung . . . 20

3.9 Synthese: Technologische Konsequenzen für das Systemdesign . . . 20

4 Systemkonzeption **22**

4.1 Überblick und Schichtenmodell . . . 22

4.1.1 Kommunikations- und Triggerarchitektur . . . 23
4.1.2 Partner-Team-Konzept und White-Label-Steuerung . . . 24
4.2 Backend-Konzeption . . . 24
4.2.1 Framework-Wahl, interne Struktur und Datenzugriff . . . 24
4.2.2 API-Design und Endpunktstrategie . . . 25
4.2.3 Asynchronität, Fehlerbehandlung und Robustheit . . . 25
4.2.4 Sicherheitskonzept . . . 26
4.3 Datenbankkonzeption . . . 26
4.3.1 Schemadesign-Prinzipien . . . 27
4.3.2 Status-Lifecycle der Ticker-Einträge . . . 27
4.3.3 Die drei Betriebsmodi . . . 28
4.3.4 Integrität, Idempotenz und Auswertbarkeit . . . 29
4.4 Workflow-Konzeption (n8n) . . . 29
4.4.1 Workflow-Klassen und Verantwortlichkeiten . . . 29
4.4.2 Workflow-Grenzen und projektspezifische Besonderheiten . . . 30
4.5 KI-Komponente . . . 31
4.5.1 Multi-Provider-Architektur . . . 31
4.5.2 Prompt-Engineering-Strategie . . . 32
4.6 Frontend-Konzeption . . . 33
4.6.1 Architektur, Zustandsmanagement und Hooks . . . 33
4.6.2 Layout, Moduslogik und Kommunikation . . . 34
4.7 Skalierbarkeit, Erweiterbarkeit und Systemgrenzen . . . 34
4.7.1 Skalierung, Erweiterbarkeit und Deployment . . . 34
4.7.2 Deployment-Architektur . . . 35
4.8 Fazit der Systemkonzeption . . . 35

5 Implementierung **36**
5.1 Backend-Implementierung . . . 36
5.1.1 Projektstruktur und Application Entry Point . . . 36
5.1.2 Datenmodelle: Datenbankschema und API-Validierung . . . 37
5.1.3 API-Router: Endpunktstruktur . . . 39
5.1.4 Ticker-Service: Domänenlogik . . . 40
5.1.5 LLM-Service: Prompt-Aufbau und Provider-Dispatch . . . 40
5.2 KI-Generierungspipeline . . . 42
5.3 Frontend-Implementierung . . . 44
5.3.1 Komponentenhierarchie . . . 44
5.3.2 React Context-Architektur . . . 44
5.3.3 Hook-Architektur . . . 45
5.3.4 Slash-Command-Parser (`parseCommand.ts`) . . . 45

5.3.5 Modusimplementierung im Frontend . . . 47
5.3.6 TypeScript-Migration . . . 47
5.4 n8n Workflow-Implementierung . . . 48
5.5 Qualitätssicherung und Tests . . . 50
5.6 Deployment . . . 50

6 Evaluation **51**
6.1 Zielsetzung der Evaluation . . . 51
6.2 Technische Qualitätssicherung . . . 51
6.2.1 Teststrategie und Testpyramide . . . 51
6.2.2 Frontend-Unit-Tests (Jest + React Testing Library) . . . 52
6.2.3 Backend-Unit-Tests (pytest) . . . 52
6.2.4 Browser-Smoke-Tests (Playwright) . . . 53
6.2.5 TypeScript-Typsicherheit . . . 54
6.3 Evaluation der KI-Textgenerierung . . . 54
6.3.1 Quantitative Metriken und Evaluationsinfrastruktur . . . 55
6.3.2 Modell- und Provider-Evaluation . . . 56
6.3.3 Einfluss der Stilprofile . . . 56
6.3.4 Qualitative Textbewertung . . . 58
6.3.5 Experteninterview-Auswertung . . . 62
6.3.6 Erweiterte Evaluation: LLM-as-Judge-Verfahren . . . 62
6.4 Latenzpotenzial der Betriebsmodi . . . 64
6.5 Performance und Laufzeitverhalten . . . 65
6.5.1 LLM-Latenz . . . 66
6.5.2 Frontend-Polling . . . 67
6.6 Anforderungsabgleich (Soll-Ist-Vergleich) . . . 67
6.7 Limitationen . . . 69
6.8 Zusammenfassung . . . 70

7 Diskussion **71**
7.1 Einordnung in den Stand der Technik . . . 71
7.1.1 Abgrenzung zu bestehenden Systemen . . . 71
7.1.2 Beitrag zur Forschung . . . 72
7.2 Kritische Reflexion . . . 72
7.2.1 Methodische Einordnung . . . 72
7.2.2 Technische Einordnung . . . 72
7.2.3 Inhaltliche Reflexion . . . 73
7.3 Diskussion der Betriebsmodi . . . 74
7.4 Diskussion der Prompt-Architektur . . . 74
7.4.1 Dynamische Stilreferenzen . . . 74

7.4.2 Temperaturparameter und Context-Builder 75
7.5 Implikationen für den Sportjournalismus 75
7.5.1 Veränderung der Redakteursrolle 75
7.5.2 Skalierbarkeit und White-Label 75
7.5.3 Ethische Überlegungen 76
7.6 Synthese 76

8 Fazit und Ausblick 78
8.1 Zusammenfassung der Ergebnisse 78
8.1.1 Technische Ergebnisse 78
8.1.2 KI-Textgenerierung 78
8.1.3 Systemarchitektur und Designentscheide 79
8.2 Beantwortung der Forschungsfrage 79
8.2.1 Synthese 80
8.3 Ausblick 80
8.3.1 Kurzfristige Erweiterungen 80
8.3.2 Mittelfristige Erweiterungen 81
8.3.3 Langfristige Perspektiven 82
8.4 Schlusswort 82

Kapitel 1

Einleitung

Ein Liveticker ist eine Form der Echtzeit-Berichterstattung, bei der Spielereignisse eines Fußballspiels (Tore, Karten, Auswechslungen, Atmosphäre und Spielanalyse) fortlaufend als kurze Texteinträge publiziert werden. Als digitale Textgattung verbindet er Berichts-, Meinungs- und Unterhaltungselemente und hat sich im Profisport zu einer zentralen Form der synchronen medialen Spielbegleitung entwickelt. Wie die Produktion dieser Texte unter zunehmendem Zeitdruck und wachsenden Anforderungen technologisch unterstützt werden kann, untersucht die vorliegende Arbeit.

1.1 Problemstellung und Zielsetzung

„Ein vernünftiger Live-Ticker ist fast unmöglich zu schreiben. Denn sein Konzept ist die komplette Überforderung. Des Autors. Des Schreibens. Und der Wirklichkeit.“ Mit diesen Worten beschreibt der Journalist Constantin Seibt das Grundproblem der Liveticker-Berichterstattung, eine Einschätzung, die im modernen digitalen Sportjournalismus bis heute nichts an Aktualität verloren hat (Seibt, zit. n. Beils 2023, S. 56). Der Liveticker hat sich zur prägenden Darstellungsform der Echtzeit-Berichterstattung im Profifußball entwickelt und steht dabei exemplarisch für einen Zielkonflikt zwischen Geschwindigkeit und Qualität, der mit rein manuellen Mitteln nicht auflösbar ist.

Im modernen digitalen Sportjournalismus lassen sich drei strukturelle Herausforderungen identifizieren, die den Einsatz technologischer Lösungen notwendig machen. Erstens zwingt der **operative Zeitdruck** Redakteure dazu, Spielereignisse in Echtzeit zu beobachten, einzuordnen und in stilistisch kohärente Kurztexte zu überführen, typischerweise innerhalb von 30 bis 120 Sekunden pro Eintrag (vgl. Walter 2020, S. 29; Bluhm & Schäfer 2023, S. 35). Zweitens stehen Redaktionen vor wachsenden **Internationalisierungsanforderungen**: Profifußballvereine bedienen zunehmend internationale Fanbases und müssen Inhalte parallel in mehreren Sprachen bereitstellen, ohne ausreichend mehrsprachiges Personal vorzuhalten. Drittens vollzieht sich ein **struktureller Wandel** der Vereine zu eigenständigen Medienproduzenten, die Redaktionswerkzeuge mit konfigurierbarem vereinsspezifischem Ton

und CI-konformer Gestaltung benötigen.

Diese Dimensionen werden in Kapitel 2 wissenschaftlich fundiert und in konkrete Systemanforderungen überführt.

Das übergeordnete Ziel der vorliegenden Arbeit ist daher die Konzeption und Implementierung eines produktionsfähigen Redaktionssystems, das diesen Konflikt durch den gezielten Einsatz von **Large Language Models** auflöst, während die finale Entscheidungshoheit und publizistische Verantwortung beim Menschen verbleiben. Das System wird dabei in zwei Ausprägungen realisiert: einer generischen Instanz für beliebige Vereine sowie einer vereinsspezifischen White-Label-Instanz am Beispiel Eintracht Frankfurt.

Um dieses Ziel zu operationalisieren, definiert und evaluiert die Arbeit drei Betriebsmodi: den **Status quo** (`manual`) als vollmanuell erstellte Vergleichsbasis, den **vollautonomen Modus** (`auto`) mit KI-basierter Texterstellung und Publikation ohne menschliches Korrektiv sowie den **hybriden Modus** (`coop`) als Kombination aus KI-generierten Textvorschlägen mit redaktioneller Prüfung und Freigabe.

1.2 Forschungsfrage

Aus der in Abschnitt 1.1 entwickelten Zielsetzung ergibt sich folgende Forschungsfrage, die der vorliegenden Arbeit zugrunde liegt:

> Wie lässt sich ein hybrides KI-gestütztes Redaktionssystem für die Liveticker-Erstellung im Profifußball konzipieren und prototypisch implementieren, das journalistische Qualitätsanforderungen hinsichtlich Korrektheit, Tonalität und Verständlichkeit erfüllt — und welches Potenzial zur Reduktion der Publikationslatenz lässt sich auf Basis der Systemarchitektur und gemessener Latenzdaten belegen?

Die Frage gliedert sich in zwei aufeinander aufbauende Teile: Der erste Teil ist ein **konstruktiver Beitrag** im Sinne des Design Science Research (Hevner et al. 2004) — er fragt, ob sich ein solches System überhaupt realisieren lässt, das gleichzeitig Echtzeitanforderungen, Mehrsprachigkeit und White-Label-Betrieb abdeckt. Der zweite Teil ist eine **Potenzialabschätzung**: Da ein kontrolliertes Feldexperiment mit echten Redakteuren im Evaluationszeitraum nicht durchführbar war, wird die erreichbare Publikationslatenz auf Basis gemessener LLM-Latenzen und der implementierten Systemarchitektur abgeschätzt — vergleichend gegenüber der in Kapitel 2.1 dokumentierten manuellen Baseline von 30–120 Sekunden pro Eintrag. Beide Teile werden in Abschnitt 1.3 methodisch vorbereitet und in Kapitel 6 vollständig durchgeführt.

1.3 Methodisches Vorgehen

Die Arbeit folgt einem konstruktiven Forschungsansatz im Sinne des **Design Science Research** (Hevner et al. 2004): Ein softwarebasiertes Artefakt wird als Antwort auf ein identifiziertes Praxisproblem entworfen, implementiert und evaluiert. Der Erkenntnisgewinn entsteht dabei nicht nur aus der Evaluation, sondern auch aus dem Entwurfs- und Implementierungsprozess selbst.

Das methodische Vorgehen gliedert sich in fünf Phasen. Die **Problemanalyse und Anforderungserhebung** vertieft literaturgestützt die Problemdimensionen und leitet funktionale sowie nicht-funktionale Anforderungen ab. Die **Technologierecherche** arbeitet den Stand der Technik in den relevanten Bereichen auf und bewertet ihn hinsichtlich der Eignung für das identifizierte Problem. Die **Systemkonzeption** entwirft eine dreischichtige Systemarchitektur mit Datenmodell, LLM-Pipeline und Prompt-Design. Die **Implementierung** setzt das System als produktionsfähige Webanwendung mit vollständiger API, browserbasiertem Frontend, automatisiertem ETL-System und Cloud-Deployment um. Die **Evaluation** umfasst technische Qualitätssicherung, Bewertung der KI-Textgenerierung sowie einen systematischen Anforderungsabgleich.

Die Evaluationsmethodik operationalisiert die Forschungsfrage entlang zweier Dimensionen: Die **Qualitätsdimension** wird über eine manuelle Bewertung von 16 KI-generierten Ticker-Einträgen aus 9 Bundesliga-Spielen auf den Skalen Korrektheit, Tonalität und Verständlichkeit (je 1–5 Punkte) operationalisiert, ergänzt durch ein **strukturiertes Experteninterview** mit einem professionellen Sportredakteur zur Validierung der Systemeignung im redaktionellen Arbeitskontext. Die **Latenzpotenzial-Dimension** stützt sich auf gemessene End-to-End-Latenzen des LLM-Service (25 Messpunkte, Render-Deployment) sowie die implementierte Systemarchitektur (Polling- und Prozessmodell), aus der die erreichbare TTP je Betriebsmodus abgeleitet wird. Für den statistischen Vergleich stehen nicht-parametrische Verfahren (Cliff's Delta, Bootstrap-Konfidenzintervalle) bereit; aufgrund der Stichprobengröße ($n = 16$) kommen diese im vorliegenden Evaluationsdesign nicht zur Anwendung, stehen jedoch für eine Folgestudie mit größerer Datenbasis zur Verfügung.

1.4 Abgrenzung

Das System geht bewusst über den Umfang eines typischen akademischen Projekts hinaus und zielt auf Produktionsfähigkeit: Eine umfassende automatisierte Testsuite, ein Cloud-Deployment und die Integration realer Datenquellen dokumentieren diesen Anspruch. Diese Praxisorientierung ist durch den beruflichen Kontext des Autors motiviert: Als Mitarbeiter der **Stackwork GmbH** im IT-Bereich von Eintracht Frankfurt entstand das System in direkter Kooperation mit den fachlichen Anforderungen einer professionellen Redaktion. Das Ergebnis ist ein vollständiges, eigenständig lauffähiges Redaktionssystem, das als Cloud-

Service alle Spiele der konfigurierten Wettbewerbe verarbeitet. Die technischen Details der Architektur und Implementierung werden in Kapitel 4 und 5 beschrieben.

Dennoch bestehen Einschränkungen hinsichtlich Authentifizierung, Evaluationsunabhängigkeit und thematischer Reichweite, die in Kapitel 7 kritisch reflektiert werden.

1.5 Aufbau der Arbeit

Die vorliegende Arbeit gliedert sich in acht Kapitel:

- **Kapitel 1 – Einleitung** *(vorliegendes Kapitel)*: Problemstellung und Zielsetzung, Forschungsfrage, methodisches Vorgehen und Aufbau der Arbeit.
- **Kapitel 2 – Motivation und Anforderungen:** Vertiefung der drei Problemdimensionen (redaktioneller Aufwand, Mehrsprachigkeit, White-Label-Bedarf), Ableitung der Systemanforderungen am Beispiel Eintracht Frankfurt, Experteninterview-Leitfaden und formale Anforderungsdefinition.
- **Kapitel 3 – Stand der Technik:** Large Language Models, Prompt Engineering, Natural Language Generation im Sport sowie Echtzeit-Technologien und ETL-Prozesse.
- **Kapitel 4 – Systemkonzeption:** Dreischichtige Architektur, Datenmodell, LLM-Pipeline und Prompt-Design.
- **Kapitel 5 – Implementierung:** Umsetzung als produktionsfähige Webanwendung mit TypeScript-Frontend, REST-API und n8n-ETL-System; ergänzt durch vollständige TypeScript-Typisierung, automatisierte Testsuite und Cloud-Deployment.
- **Kapitel 6 – Evaluation:** Technische Qualitätssicherung (Tests, Coverage, Typsicherheit), Evaluation der KI-Textgenerierung und systematischer Anforderungsabgleich.
- **Kapitel 7 – Diskussion:** Einordnung in den Stand der Technik, kritische Reflexion der Limitationen und Implikationen für den Sportjournalismus.
- **Kapitel 8 – Fazit und Ausblick:** Zusammenfassung der Erkenntnisse, Beantwortung der Forschungsfrage und Ausblick.

Kapitel 2

Motivation und Anforderungen

Dieses Kapitel begründet die Notwendigkeit eines KI-gestützten Liveticker-Systems und leitet daraus formale Anforderungen ab. Zunächst werden drei strukturelle Problemdimensionen der manuellen Liveticker-Produktion herausgearbeitet: der redaktionelle Aufwand unter Echtzeit-Bedingungen (Kap. 2.1), die Herausforderung der Mehrsprachigkeit (Kap. 2.2) sowie der White-Label-Bedarf im Profisport (Kap. 2.3). Kapitel 2.4 konkretisiert diese Dimensionen am Fallbeispiel Eintracht Frankfurt und beschreibt das zur empirischen Validierung eingesetzte Experteninterview. Kapitel 2.5 ergänzt die linguistischen Anforderungen an die Textgenerierung. Kapitel 2.6 fasst alle Anforderungen in einem formalen Pflichtenheft zusammen.

2.1 Redaktioneller Aufwand bei Livetickern

Die manuelle Erstellung von Livetickern zählt zu den kognitiv anspruchsvollsten Tätigkeiten im Journalismus (Beils 2023, S. 10). Was von außen wie eine schlichte Abfolge kurzer Texteinheiten wirkt, erweist sich bei näherer Betrachtung als hochkomplexer Produktionsprozess, der mehrere journalistische Kompetenzen simultan abverlangt. Bluhm und Schäfer (2023, S. 27) beschreiben unter Rückgriff auf Ojomo und Olomojobi (2021) die strukturelle Komposition von Liveticker-Einträgen als einen Vierschritt: **Narration** (Beschreibung von Aktionen), **Evaluation** (Äußerung von Meinungen über Szenen), **Elaboration** (Vermittlung von Hintergrundinformationen) und **Zusammenfassung** (Erstellung eines Überblicks über den Wettbewerbsverlauf).

In ihrer Konstruktion sind Liveticker damit als hybride Form zu verstehen, in der sich Meinungsbeiträge, Berichte und Unterhaltungselemente verschmelzen (Bluhm & Schäfer 2023, S. 27). Diese inhaltliche Vielschichtigkeit unterscheidet den Liveticker grundlegend von anderen journalistischen Kurzformaten. Liveticker werden dabei in Abständen von einer bis drei Minuten aktualisiert (Hauser 2008, S. 3–4; Bluhm & Schäfer 2023, S. 32), wobei Schnelligkeit und Fehlerfreiheit in einem permanenten Widerspruch stehen: Trotz der extremen Publikationsfrequenz gilt sprachliche und inhaltliche Korrektheit als oberste Priorität

(Bluhm & Schäfer 2023, S. 35).

Aus den genannten Merkmalen ergibt sich folgender konkreter Workflow: Ein Ereignis findet auf dem Spielfeld statt — etwa ein Tor, eine Auswechslung oder ein Foul. Die Autorin oder der Autor muss das Geschehen in Sekundenbruchteilen wahrnehmen und die journalistische Relevanz bewerten (vgl. Bluhm & Schäfer 2023, S. 27; Walter 2020, S. 29). Unmittelbar muss der Eintrag mental strukturiert und in sprachlich angemessene Form gebracht werden. Gleichzeitig sind Kontextinformationen abzurufen: Wie steht es? Wer hat das letzte Tor erzielt? In welcher Minute war der letzte Eintrag? Diese kognitive Parallelverarbeitung aus Beobachtung, Bewertung, Kontextualisierung und Textproduktion macht den Liveticker zu einer Echtzeit-Textgattung, die sich fundamental von retrospektiven Formaten wie dem Spielbericht unterscheidet (vgl. Hauser 2008, S. 2–3).

Erschwerend kommt der extreme Zeitdruck hinzu. Unmittelbar nach einer relevanten Spielszene haben die Autorinnen und Autoren nur wenige Augenblicke Zeit, das Geschehen konzise, anschaulich und vor allem spannend-unterhaltsam zu beschreiben (Meier 2018, zitiert nach Bluhm & Schäfer 2023, S. 27). Da zum Gegenlesen keine Zeit bleibt, müssen Verfassen und Redigieren in einem einzigen Prozess erfolgen (Walter 2020, S. 29). Verschärft wird die Situation dadurch, dass Ticker-Autoren häufig parallel weitere Aufgaben übernehmen müssen — wie etwa die Erstellung des abschließenden Spielberichts (Bluhm & Schäfer 2023, S. 35). Die Fehleranfälligkeit unter diesen Bedingungen ist systemimmanent: Zahlendreher bei Spielminuten, falsch zugeordnete Spielernamen oder stilistische Brüche innerhalb eines Tickers sind keine Ausnahme, sondern eine vorhersehbare Konsequenz des Produktionsdrucks (vgl. Walter 2020, S. 29; Bluhm & Schäfer 2023, S. 35).

Das Skalierungsproblem verschärft sich an Spieltagen mit Parallelspielen erheblich. An einem typischen Bundesliga-Samstag finden bis zu sechs Partien zeitgleich statt (vgl. DFL 2024, Rahmenspielplan Bundesliga). Redaktionen, die für mehrere Spiele Ticker anbieten, müssen entweder entsprechend viele Autoren gleichzeitig einsetzen oder einzelne Autoren mehrere Ticker parallel betreuen lassen — mit absehbaren Qualitätseinbußen (vgl. Bluhm & Schäfer 2023, S. 32, 35). Für Vereinsredaktionen, die zusätzlich zu Pflichtspielen auch Jugend-, Frauen- oder Freundschaftsspiele abdecken wollen, ist eine rein manuelle Skalierung personell und wirtschaftlich kaum darstellbar.

Die personellen Konsequenzen sind in der Praxis spürbar. Liveticker-Stellen werden häufig mit freien Mitarbeitern oder studentischen Hilfskräften besetzt, die teils räumlich isoliert arbeiten (Bluhm & Schäfer 2023, S. 32). Der Aufwand bleibt hoch, die Ressourcen begrenzt, und die Qualität unterliegt einer strukturellen Inkonsistenz: Wechselndes Personal, unterschiedliche Erfahrungsstufen und variierende Arbeitsbedingungen — Stadion, Pressebereich oder Home-Office — führen dazu, dass Tonalität und sprachliches Niveau von Spiel zu Spiel schwanken. Diese Inkonsistenz ist kein individuelles Versagen, sondern eine systemische Folge des manuellen Produktionsprozesses (vgl. Bluhm & Schäfer 2023, S. 32, 35). Eine technologische Unterstützung ist daher keine optionale Ergänzung, sondern eine strukturelle

Notwendigkeit.

Die empirische Validierung dieser Problemdimension erfolgt über das Experteninterview (Kap. 2.4). IF1 fragt nach dem typischen Ablauf der Liveticker-Produktion an einem Spieltag; IF2 nach der Personalbindung; IF3 nach den Parallelaufgaben des Ticker-Autors; IF4 nach typischen Fehlerquellen unter Zeitdruck (Zahlendreher, falsche Spielernamen, stilistische Brüche); IF5 nach der Handhabung von Parallelsspielen sowie Jugend- und Frauenspielen. *(IF6 ist thematisch in Kap. 2.2 eingebettet. IF7–IF9 sind thematisch in Kap. 2.3 eingebettet.)*

2.2 Mehrsprachigkeit als Herausforderung

Die Globalisierung des Profifußballs hat die Anforderungen an die Reichweite digitaler Inhalte grundlegend verändert. Internationale Fangemeinschaften erwarten heute eine mediale Begleitung in ihrer Landessprache. Am Beispiel der Premier League lässt sich dieser Wandel besonders deutlich illustrieren: Der internationale Markt ist für die Liga längst bedeutender als der heimische englische (Beils 2023, S. 221). Dies betrifft zunehmend auch Bundesliga-Vereine mit zweistelligen Millionen-Reichweiten auf globalen Plattformen (vgl. Beils 2023, S. 200). Am Beispiel von Eintracht Frankfurt zeigt sich dies konkret: Durch Spieler wie Makoto Hasebe und Daichi Kamada verfügte der Verein in zurückliegenden Spielzeiten über eine beachtliche internationale Anhängerschaft, insbesondere in Japan — erkennbar an den fremdsprachigen Interaktionen auf den offiziellen Social-Media-Kanälen des Vereins, die im Rahmen der Projektbeobachtung dokumentiert wurden. Eine rein deutschsprachige Berichterstattung wird dieser Reichweite nicht mehr gerecht.

Die Herausforderung beschränkt sich dabei nicht auf eine reine Übersetzung existierender Inhalte. Smulczynski (2020, S. 57) zeigt am Beispiel der mehrsprachigen Berichterstattung von *Die Zeit*, dass mehrsprachige Echtzeitinhalte ein journalistisches Qualitätsmerkmal darstellen, das über wörtliche Translation hinausgeht. Im Kontext von Sport-Livetickern kommt hinzu, dass sprachliche Konventionen, idiomatische Wendungen und die emotionale Tonalität kulturspezifisch variieren. Ein euphorischer Ticker-Eintrag auf Deutsch folgt anderen stilistischen Mustern als sein englisches oder japanisches Pendant. Es handelt sich daher weniger um ein Übersetzungs- als um ein Lokalisierungsproblem.

Der operative Zeitdruck potenziert sich mit jeder zusätzlichen Sprache erheblich: Jede Sprache erfordert spezialisiertes Personal für zeitaufwendige Übersetzungs- und Anpassungsprozesse, die manuell kaum mit der Echtzeit-Anforderung vereinbar sind (vgl. Bluhm & Schäfer 2023, S. 32). Eine manuelle Skalierung auf mehrere Sprachen ist wirtschaftlich und personell für die meisten Vereinsredaktionen nicht darstellbar. Moderne Large Language Models eröffnen hier eine neue Perspektive: Sie können Texte nicht nur übersetzen, sondern direkt in der Zielsprache generieren — unter Berücksichtigung von Stil, Tonalität und kulturellem Kontext (vgl. OpenAI 2023, S. 1; Kap. 3.1). Diese Fähigkeit macht eine mehrsprachige

Liveticker-Produktion erstmals ohne proportionalen Personalaufwand denkbar.

Die empirische Validierung dieser Problemdimension erfolgt über das Experteninterview (Kap. 2.4); Interview-Frage IF6 adressiert direkt, ob bei Eintracht Frankfurt aktuell Bedarf an mehrsprachigen Tickern besteht und welche Sprachen dabei relevant wären.

2.3 White-Label-Bedarf im Profisport

Vereine und Verbände kommunizieren heute zunehmend direkt mit ihrer Anhängerschaft und haben eigene Redaktionen aufgebaut (Beils 2023, S. 155). Die Konsequenz für die Liveticker-Produktion ist erheblich: Vereinseigene Inhalte folgen einer anderen Logik als neutrale Berichterstattung.

Der entscheidende Unterschied liegt im Blickwinkel. Während überregionale Medien neutral bleiben, betrachten Vereinsticker die Partie aus der Perspektive der eigenen Fans (Beils 2023, S. 57). Ein Ereignis muss je nach Medium anders getönt sein: emotional für den Vereinskanal, sachlich für neutrale Plattformen. Darüber hinaus erfordert die fanbezogene Perspektive einen dritten Tonfall: Bei anhaltenden Rückständen oder schwachen Leistungen des eigenen Teams kann ein kritisch-analysierender Stil angemessener sein als unkritische Euphorie. Daraus resultieren drei funktional unterschiedliche Stilprofile: neutral, euphorisch und kritisch. Die in Kapitel 2.1 beschriebene personelle Fluktuation verschärft das Problem zusätzlich: Eine konsistente Markenstimme ist mit wechselndem Personal manuell kaum zu gewährleisten. Eine technologische Lösung muss daher auch die stilistische Konsistenz adressieren.

Im Softwarekontext wird dieses Problem durch sogenannte White-Label-Architekturen gelöst: Ein generisches Kernsystem wird so konzipiert, dass es für verschiedene Abnehmer individuell konfigurierbar ist — ohne den Quellcode zu ändern (vgl. Kap. 4). Überträgt man dieses Prinzip auf die Liveticker-Produktion, ergibt sich die Anforderung, dass ein und dasselbe System sowohl einen vereinsspezifischen Ticker mit emotionaler Fanbindung als auch einen neutralen, anbieterunabhängigen Ticker bedienen können muss. Die stilistische Differenzierung — von der Wortwahl über die Tonalität bis hin zu vereinsspezifischen Redewendungen — muss dabei als Konfiguration abbildbar sein, nicht als separate Implementierung.

Die empirische Validierung dieser Problemdimension erfolgt über das Experteninterview (Kap. 2.4): IF7 fragt nach vorhandenen Styleguides und der aktuellen Stilkonsistenz; IF8 nach den charakteristischen Merkmalen des Eintracht-Ticker-Stils im Vergleich zur neutralen Berichterstattung; IF9 nach der Bedeutung stilistischer Konsistenz über Spiele hinweg.

Wie sich diese Anforderungen am konkreten Beispiel eines Bundesliga-Vereins ausprägen, zeigt der folgende Abschnitt.

2.4 Anforderungsanalyse am Beispiel Eintracht Frankfurt

Eintracht Frankfurt dient in dieser Arbeit als konkreter Anwendungsfall für die vereinsspezifische White-Label-Instanz. Die Anforderungsanalyse stützt sich auf die in den vorangegangenen Abschnitten hergeleiteten Problemdimensionen, eine Analyse der öffentlich zugänglichen digitalen Kanäle des Vereins sowie ein strukturiertes Experteninterview mit einem professionellen Sportredakteur von Eintracht Frankfurt. Die theoretisch hergeleiteten Anforderungen werden dabei durch das Interview empirisch validiert — insbesondere hinsichtlich des redaktionellen Workflows (Kap. 2.1), des Bedarfs an mehrsprachiger Berichterstattung (Kap. 2.2) und der stilistischen Konsistenzanforderungen (Kap. 2.3). Das strukturierte Leitfadeninterview nach Kuckartz (2018) wurde gewählt, weil es bei kleiner Fallzahl eine systematische Vergleichbarkeit der Antworten mit den hergeleiteten Anforderungen ermöglicht. Der Interviewpartner ist als Experte im Sinne von Meuser und Nagel (1991) einzustufen, da er über mehrjährige Redaktionserfahrung im Vereinsmedienbereich verfügt. Die Auswahl eines Einzelfalls ist durch die explorative Zielsetzung der Arbeit gerechtfertigt. Die vollständige Auswertung des Interviews erfolgt in Abschnitt 6.3.5 mittels thematischer Inhaltsanalyse nach Kuckartz (2018).

Aus den drei Problemdimensionen ergeben sich folgende vereinsspezifische Anforderungen: Eintracht Frankfurt bestreitet pro Saison bis zu 60 Pflichtspiele; pro Spiel fallen durchschnittlich 20–40 Ticker-Einträge an (Schätzung auf Basis öffentlich zugänglicher Vereinsticker; im Experteninterview bestätigt), was den **Zeitdruck** auf manuelle Abdeckung mit konsistenter Qualität auf ein personell kaum darstellbares Niveau treibt. Die internationale Fanbasis — insbesondere in Japan (vgl. Kap. 2.2) — begründet den Bedarf an mehrsprachigen Tickern in mindestens Deutsch und Englisch, perspektivisch auch Japanisch. Hinsichtlich der Tonalität folgen vereinseigene Ticker einer emotionalen, fanbezogenen Perspektive, die sich von neutraler Berichterstattung überregionaler Medien unterscheidet (vgl. Beils 2023, S. 57); das System muss diese stilistische Differenzierung konfigurierbar abbilden.

Über die in den Kapiteln 2.1–2.3 eingebetteten Validierungsfragen (IF1–IF9) hinaus adressiert das Experteninterview zwei weitere Themenblöcke, die erst nach der Systemdemonstration beantwortet werden können: die **Systembewertung** (IF10–IF15, eingebettet in Abschnitt 6.3.5) und die **Zukunftsperspektive** (IF16–IF18, adressiert in Kap. 7.5.2). Ein valides Systemurteil zu Block 3 und 4 setzt die vorherige Exposition mit dem laufenden System voraus, weshalb diese Blöcke sequenziell nach der Systemdemonstration eingesetzt werden. Das Interview bestätigte die drei Problemdimensionen in allen wesentlichen Punkten: den hohen manuellen Aufwand an Spieltagen, den Bedarf an konsistenter Stilführung sowie das Interesse an mehrsprachiger Berichterstattung. [**INTERVIEW-ERGEBNISSE ERGÄNZEN**] Die vollständige Auswertung mit Einzelergebnissen erfolgt in Abschnitt 6.3.5. Der vollständige Interviewleitfaden mit 18 Fragen ist als separates Dokument beigefügt.

2.5 Linguistische Anforderungen an die Textgenerierung

Neben den organisatorischen und stilistischen Anforderungen stellt die automatisierte Liveticker-Produktion auch spezifische linguistische Anforderungen. Liveticker stellen eine linguistisch eigenständige Textgattung dar (Hauser 2008, S. 1). Hauser charakterisiert sie als **schriftlich-grafische Simultanerzählung**. Im Gegensatz zum Spielbericht fehlt der *benefit of hindsight*; es handelt sich um eine quasi-synchrone Berichterstattung (Hauser 2008, S. 2).

Für das Prompt-Design sind vier linguistische Merkmale leitend:

- **Konzeptionelle Mündlichkeit** (Oraliteralität): Verschriftlichung mündlicher Ausdrucksweisen (Smulczynski 2020, S. 54).
- **Syntax**: Ellipsen und verblose Satzeinheiten dominieren; Sätze sollten nicht mehr als 15 Wörter umfassen, Hauptsatzkonstruktionen sind vorherrschend (Walter 2020, S. 40, 52).
- **Tempus**: Präsens für gleichzeitige Situationen, Perfekt statt Präteritum für vergangene Ereignisse (Walter 2020, S. 39).
- **Graphostilistik**: Großschreibung und Zeicheniterationen (z. B. „TOOOOR!!!") als Äquivalent zu Lautstärke und Betonung (Smulczynski 2020, S. 54; Walter 2020, S. 41).

Diese Merkmale definieren das Zielformat, auf das das LLM durch gezieltes Prompt Engineering konditioniert werden muss (vgl. Kap. 3.2). Sie bilden damit die linguistischen Anforderungen, die bei der Prompt-Konzeption in Kapitel 4.5.2 operationalisiert werden. Die linguistischen Anforderungen werden nicht separat im Interview validiert, sondern fließen als Operationalisierungsvorgaben in das Prompt Design ein (Kap. 4.5.2); ihre Wirksamkeit wird in der Qualitätsevaluation (Kap. 6.3.4) anhand der Dimension Verständlichkeit bewertet.

2.6 Zusammenfassung der Systemanforderungen

Aus den vorangegangenen Problemdimensionen und der Anforderungsanalyse werden im Folgenden die formalen Anforderungen an das System abgeleitet. Diese gliedern sich in funktionale Anforderungen (F), nicht-funktionale Anforderungen (N) und architektonische Anforderungen (A). Die Evaluation in Kapitel 6.6 überprüft jeden einzelnen Punkt gegen den implementierten Stand. Technische Qualitätsanforderungen (N3, N4) werden zusätzlich in Abschnitt 6.2 belegt; die KI-Textgenerierung (F2, F3, F11) in Abschnitt 6.3; die Betriebsmodi (F1) in Abschnitt 6.4.

2.6.1 Funktionale Anforderungen

Nr.	Anforderung	Herleitung
F1	Drei Betriebsmodi (auto, coop, manual)	Kap. 1.1, 2.1
F2	KI-Textgenerierung für alle Event-Typen	Kap. 2.1, 2.5
F3	Drei Stilprofile (neutral, euphorisch, kritisch)	Kap. 2.3, 2.4
F4	Few-Shot-Prompting mit Stilreferenzen	Kap. 2.5, 3.2
F5	Ticker-Lifecycle (draft → published / rejected / deleted)	Kap. 2.1
F6	Slash-Command-Parser für manuellen Modus	Kap. 2.1 (Status quo)
F7	Mehrsprachige Textgenerierung (Deutsch, Englisch; erweiterbar auf weitere Sprachen)	Kap. 2.2
F8	Idempotenter Datenimport via n8n	Kap. 2.1 (Robustheit)
F9	Provider-Fallback-Kette	Kap. 2.1 (Verfügbarkeitsanforderung im Echtzeit-Produktionsbetrieb)
F10	Deduplizierung von Ticker-Einträgen	Kap. 2.1 (Korrektheit)
F11	Pre-Match-Kontextgenerierung (Verletzungen, H2H, etc.)	Kap. 2.5 (Elaboration)
F12	Live-Statistik-Updates	Kap. 2.1 (Echtzeit)

2.6.2 Nicht-funktionale Anforderungen

Nr.	Anforderung	Herleitung
N1	Concurrency-Begrenzung für LLM-Aufrufe	Kap. 2.1 (Zeitdruck)
N2	Retry-Logik mit Rate-Limit-Erkennung	Kap. 3.1 (Robustheit)
N3	Transaktionale Testisolierung	Technische Qualitätssicherung (Echtzeit-Produktionssysteme erfordern Testisolierung aufgrund der Datenbankkopplung)
N4	TypeScript-Typsicherheit	Technische Qualitätssicherung (Wartbarkeit und Fehlervermeidung im Team)
N5	Responsive UI (Mobile-tauglich)	Kap. 2.1 (Einsatzszenarien: Stadion und Home-Office erfordern mobile Bedienbarkeit)
N6	Fehlerresistenz im Frontend	Kap. 2.1 (Robustheit)

2.6.3 Architektur-Anforderungen

Nr.	Anforderung	Herleitung
A1	Drei-Schichten-Architektur	Kap. 2.1, 2.3 (Modularität)
A2	Isolierte Datenzugriffsschicht: Persistenzlogik muss von API-Logik getrennt sein, sodass Datenbankzugriffe unabhängig testbar und austauschbar bleiben	Wartbarkeit, Testbarkeit; Mandanten mit unterschiedlichen Backend-Anforderungen müssen bedienbar sein, ohne den Anwendungskern zu ändern (vgl. Kap. 2.3)
A3	REST-API zur vollständigen Systemsteuerung und Mandantenkonfiguration	Kap. 2.3 (White-Label)
A4	Vollständiges Datenmodell	Kap. 2.1, 2.4
A5	White-Label-Fähigkeit (ef_whitelabel / generic)	Kap. 2.3, 2.4

Die drei Problemdimensionen — operativer Zeitdruck, Mehrsprachigkeit und White-Label-Bedarf — sind dabei keine isolierten Herausforderungen, sondern greifen strukturell ineinander: Zeitdruck macht menschliche Skalierung unmöglich, Mehrsprachigkeit multipliziert diesen Aufwand, und White-Label-Anforderungen verlangen eine stilistische Konsistenz, die wechselndes Personal nicht zuverlässig gewährleisten kann. Dies begründet die Notwendigkeit einer integrierten technologischen Lösung: KI-gestützte Textgenerierung mit konfigurierbarer Stilsteuerung (F1–F4) als Antwort auf den Zeitdruck, mehrsprachige Generierung (F7) als Antwort auf die Internationalisierungsanforderung, sprachgattungskonformes Prompt Design (F2, F4, F11) als Antwort auf die in Kapitel 2.5 hergeleiteten linguistischen Anforderungen, und eine konfigurierbare Instanzlogik (A5) als Antwort auf den White-Label-Bedarf. Die vorstehenden 23 Anforderungen (F1–F12, N1–N6, A1–A5) bilden damit das vollständige Pflichtenheft für die Systemkonzeption in Kapitel 4; die hierfür notwendigen technologischen Grundlagen werden in Kapitel 3 aufgearbeitet.

Kapitel 3

Stand der Technik

Dieses Kapitel legt die technologischen Grundlagen für das vorliegende System. Es gliedert sich in neun Bereiche: KI-gestützte Textgenerierung mit Large Language Models und Prompt Engineering (Kap. 3.1–3.2), automatisierte Sprachgenerierung im Sportjournalismus (Kap. 3.3), Echtzeit-Kommunikationsmuster (Kap. 3.4), ETL-Workflows und Automatisierungsplattformen (Kap. 3.5) sowie die eingesetzten Backend-, Frontend- und Deployment-Technologien (Kap. 3.6–3.8). Kapitel 3.9 synthetisiert die Erkenntnisse und leitet daraus die Technologieentscheidungen für das Systemdesign ab.

3.1 Large Language Models

Große Sprachmodelle (Large Language Models, LLMs) sind KI-Systeme, die auf Basis umfangreicher Textkorpora trainiert werden und natürliche Sprache verstehen sowie erzeugen können. Ihr Einsatz hat sich in den vergangenen Jahren von der Forschung in die industrielle Anwendung verbreitet — unter anderem für automatisierte Textproduktion im Medienbereich (vgl. Bluhm & Schäfer 2023, S. 33).

Den technischen Ausgangspunkt bildet die **Transformer-Architektur** (Vaswani et al. 2017), die Text mithilfe sogenannter **Aufmerksamkeitsmechanismen** (Attention) verarbeitet: Das Modell gewichtet bei der Verarbeitung eines Wortes alle anderen Wörter im Kontext nach ihrer Relevanz. Dadurch kann es semantische Zusammenhänge auch über lange Textabschnitte hinweg erfassen. Die Textgenerierung erfolgt autoregressiv — das Modell erzeugt ein Wort nach dem anderen, wobei jedes neue Wort auf den bereits generierten Text aufbaut.

Diese Modelle werden auf umfangreichen Textkorpora vortrainiert und erwerben dabei ein breites Sprachverständnis. Aufgrund dieser Breite können sie direkt auf neue Aufgaben angewendet werden, ohne aufgabenspezifisches Fine-tuning zu erfordern — eine Eigenschaft, die als **Zero-Shot-Generalisierung** bezeichnet wird (Brown et al. 2020, S. 2). Das ermöglicht den Einsatz von LLMs für spezialisierte Aufgaben wie die Liveticker-Generierung, ohne modellseitige Anpassungen vornehmen zu müssen.

Die empirische Grundlage für das **In-Context Learning** wurde mit GPT-3 gelegt (Brown et al. 2020). Das Prinzip, Modelle über Beispiele im Prompt zu konditionieren, ohne ihre Gewichte anzupassen, hat sich seitdem als Standard etabliert und wird auch im vorliegenden System genutzt.

Für die Liveticker-Produktion sind **kompakte Modelle** besonders relevant. Die Generierungsaufgabe erfordert kurze Texte (1–3 Sätze) auf Basis strukturierter Eingabedaten (Spielereignisse, Kontext) — eine Aufgabe, die keine komplexe Inferenz oder umfangreiches Weltwissen voraussetzt. Für solche klar definierten, strukturierten Generierungsaufgaben erzielen kompakte Modelle eine für den Anwendungsfall ausreichende Qualität bei deutlich niedrigerer Latenz und geringeren Kosten — Eigenschaften, die für hochfrequente Echtzeit-Generierungsszenarien entscheidend sind (vgl. Ouyang et al. 2022, S. 3).

Trotz ihrer Leistungsfähigkeit weisen LLMs strukturelle Schwächen auf, insbesondere die Neigung zu **Halluzinationen** — also der Generierung von Inhalten, die sachlich unzutreffend oder erfunden sind (OpenAI 2023, S. 46). Im journalistischen Kontext ist dies kritisch, da faktische Fehler die Glaubwürdigkeit nachhaltig beschädigen (Bluhm & Schäfer 2023, S. 33). Dies begründet den **hybriden Ansatz** der vorliegenden Arbeit: Das System generiert Textvorschläge, die finale Freigabeentscheidung verbleibt jedoch beim Redakteur.

3.2 Prompt Engineering

Die Qualität der Ausgaben eines LLM hängt maßgeblich vom **Prompt** ab. Prompt Engineering bezeichnet die systematische Gestaltung dieser Eingaben, um das Modell ohne Änderung seiner Gewichte zu einem gewünschten Verhalten zu lenken (Brown et al. 2020).

Für die Anwendung von LLMs auf spezifische Aufgaben existieren grundsätzlich zwei Ansätze: **Fine-tuning** erfordert ein aufgabenspezifisches Nachtrainieren des Modells auf kuratierten Datensätzen und ist ressourcenintensiv (Rechenzeit, Infrastruktur, Datenkuration). **Prompting-basierte Ansätze** hingegen nutzen das vortrainierte Modell direkt und steuern dessen Verhalten ausschließlich über die Eingabeformulierung (Brown et al. 2020, S. 1). Für ressourcenbeschränkte Kontexte wie den Vereinssport ist Prompting dabei nicht nur kostengünstiger, sondern auch methodisch vorzuziehen: Da kein domänenspezifischer Trainingsdatensatz in ausreichender Größe existiert, würde Fine-tuning zu Overfitting neigen, während Few-Shot-Prompting mit wenigen kuratierten Beispielen auskommt.

Innerhalb des Prompting-Paradigmas lassen sich zwei wesentliche Techniken unterscheiden. Beim **Zero-Shot Prompting** erhält das Modell ausschließlich eine Aufgabenbeschreibung ohne Demonstrationsbeispiele (Brown et al. 2020, S. 13); dieser Ansatz maximiert Flexibilität und eignet sich für generische Anwendungsfälle ohne spezifische Stilerwartungen. Beim **Few-Shot Prompting** werden dem Modell neben der Aufgabenbeschreibung zusätzlich wenige Beispiele (typischerweise 1–5) als Stilreferenz bereitgestellt, die das Modell auf eine gewünschte Tonalität konditionieren, ohne dass modellseitiges Fine-tuning erforderlich

ist (Brown et al. 2020, S. 14).

Moderne Chat-Modelle unterscheiden strukturell zwischen zwei Prompt-Rollen: Der **System Prompt** legt die übergeordnete Persona und Constraints des Modells fest (z. B. Stilanforderungen, inhaltliche Grenzen, Ausgabeformat), während der **User Prompt** die konkrete Aufgabe mit aufgabenspezifischen Daten enthält. Diese Trennung erlaubt es, allgemeine Verhaltensregeln einmalig im System Prompt zu definieren und den User Prompt auf die jeweilige Aufgabe zu beschränken.

Ein weiterer zentraler Steuerungsparameter ist die **Temperature**, die die Wahrscheinlichkeitsverteilung der Token-Auswahl beeinflusst (Brown et al. 2020, S. 24). Niedrige Werte (z. B. 0,0–0,3) führen zu deterministischeren, faktentreueren Ausgaben; höhere Werte (z. B. 0,7–1,0) erhöhen die kreative Varianz. Für journalistische Anwendungen, bei denen faktische Korrektheit priorisiert wird, sind niedrige Temperature-Werte vorzuziehen.

Die konkrete Prompt-Struktur, Few-Shot-Beispiele und Temperature-Konfiguration des vorliegenden Systems werden in Kapitel 4.5.2 beschrieben.

3.3 Natural Language Generation im Sportjournalismus

Die automatisierte Erzeugung natürlichsprachlicher Texte aus strukturierten Daten (Data-to-Text Generation) bildet die technische Grundlage für KI-gestützte Berichterstattungssysteme (Puduppully & Lapata 2021, S. 510). Historische Ansätze basierten auf regelbasierten Template-Architekturen mit vorhersagbarer Ausgabe, aber stilistischer Monotonie; neuronale End-to-End-Systeme verbesserten die sprachliche Natürlichkeit auf Kosten erhöhter Halluzinationsanfälligkeit (Puduppully & Lapata 2021, S. 520; vgl. Beils 2023, S. 207). Klassische NLG-Architekturen werden durch Large Language Models (Kapitel 3.1–3.2) für die meisten Anwendungsfälle weitgehend ersetzt — für diese Arbeit ist ausschließlich der LLM-basierte Ansatz relevant: Das vorliegende System nutzt Few-Shot-Prompting (Kapitel 3.2) zur stilkontrollierten Textgenerierung; Halluzinationsrisiken werden durch das Human-in-the-Loop-Design des `coop`-Modus (Kapitel 4.3.3) abgefangen.

3.4 Echtzeit-Kommunikation im Web

Die Übertragung von Daten zwischen Frontend und Backend erfordert eine ausgewogene Balance zwischen Latenz, Ressourcenverbrauch und Implementierungskomplexität. Für Echtzeitanwendungen wie Liveticker stehen mehrere etablierte Kommunikationsmuster zur Verfügung, die sich hinsichtlich dieser Dimensionen unterscheiden.

- **HTTP-Polling** ist das einfachste Muster: Der Client sendet in regelmäßigen Intervallen Anfragen an den Server, um neue Daten abzurufen. Der Vorteil liegt in der Ein-

fachheit und breiten Kompatibilität — das Muster nutzt Standard-HTTP-Requests ohne zusätzliche Protokollschichten. Nachteile sind die inhärente Latenz (Daten werden erst beim nächsten Polling-Zyklus sichtbar) und der erhöhte Ressourcenverbrauch durch wiederholte Request/Response-Zyklen, selbst wenn keine neuen Daten vorliegen. Für Anwendungen mit akzeptabler Latenztoleranz (z. B. wenige Sekunden) und begrenzter Updatefrequenz ist Polling jedoch eine robuste und wartbare Lösung.

- **Server-Sent Events (SSE)** bieten als Zwischenlösung eine HTTP-basierte Push-Verbindung vom Server zum Client (Hickson 2015) — ressourceneffizienter als Polling, jedoch auf unidirektionale Kommunikation beschränkt.
- **WebSocket** etabliert eine vollständig bidirektionale, persistente Verbindung über das WebSocket-Protokoll (RFC 6455). Nach einem initialen HTTP-Handshake wechselt die Verbindung in ein binäres Protokoll mit minimalen Framing-Overhead (Fette & Melnikov 2011, S. 1). WebSocket eignet sich für latenzkritische Echtzeitanwendungen, bei denen minimale Latenz und eine persistente Verbindung erforderlich sind — auch wenn die Kommunikation faktisch überwiegend unidirektional (Server → Client) verläuft. RFC 6455 nennt Börsen-Ticker explizit als Einsatzszenario — eine strukturelle Parallele zum Sport-Liveticker. Der Preis für niedrige Latenz sind höhere Implementierungskomplexität und erhöhter Server-Ressourcenbedarf durch persistente Verbindungen. Clientseitig ist eine Reconnect-Strategie erforderlich, da Netzwerkunterbrechungen die persistente Verbindung unerwartet beenden können — üblich ist dabei ein exponentielles Backoff, um den Server bei wiederholten Verbindungsversuchen nicht zu überlasten.

Die Wahl des Kommunikationsmusters hängt von den Anforderungen der Anwendung ab: Polling bevorzugt Einfachheit, SSE bietet effizienten unidirektionalen Push, WebSocket minimiert Latenz bei bidirektionaler Kommunikation. Hybridansätze nutzen verschiedene Muster für unterschiedliche Datenströme innerhalb derselben Anwendung, um Trade-offs gezielt zu optimieren.

Die konkrete Umsetzung im vorliegenden System wird in Kapitel 4.6.5 beschrieben.

3.5 ETL-Prozesse und Workflow-Automatisierung

Die Datenbeschaffung in datengetriebenen Anwendungen folgt überwiegend dem **Extract-Transform-Load-Prinzip (ETL)**. Daten werden aus heterogenen Quellen extrahiert, in ein einheitliches Zielformat transformiert und in den Datenspeicher geladen (Freitas et al. 2025, S. 807). Für Sport-Liveticker bedeutet dies die Integration von Live-Spielereignissen, Statistiken, Aufstellungen und Mediendaten aus unterschiedlichen APIs oder Datenfeeds. Die Herausforderung besteht darin, diese Daten zeitnah, konsistent und in einer für die Anwendungslogik verwertbaren Struktur bereitzustellen.

Historisch wurden ETL-Prozesse durch monolithische Batch-Jobs oder direkt in Anwendungscode implementiert. Moderne Ansätze setzen zunehmend auf **Workflow-Orchestrierungsplattformen**, die ETL-Logik als konfigurierbare Prozesse abbilden. Diese Plattformen basieren typischerweise auf **Directed Acyclic Graphs (DAGs)**, die Datenflüsse als gerichtete, azyklische Graphen modellieren und damit Abhängigkeiten zwischen Verarbeitungsschritten explizit machen. Jeder Knoten im DAG repräsentiert eine Transformation oder einen API-Aufruf, während Kanten die Datenflussrichtung definieren (vgl. Venkiteela 2025, S. 3).

Low-Code- und No-Code-Plattformen haben diesen Ansatz weiter demokratisiert: Statt prozeduralen Code zu schreiben, werden Workflows grafisch zusammengestellt. Diese visuellen Editoren erleichtern sowohl die Entwicklung als auch die Wartung von Integrationspfaden. Für kleinere Teams oder prototypische Systeme bieten solche Plattformen eine erheblich niedrigere Einstiegshürde als vollprogrammierte ETL-Pipelines. Gleichzeitig erlauben sie bei Bedarf die Einbettung von Custom-Code-Blöcken für komplexe Transformationen (vgl. Venkiteela 2025, S. 2).

Innerhalb dieser Klasse lassen sich cloud-basierte iPaaS-Dienste (z. B. Zapier, Make) von self-hostbaren Open-Source-Plattformen (z. B. n8n) unterscheiden (vgl. Venkiteela 2025, S. 2). Cloud-Dienste bieten sofortige Verfügbarkeit ohne Infrastrukturaufwand, unterliegen jedoch Datenschutzeinschränkungen und eingeschränkter Kontrolle über Ausführungsumgebung und Kosten. Self-hostbare Plattformen wie n8n ermöglichen dagegen den Betrieb in der eigenen Infrastruktur, bieten volle Kontrolle über Datenflüsse (vgl. Nakavisute & Sincharoonsak 2025) und erlauben den direkten Datenbankzugriff — ein entscheidender Vorteil, wenn Workflows transaktionale Schreiboperationen direkt in die Persistenzschicht ausführen sollen, ohne den Umweg über HTTP-Endpunkte nehmen zu müssen (vgl. Venkiteela 2025, S. 4).

Für Echtzeitanwendungen wie Liveticker ist zusätzlich die **Latenz der Orchestrierung** kritisch. Während klassische Batch-ETL-Prozesse mit Stunden- oder Minutentakten arbeiten (vgl. Freitas et al. 2025, S. 808), müssen Sport-Events innerhalb von Sekunden verarbeitet werden. Das erfordert Orchestrierungsplattformen mit niedrigem Overhead, asynchroner Verarbeitung und effizienten Webhook-Mechanismen, um auf externe Ereignisse sofort reagieren zu können (vgl. Nakavisute & Sincharoonsak 2025).

Die konkrete Umsetzung der ETL-Architektur im vorliegenden System wird in Kapitel 4.4 beschrieben.

3.6 Backend-Technologien

Python hat sich als Standard für datenintensive Webanwendungen und KI-Systeme etabliert (Stack Overflow Developer Survey 2024). Pythons `asyncio`-Eventloop ermöglicht über **ASGI**-Frameworks wie `FastAPI` nicht-blockierende Verarbeitung: Während eine Anfrage auf

einen LLM-Aufruf oder eine Datenbankabfrage wartet, können andere Anfragen parallel bearbeitet werden (Grigorev 2019). `FastAPI` generiert aus `Pydantic`-Schemata automatisch eine OpenAPI-konforme Dokumentation, die gleichzeitig als maschinenlesbarer Vertrag für Frontend und n8n-Workflows dient. Das **Repository Pattern** (Fowler 2002, S. 322) kapselt den Datenzugriff und entkoppelt die Service-Schicht von der Persistenzschicht — eine Voraussetzung für transaktionale Testbarkeit. `SQLAlchemy` implementiert dieses Muster als typsichere ORM-Schicht über PostgreSQL, das mit vollständigen ACID-Garantien referenzielle Integrität erzwingt und über native JSONB-Spalten auch semistrukturierte Daten verwaltet. Die konkrete Backend-Architektur wird in Kapitel 4.2 beschrieben.

3.7 Frontend-Technologien

TypeScript erweitert JavaScript um ein statisches Typsystem, das Typfehler zur Entwicklungszeit erkennt und Schnittstellenänderungen sofort an allen Aufrufstellen als Compilerfehler sichtbar macht (Microsoft 2012). React zerlegt die Benutzeroberfläche in isolierte, wiederverwendbare Komponenten; Zustand und Seiteneffekte werden über React Hooks verwaltet, während die `Context API` global geteilten Zustand bereitstellt, ohne ihn über Prop-Ketten durchzureichen. Als **Single-Page Application (SPA)** aktualisiert das Frontend die Darstellung dynamisch ohne vollständige Seitenneuladen — geeignet für interaktive Werkzeuge mit häufigen Zustandsänderungen. Die konkrete Frontend-Architektur wird in Kapitel 4.6 beschrieben.

3.8 Deployment und Containerisierung

Docker (Merkel 2014) paketiert die Anwendung mitsamt Laufzeitumgebung in reproduzierbare Container; `Docker Compose` koordiniert mehrere Dienste über eine deklarative Konfigurationsdatei. **Platform-as-a-Service (PaaS)**-Anbieter wie Render abstrahieren die Infrastrukturverwaltung und ermöglichen Deployments direkt aus Git-Branches. PaaS setzt ein **Stateless-Design** voraus: Persistente Daten werden in externe Dienste ausgelagert, sodass zusätzliche Instanzen ohne Zustandskonsistenzprobleme gestartet werden können. Die konkrete Deployment-Architektur wird in Kapitel 4.7.4 beschrieben.

3.9 Synthese: Technologische Konsequenzen für das Systemdesign

Der Stand der Technik zeigt, dass für das vorliegende System kein einzelnes Werkzeug ausreicht, sondern ein abgestimmtes Zusammenspiel mehrerer Technologieschichten notwendig

ist. Kompakte Sprachmodelle mit Few-Shot-Prompting (Kap. 3.1–3.2) liefern die erforderliche Textqualität bei vertretbarer Latenz und ohne ressourcenintensives Fine-tuning. Der hybride NLG-Ansatz mit menschlicher Kontrollinstanz (Kap. 3.3) adressiert das systemimmanente Halluzinationsproblem von LLMs im journalistischen Kontext und entspricht der in Kap. 2.1 beschriebenen Anforderung, redaktionellen Aufwand zu reduzieren, ohne editoriale Kontrolle aufzugeben. Das vorliegende System nutzt HTTP-Polling für Ticker- und Ereignisdaten sowie WebSocket für Medienupdates; die Begründung dieser Hybridwahl erfolgt in Kapitel 4.6.5. n8n als Low-Code-Orchestrierungsplattform (Kap. 3.5) entkoppelt die ETL-Komplexität vom Anwendungskern und ermöglicht eine schnelle Anpassung von Datenquellen unabhängig von Backend-Deployment-Zyklen.

Die Anforderungen aus Kapitel 2 begründen konkret folgende Technologieentscheidungen: Die Datenschutz- und Betriebsanforderung aus Kap. 2.3 motiviert die Wahl von n8n gegenüber cloud-basierten iPaaS-Diensten; die Kostenrestriktion und Latenzanforderung aus Kap. 2.1 den Einsatz kompakter Sprachmodelle; die stark verknüpften Spielentitäten eine relationale Datenbankstruktur gegenüber dokumentbasierten Alternativen. Diese technologischen Grundlagen münden in die Systemkonzeption (Kapitel 4) und deren Implementierung (Kapitel 5).

Kapitel 4

Systemkonzeption

Dieses Kapitel beschreibt die Konzeption des Liveticker-Systems auf Basis der in Kapitel 2 hergeleiteten Anforderungen und der in Kapitel 3 aufgearbeiteten technologischen Grundlagen. Es gliedert sich in acht Bereiche: das Schichtenmodell und die Kommunikationsarchitektur (Kap. 4.1), die Backend-Konzeption mit API-Design und Sicherheitskonzept (Kap. 4.2), die Datenbankkonzeption mit Ticker-Lifecycle und Betriebsmodi (Kap. 4.3), die Workflow-Konzeption in n8n (Kap. 4.4), die KI-Komponente mit Multi-Provider-Architektur und Prompt-Design (Kap. 4.5), die Frontend-Konzeption mit Hook- und Kontextarchitektur (Kap. 4.6), Skalierbarkeit und Systemgrenzen (Kap. 4.7) sowie ein abschließendes Fazit der Systemkonzeption (Kap. 4.8).

4.1 Überblick und Schichtenmodell

Das System ist als **eigenständiger Cloud-Service** konzipiert, der alle Spiele der konfigurierten Wettbewerbe automatisch verarbeitet — von der Datenimportierung über die KI-Textgenerierung bis zur Publikation. Es ist eine serviceorientierte, dreischichtige Architektur, die Datenbeschaffung, Anwendungslogik und Präsentation klar voneinander trennt. Diese Schichtentrennung folgt dem etablierten Prinzip der *Separation of Concerns* (Parnas 1972), das die unabhängige Weiterentwicklung einzelner Systemteile ermöglicht.

Die **Datenbeschaffungs- und Automatisierungsschicht** wird über n8n umgesetzt. Sie integriert externe Datenquellen (insbesondere Football-API sowie Medienquellen wie ScorePlay und Social-Feeds), transformiert die Rohdaten in ein einheitliches internes Format und schreibt diese strukturiert in die Persistenzschicht beziehungsweise über definierte Backend-Endpunkte in das System ein. Die Entkopplung der Integrationslogik in n8n ermöglicht eine schnelle Anpassung von Import-Workflows, ohne den Anwendungskern ändern zu müssen.

Die **Anwendungs- und Persistenzschicht** basiert auf FastAPI (Python) und PostgreSQL. Sie stellt die REST-Schnittstellen bereit, verwaltet den Zustandsübergang der Ticker-Einträge (z. B. draft, published, rejected), orchestriert KI-Generierungsschritte und

übernimmt die konsistente Speicherung aller Domänendaten (u. a. Teams, Wettbewerbe, Spiele, Events, Statistiken, Ticker- und Media-Daten).

Die **Präsentationsschicht** ist als White-Label-Frontend in React und TypeScript realisiert. Sie bildet den Redaktionsworkflow für Auswahl, Sichtung, Bearbeitung und Publikation ab und bleibt dabei mandantenfähig konfigurierbar (z. B. Instanz, Stil, Sprache, Branding). Technisch kombiniert das Frontend REST-basiertes Polling für Ticker- und Spieldaten mit einem dedizierten WebSocket-Kanal für Echtzeit-Medienupdates.

Die folgende Abbildung zeigt die drei Schichten mit ihren Technologien und Kommunikationspfaden:

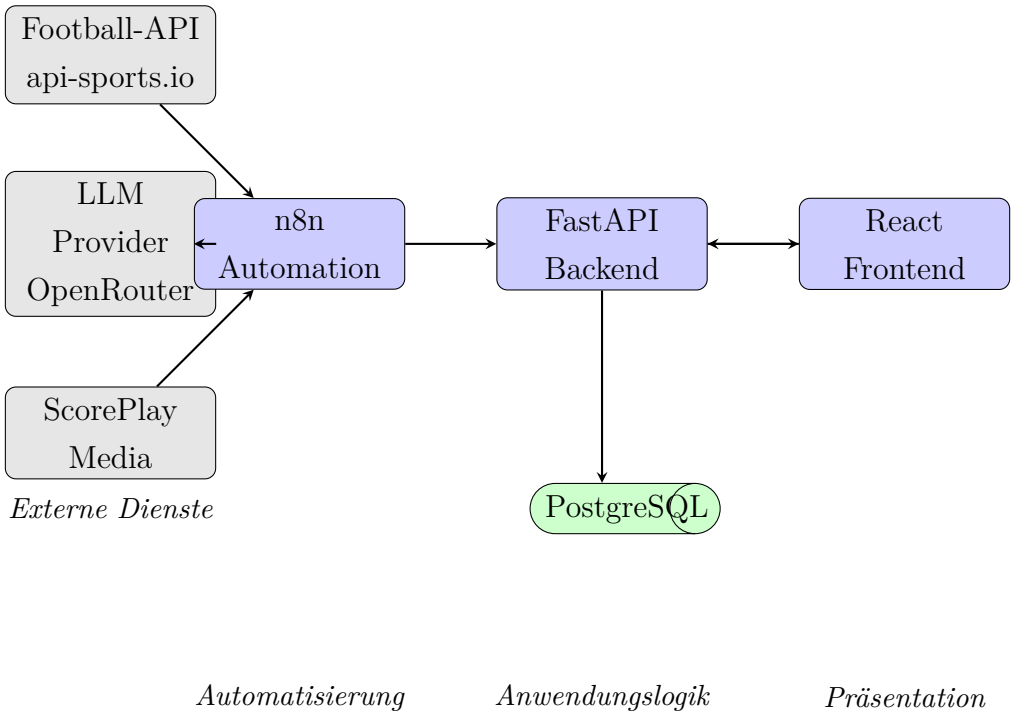

Abbildung 4.1: Systemarchitektur: drei Schichten und externe Dienste

4.1.1 Kommunikations- und Triggerarchitektur

Der Kommunikationsfluss folgt dem Entwurfsprinzip **„read first, trigger if missing"** — strukturell verwandt mit dem Cache-Aside Pattern (vgl. Fowler 2002): Das Frontend liest zunächst den vorhandenen Datenbestand über REST; nur bei fehlenden Daten wird ein passender n8n-Webhook ausgelöst. Dadurch werden externe API-Aufrufe minimiert und redundante Importe vermieden.

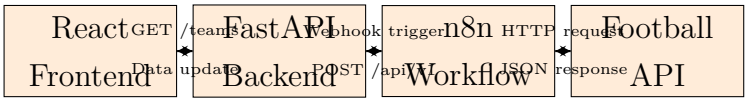

Abbildung 4.2: Kommunikationsfluss: Frontend, Backend, n8n und externe APIs

Die Systemkopplung erfolgt über drei klar getrennte Schnittstellentypen. Die **REST-API** bildet den stabilen Kern für alle Lese- und Schreiboperationen: Navigation, Ticker-CRUD, Statusübergänge und manuelle Eingaben. **n8n-Webhooks** werden bedarfsgesteuert in drei Klassen eingesetzt: Initialisierungs-Trigger bauen fehlende Stammdaten auf; Match-Kontext-Trigger lösen spielbezogene Importe nach Matchauswahl aus; Generierungs-Trigger stoßen KI-Textprozesse für Events, Matchphasen und Zusammenfassungen an. Der **WebSocket-Kanal** `/ws/media` übernimmt den Echtzeit-Push für latenzkritische Medieninhalte; Ticker- und Matchdaten verbleiben bewusst im stabileren Polling-Modell. Diese hybride Triggerarchitektur unterstützt sowohl redaktionelle Kontrolle im Sinne des **Human-in-the-Loop**-Ansatzes als auch weitgehend automatisierte Live-Prozesse.

4.1.2 Partner-Team-Konzept und White-Label-Steuerung

Das System unterscheidet zwischen dem **Partner-Team** — dem Verein, für den die White-Label-Instanz konfiguriert ist — und den jeweiligen Gegnern. Diese Unterscheidung wird über ein konfigurierbares Team-Keyword umgesetzt, das sowohl in den n8n-Workflows als auch im Frontend gegen den Teamnamen abgeglichen wird.

Auf Basis dieser Erkennung werden zwei Verarbeitungspfade gesteuert: Partner-Team-Spiele verwenden den `ef_whitelabel`-Instanzpfad mit euphorischem Stil, alle anderen Spiele den `generic`-Pfad mit neutralem Stil. Das Frontend blendet vereinsspezifische Bereiche (Social-Media-Panels, Torjubel-Videos) ausschließlich bei erkannten Partner-Team-Spielen ein. Für Tormomente des Partner-Teams werden zusätzlich Spielerdaten und Torjubel-Video-URLs aus einer externen Vereinsquelle in den Prompt-Kontext eingebettet.

Das Datenbankschema enthält ergänzend ein `is_partner_team`-Flag, das für zukünftige Erweiterungen vorgesehen ist. Die konkreten Implementierungen sind in Kapitel 5.2.2 (Datenbankmodell, `is_partner_team`-Flag) und Kapitel 5.4.6 (Frontend-Moduslogik, `isOurTeam`-Erkennung) dokumentiert.

4.2 Backend-Konzeption

Die Backend-Konzeption umfasst die Wahl des Frameworks, die interne Schichtstruktur, das API-Design sowie das Sicherheitskonzept. Alle Entscheidungen orientieren sich an den Anforderungen aus Kap. 2.6 und den technologischen Grundlagen aus Kap. 3.6.

4.2.1 Framework-Wahl, interne Struktur und Datenzugriff

Als Backend-Framework wurde FastAPI gewählt (vgl. Abschnitt 3.6 für die technische Einordnung). Die Entscheidung begründet sich durch drei systemspezifische Anforderungen: Die parallele Verarbeitung von LLM-Aufrufen, externen API-Abfragen und Medienverarbeitung erfordert native Unterstützung für asynchrone I/O; die automatisch generierte

OpenAPI-Spezifikation vereinfacht die Integration mit n8n-Workflows erheblich; und die enge Pydantic-Integration reduziert Schnittstellenfehler zwischen Frontend, Backend und n8n, indem alle Eingaben gegen typisierte Schemas validiert werden.

Die interne Struktur folgt einer klaren Trennung aus API-Routern, Repository-Schicht (vgl. Kap. 3.6) sowie SQLAlchemy-ORM-Modellen und Pydantic-Schemas. Die Router bilden die HTTP-Schnittstelle und übernehmen Validierung, Statuscodes sowie die Orchestrierung einzelner Use Cases. Datenbankzugriffe sind in dedizierten Repositories gekapselt. Dadurch bleibt die API-Schicht frei von SQL-Details, während Persistenzlogik zentral gebündelt und wiederverwendbar gehalten wird.

Für LLM-nahe Abläufe wird ergänzend eine Service-Schicht genutzt (insbesondere `ticker_service` und `llm_service`). Die Service-Schicht wird dabei gezielt, aber nicht flächendeckend eingesetzt: Viele Standard-CRUD- und Listenoperationen folgen dem Muster Router → Repository, während komplexere Generierungs- und Orchestrierungspfade über Services laufen.

Für die Datenrepräsentation gilt eine bewusste Trennung: ORM-Modelle definieren die Persistenzstruktur, Pydantic-Schemas die API-Verträge. Das stabilisiert die Schnittstellen gegenüber internen Refactorings und verbessert Wartbarkeit und Testbarkeit.

4.2.2 API-Design und Endpunktstrategie

Die Backend-Schnittstelle ist als versionierte REST-API unter `/api/v1` ausgelegt und orientiert sich am Architekturstil **Representational State Transfer** (Fielding 2000, Kap. 5). Endpunkte sind ressourcenorientiert strukturiert (z. B. `teams`, `matches`, `ticker`, `media`, `clips`) und folgen konsistenten Benennungs- und Methodenregeln. `GET` dient lesenden Abfragen, `POST` dem Erzeugen bzw. Triggern, `PATCH` partiellen Zustandsänderungen und `DELETE` kontrollierten Löschoperationen. Insbesondere im Ticker-Kontext bildet `PATCH` den redaktionellen Lebenszyklus (`draft` → `published` bzw. `rejected`) explizit ab.

Konzeptionell wird zwischen fachlichen Datenendpunkten und prozessualen Triggerendpunkten getrennt. Dadurch bleiben Leseoperationen seiteneffektfrei, während Import- und Generierungsprozesse gezielt ausgelöst werden können. Fehlerfälle werden konsistent über HTTP-Statuscodes und strukturierte Fehlermeldungen kommuniziert, sodass Frontend und n8n-Workflows deterministisch reagieren können.

4.2.3 Asynchronität, Fehlerbehandlung und Robustheit

Die Laufzeitkonzeption setzt Asynchronität gezielt auf I/O-intensiven Pfaden ein — insbesondere bei LLM-Generierungsrouten, Media-Verarbeitung und WebSocket-Kommunikation — um die Reaktionsfähigkeit unter paralleler Last zu erhöhen, ohne Worker dauerhaft zu blockieren. Die Anzahl gleichzeitiger LLM-Aufrufe begrenzt ein konfigurierbarer **Semaphore-Mechanismus** (Standard: 8 parallele Anfragen), der Lastspitzen und Rate-Limit-Probleme

reduziert. Ergänzend greift ein **Retry-Mechanismus mit Backoff**: Bei Rate-Limit-Fehlern folgen automatische Wiederholungsversuche mit linearem Backoff (3 Versuche, Wartezeiten 30 s / 60 s); bei sonstigen transienten Fehlern exponentielles Backoff (1 s / 2 s). Nicht alle Endpunkte sind vollständig asynchronisiert; langlaufende Integrationslogik ist in n8n ausgelagert, wodurch der API-Server als transaktionaler Kern entlastet wird. Für den Live-Betrieb ist das Backend auf kontrollierte Degradation ausgelegt: Fehler werden über konsistente HTTP-Antworten strukturiert zurückgegeben; bei Teilausfällen externer Dienste bleiben bereits persistierte Daten verfügbar und redaktionelle Kernprozesse können fortgeführt werden. Die Datenpersistenz ist auf **idempotente Verarbeitung** ausgelegt (vgl. Kap. 4.3.4).

4.2.4 Sicherheitskonzept

Das System adressiert Sicherheit auf drei Ebenen, wobei der Projektrahmen einer Bachelorarbeit eine bewusste Priorisierung erfordert:

Die gesamte Kommunikation erfolgt über HTTPS; die Render-Plattform übernimmt TLS-Terminierung und Zertifikatsverwaltung automatisch. Der Backend-Server konfiguriert eine **CORS-Whitelist**, die Anfragen ausschließlich von autorisierten Frontend-Ursprüngen akzeptiert; API-Schlüssel werden über Umgebungsvariablen injiziert und sind weder im Quellcode noch im Frontend exponiert. Die Pydantic-basierte Eingabevalidierung schützt vor Injection-Angriffen, da sämtliche Request-Payloads gegen typisierte Schemas geprüft werden, bevor sie die Geschäftslogik erreichen. Eine feingranulare Benutzerauthentifizierung ist im aktuellen Stand **nicht implementiert**; das System setzt als internes Redaktionswerkzeug auf Netzwerksegmentierung. Für produktiven Mehrbenutzerbetrieb wäre ein Authentifizierungsframework (z. B. OAuth 2.0 oder JWT) erforderlich — dies wird in Kapitel 7 eingeordnet.

4.3 Datenbankkonzeption

Die Persistenzschicht basiert auf PostgreSQL und umfasst in der aktuellen Fassung **18 Tabellen** (17 ORM-Modelle sowie eine schlüsselwertbasierte `settings`-Tabelle, die ausschließlich über eine Alembic-Migration verwaltet wird). Die Wahl eines relationalen Systems gegenüber dokumentenorientierten Ansätzen (z. B. MongoDB) begründet sich durch die stark strukturierten Domänendaten: FK-Beziehungen zwischen Teams, Wettbewerben, Spielen, Events und Ticker-Einträgen sind klar definiert und referenziell integer zu halten. JSONB-Felder werden selektiv für semistrukturierte Daten (z. B. Statistik-Rohdaten) eingesetzt, sodass die ACID-Garantien relationaler Datenbanken erhalten bleiben. Das Schema ist auf einen stabilen Live-Betrieb mit wiederholbaren Importen, klaren Zustandsübergängen und nachvollziehbaren Redaktionsentscheidungen ausgelegt.

4.3.1 Schemadesign-Prinzipien

Die folgende Abbildung zeigt die zentralen Entitäten und ihre Beziehungen:

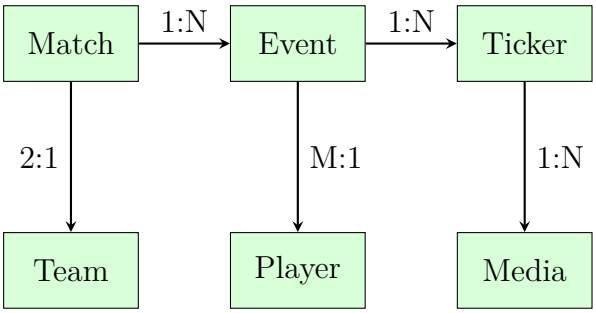

Abbildung 4.3: Vereinfachtes ER-Diagramm der zentralen Systemtabellen (konzeptionell)

> **Hinweis:** Die Tabellen `media_queue`, `media_clips`, `style_references` und `settings` sind als eigenständige Entitäten ohne Fremdschlüsselbeziehungen modelliert und daher im ER-Diagramm nicht abgebildet. `style_references` dient als Few-Shot-Datenquelle für die LLM-Promptgenerierung; `media_queue` wird applikationsseitig über die Media-Endpunkte verwaltet.

Das Schema folgt einem hybriden Entwurfsansatz aus strukturierter Normalisierung und gezielter Schemaflexibilität:

- **Normalisierte Kernentitäten:** Zentrale Domänenobjekte wie `teams`, `matches`, `events`, `ticker_entries`, `players`, `competitions` und Zuordnungstabellen sind in dritter Normalform modelliert (Codd 1970), sodass Beziehungen, Integrität und Auswertbarkeit im Live-Betrieb stabil bleiben.
- **Gezielter JSONB-Einsatz:** JSONB-Felder werden bewusst nur dort verwendet, wo sich Strukturen dynamisch ändern oder kontextabhängig sind (z. B. `synthetic_events.data` sowie Kontextfelder in `matches`). Die Statistiktabellen sind dagegen stark typisiert über explizite Spalten modelliert, um konsistente Abfragen und robuste Aggregationen zu ermöglichen.
- **Schlüsselstrategie:** Numerische Primärschlüssel (`id`) für performante Joins werden mit einem eindeutigen `uid`-Feld (UUID) als stabilem externen Referenzschlüssel kombiniert — kein reines UUID-Schema, sondern eine kombinierte Strategie.

4.3.2 Status-Lifecycle der Ticker-Einträge

Jeder Ticker-Eintrag in `ticker_entries` folgt einem klaren Statusmodell mit vier Zuständen: `draft` (Entwurf, noch nicht freigegeben), `published` (redaktionell oder automatisch veröffentlicht), `rejected` (verworfen, bleibt zur Nachvollziehbarkeit erhalten) und `deleted`

(weich gelöscht; wird in Listenabfragen herausgefiltert, bleibt aber in der Datenbank erhalten).

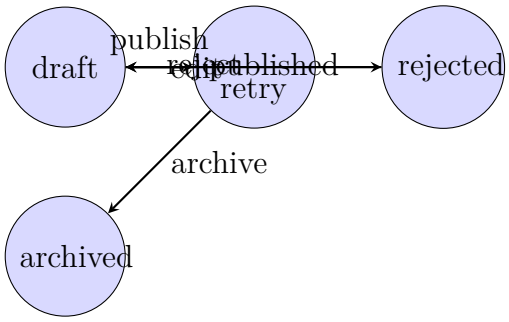

Abbildung 4.4: Zustandsautomat der Ticker-Einträge (draft, published, rejected, deleted)

Eine Rückstufung von `published` auf `draft` ist über `PATCH status=draft` möglich und wird insbesondere bei Re-Triggern von Matchphasen-Workflows genutzt (vgl. Kap. 5.5.3). `rejected` ist ein terminaler Zustand — ein verworfener Eintrag kann nicht reaktiviert werden. `deleted` kennzeichnet weich gelöschte Einträge: Sie bleiben in der Datenbank erhalten, werden aber durch ein explizites Filter-Prädikat (`status != deleted`) aus allen regulären Listenabfragen ausgeschlossen.

Ergänzend markiert das Feld `source` die Herkunft (`ai` vs. `manual`) und erlaubt eine saubere Trennung für Evaluationen (z. B. ausschließlich KI-generierte Einträge).

> **Hinweis für die Methodik**: Im aktuellen Schema ist kein separates `published_at`-Feld vorhanden. Persistiert wird `created_at` des jeweiligen Ticker-Eintrags. Für sekundengenaue Time-to-Publish-Analysen sind daher zusätzliche Messzeitpunkte im Evaluationsdatensatz bzw. in der Messlogik erforderlich, statt sich allein auf den Tabellenzustand zu stützen.

4.3.3 Die drei Betriebsmodi

Das Feld `ticker_mode` in `matches` steuert das Verhalten der Generierungspipeline pro Spiel und ist zur Laufzeit umschaltbar über `PATCH /api/v1/matches/{id}/ticker-mode`. Es sind drei Modi implementiert. Im Modus `auto` (vollautomatisch) läuft die Triggerkette ohne redaktionellen Eingriff; KI-Ergebnisse werden direkt veröffentlicht. Im Modus `coop` (kooperativ / Human-in-the-Loop) erzeugt die KI Entwürfe (`draft`), über deren Veröffentlichung oder Ablehnung die Redaktion entscheidet — dieser Modus bildet das primäre Zielbild des Systems. Im Modus `manual` (rein redaktionell) werden Einträge ausschließlich manuell erstellt; eingehende Events lösen nur die Datenpersistenz aus, nicht die LLM-Pipeline. Dieser Modus dient als Referenz für Vergleiche in der Evaluation.

Der Moduswechsel ist als Laufzeitparameter auf Datenbankebene abgebildet; der Statusfluss ist im State-Diagramm in Abschnitt 4.3.2 dargestellt.

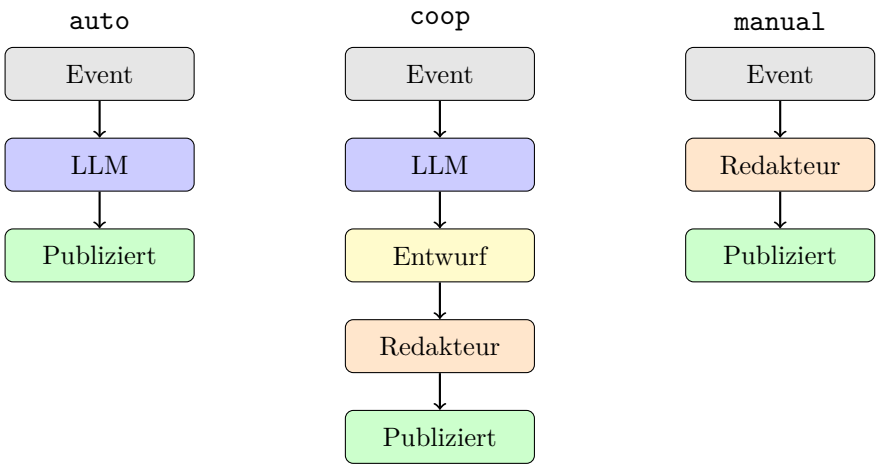

Abbildung 4.5: Die drei Betriebsmodi im Überblick: Automatisierungsgrad und Entscheidungshoheit je Modus

4.3.4 Integrität, Idempotenz und Auswertbarkeit

Für den Zusammenschluss aus n8n-Workflows, externen APIs und Backend-Routen ist die Datenbank auf idempotente Verarbeitung ausgelegt. Wiederholte Imports führen durch Unique-Constraints und Upsert-Strategien nicht zu unkontrollierten Duplikaten; eindeutige Schlüssel (z. B. `source_id` pro Ereignisquelle) sichern die Nachvollziehbarkeit importierter Ereignisse. Lösch- und Nullsetz-Strategien (`CASCADE`, `SET NULL`) halten abhängige Daten konsistent und bewahren fachlich relevante Historie. Die Kombination aus `ticker_mode`, `status`, `source`, Spielbezug und Zeitfeldern schafft die Grundlage für reproduzierbare Vergleichsauswertungen zwischen den Betriebsmodi.

4.4 Workflow-Konzeption (n8n)

Die Workflow-Schicht ist als entkoppelte Orchestrierungsebene zwischen externen Datenquellen und Backend ausgelegt. n8n wurde gegenüber Alternativen wie einem Custom-Scheduler (z. B. Celery, APScheduler) oder cron-basierten Skripten gewählt, weil es Workflow-Logik ohne Codeänderungen am Backend anpassbar macht, eine visuelle Debugging-Oberfläche bietet und HTTP-Trigger sowie Webhook-Empfang ohne zusätzliche Infrastruktur unterstützt. n8n übernimmt dabei API-Aufrufe, Transformation, Persistenzvorbereitung und Triggersteuerung, während das FastAPI-Backend als transaktionaler Kern und Integrationspunkt für Frontend und KI-Generierung fungiert.

4.4.1 Workflow-Klassen und Verantwortlichkeiten

Alle Generierungs-Workflows folgen einem konsistenten Vier-Phasen-Muster, das Idempotenz garantiert: Mehrfachaufrufe desselben Events erzeugen weder Datenduplikate noch red-

undante LLM-Aufrufe.

1. **Webhook-Trigger:** Ein HTTP-Aufruf löst den Workflow aus.
2. **SQL-Operation:** Ein Upsert-Statement persistiert die Daten mit Konfliktbehandlung (`ON CONFLICT`).
3. **Filter-Knoten:** Nur neue, noch nicht vorhandene Zeilen (erkennbar an der zurückgegebenen Datenbank-`id`) passieren weiter.
4. **Backend-Call:** Der entsprechende FastAPI-Endpunkt wird pro neuer Zeile aufgerufen; bei Generierungsworkflows dient er als LLM-Trigger.

Die n8n-Landschaft gliedert sich in sechs funktionale Klassen. Die folgende Übersicht zeigt alle 16 Workflows und ihren Auslöser:

Gruppe	Workflows	Trigger
A) Stammdaten	01–04: Countries, Teams, Competitions, Matches	Bei leeren Daten
B) Matchdaten	04–07: Lineups, Match-/Player-Statistics, Prematch	Nach Matchauswahl
C) KI-Generierung	09 Events-LLM, 13 Halftime/Aftertime, 14 Anpfiff/Abpfiff	Modus- und Event-gesteuert
D) Medien & Social	08 ScorePlay, 10 Twitter/X, 11 YouTube, 12 Instagram	Medien-Suche
E) Monitoring	15 Live-Stats-Monitor	Spieltag-gesteuert
F) Spielzeit	16 Aktuelle-Minute-Update	Webhook (Spielminute ändern)

Diese Trennung reduziert Kopplung, erleichtert Fehlersuche und erlaubt es, einzelne Teilprozesse unabhängig anzupassen. Die konkreten Workflow-Dateinamen und Implementierungsdetails sind in Kapitel 5.5 dokumentiert.

4.4.2 Workflow-Grenzen und projektspezifische Besonderheiten

Die Workflow-Landschaft ist funktional umfassend, enthält aber bewusst pragmatische Projektentscheidungen. In einzelnen Flows ist die Saisonlogik auf 2025 begrenzt (teilweise saisonspezifische Fixierung). Nicht alle Datenpfade sind eventgetrieben — ein Teil wird durch Polling und bedarfsgesteuerte Trigger ergänzt (hybride Aktualisierungslogik). Einige LLM- und Medienprozesse enthalten EF-spezifische Logik, die für eine vollständig generalisierte White-Label-Fähigkeit noch parametrisiert werden müsste (instanzspezifische Spezialpfade).

In Summe bildet n8n eine tragfähige Orchestrierungsschicht, die externe Datenquellen, interne Persistenz und KI-generierte Inhalte in einem modularen, nachvollziehbaren und erweiterbaren Ablauf zusammenführt.

4.5 KI-Komponente

Die KI-Komponente bildet das Herzstück des Systems: Sie übersetzt strukturierte Spielereignisse in natürlichsprachliche Ticker-Texte und verbindet dabei die LLM-Pipeline mit dem Betriebsmodus, der Stilkonfiguration und dem Mehrsprachigkeitsparameter. Dieser Abschnitt beschreibt die Multi-Provider-Architektur, die Prompt-Konzeption, die Concurrency-Steuerung sowie die Mehrsprachigkeitsstrategie.

4.5.1 Multi-Provider-Architektur

Die KI-Komponente ist als providerunabhängige Abstraktionsschicht implementiert. Der LLM-Dienst kapselt mehrere Anbieter hinter einer einheitlichen Schnittstelle und unterstützt aktuell OpenAI, Anthropic, Google Gemini, OpenRouter sowie einen Mock-Modus für Entwicklungs- und Fallbackszenarien ohne API-Key.

Das vorliegende System setzt primär **OpenRouter** als LLM-Gateway ein. OpenRouter fungiert als einheitliche API-Schnittstelle, über die mit einem einzelnen API-Schlüssel verschiedene Sprachmodelle unterschiedlicher Anbieter angesprochen werden können. Der Wechsel zwischen Modellen erfordert lediglich eine Änderung der Umgebungsvariable `OPENROUTER_MODEL`; verschiedene Modelle können auf demselben Prompt evaluiert werden (vgl. Abschnitt 6.3.3); und das jeweils kostengünstigste Modell lässt sich ohne Codeanpassung wählen — ein entscheidender Faktor für Vereinsredaktionen mit begrenztem Budget.

Im produktiven Einsatz wird aktuell Gemini 2.0 Flash Lite (Modell-ID `google/gemini-2.0-flash-l` über OpenRouter genutzt. Die Wahl dieses kompakten Modells begründet sich durch die Aufgabencharakteristik (vgl. Kap. 3.1): Liveticker-Texte sind kurz und faktenbasiert — eine Aufgabe, für die kompakte Modelle eine vergleichbare Qualität bei deutlich niedrigerer Latenz erzielen.

Ergänzend implementiert das Backend eine **Fallback-Kette** für den Fall, dass OpenRouter nicht konfiguriert ist. Das System prüft zur Startzeit in der festen Prioritätsreihenfolge, ob ein API-Schlüssel hinterlegt ist, und aktiviert den ersten verfügbaren Provider als Singleton:

`openrouter` → `gemini` → `openai` → `anthropic` → `mock`

Ist kein Schlüssel konfiguriert, wird auf einen regelbasierten **Mock-Provider** zurückgefallen, der Template-basierte Texte ohne LLM-Aufruf erzeugt. Diese Architektur stellt sicher,

dass das System auch ohne LLM-Anbindung lauffähig bleibt — etwa für Entwicklung und Tests.

Standardmodelle der Provider sind:

Provider	Standardmodell
OpenRouter	`google/gemini-2.0-flash-lite-001`
Gemini	`gemini-2.0-flash-lite-001`
OpenAI	`gpt-4o-mini`
Anthropic	`claude-haiku-4-5-20251001`

Zusätzlich kann die Auswahl in Generierungsrouten pro Request über `provider` und `model` überschrieben werden, was insbesondere für Evaluationsvergleiche zwischen Modellen genutzt wird (vgl. Abschnitt 6.3.3).

4.5.2 Prompt-Engineering-Strategie

Die Generierung verwendet ein template-basiertes Prompting mit sechs modularen Bausteinen, die dynamisch zusammengesetzt werden: einer Rolleninstruktion (Persona, Aufgabe, Gattung), einer Stilinstruktion (neutral / euphorisch / kritisch), einem Faktenblock zum Ereignis (Typ, Detail, Minute, beteiligte Spieler und Team), einem dynamischen Kontextblock mit relevanten Matchinformationen, einem optionalen Few-Shot-Block mit Stilbeispielen sowie einem Regelblock mit formativen und inhaltlichen Einschränkungen.

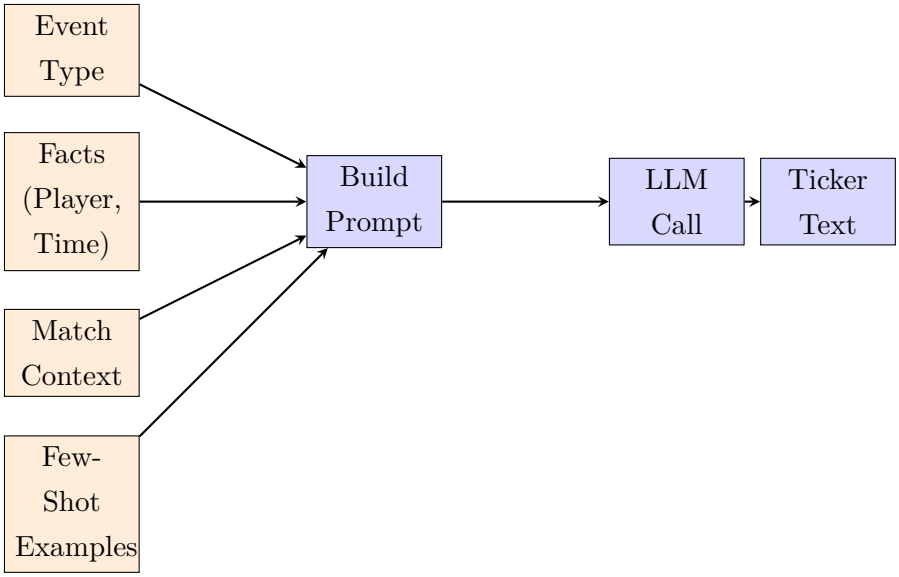

Abbildung 4.6: Prompt-Aufbau: Eingaben, System-Prompt, LLM-Aufruf und Ticker-Text

Der Few-Shot-Block wird aus der Tabelle `style_references` gespeist, die manuell kuratierte Original-Tickertexte von Eintracht Frankfurt enthält. Pro LLM-Aufruf werden bis

zu drei zufällige Referenzen selektiert, gefiltert nach `event_type`, `instance` und optional `league`. Durch die Randomisierung wird eine monotone Reproduktion vermieden, während der stilistische Korridor gewahrt bleibt. Für Pre-Match-Typen enthält der Prompt zusätzliche harte Restriktionen, um Live-Szenen-Halluzinationen zu vermeiden und die Ausgabe auf Vorschau- und Analyseinhalte zu begrenzen.

Der Inferenzparameter Temperature wird für die Textgenerierung auf `0.3` festgelegt, um die faktische Korrektheit gegenüber kreativer Varianz zu priorisieren. Für Übersetzungsaufgaben wird ein noch niedrigerer Wert von `0.1` verwendet, um semantische Abweichungen vom Originaltext zu minimieren.

4.6 Frontend-Konzeption

Das Frontend realisiert die Präsentationsschicht des Systems als React-Anwendung mit TypeScript. Dieser Abschnitt beschreibt die Architekturprinzipien, die Hook- und Zustandsarchitektur, das Dreispalten-Redaktionslayout, die Moduslogik sowie die Kommunikationsmuster zwischen Frontend und Backend.

4.6.1 Architektur, Zustandsmanagement und Hooks

Das Frontend ist als React-Client mit TypeScript und klarer Trennung zwischen UI-Komponenten, datenbezogenen Hooks und API-Zugriffsschicht umgesetzt. Der zentrale Anwendungszustand wird überwiegend lokal bzw. feature-nah gehalten; gemeinsamer Zustand für tief verschachtelte Komponenten wird über drei dedizierte Contexts bereitgestellt (`TickerModeContext`, `TickerDataContext`, `TickerActionsContext`), um Prop-Drilling zu vermeiden und Re-Renders zu kontrollieren.

Die White-Label-Fähigkeit ist über eine Konfigurationsschicht (`config/whitelabel.ts`) realisiert (u. a. Farben, Team-Keyword, API-Base-URLs). Die Oberfläche unterstützt alle drei Betriebsmodi (`auto`, `coop`, `manual`) inklusive Laufzeitumschaltung über den `ModeSelector` und Synchronisierung mit dem Backend.

Die Zustands- und Interaktionslogik ist in wiederverwendbare Custom Hooks aufgeteilt — ein Entwurfsmuster, das React seit Version 16.8 als Alternative zu Klassenkomponenten eingeführt hat. Die funktionalen Kategorien umfassen: Navigation und Datenauswahl (lösen bei leerem Datenbestand bedarfsgesteuert n8n-Importtrigger aus), Datenabfrage und Polling (laden Match-, Event- und Tickerdaten periodisch über koordinierte Abfragezyklen), Echtzeitkommunikation (WebSocket-Verbindung für den Medien-Feed), Redaktionsinteraktion (Modusverwaltung, Draft-Bestätigung und -Ablehnung, Tastatursteuerung) sowie UI-Steuerung (Panel-Größenanpassung, Spielminuten-Berechnung, Autovervollständigung). Die konkreten Implementierungen sind in Kapitel 5.4.3–5.4.5 dokumentiert.

4.6.2 Layout, Moduslogik und Kommunikation

Die Hauptansicht folgt einem responsiven Dreispalten-Ansatz mit klarer Aufgabenverteilung. Das linke Panel übernimmt die Veröffentlichungsperspektive: Es listet publizierte Ticker-Einträge als chronologischen Feed, dedupliziert nach `event_id` und sortiert nach Spielminute. Das mittlere Panel bildet die redaktionelle Kernarbeit ab: Event-Verarbeitung in Abhängigkeit vom Modus, Draft-Prüfung und Freigabe/Ablehnung, manuelle Eingabe mit Command-Parser (`/g`, `/gelb`, `/rot`, `/s`) sowie integrierte Media- und Social-Panels, die ausschließlich bei erkannten EF-Spielen eingeblendet werden. Das rechte Panel liefert Kontext- und Analyseinformationen: Matchstatistiken, Aufstellungen sowie spielerbezogene Leistungsdaten.

Die Modusumschaltung (vgl. Kap. 4.3.3) erfolgt über einen dedizierten `ModeSelector` mit Portal-basiertem Bestätigungsdialog, visueller Toast-Rückmeldung (2200 ms) und Tastatur-Shortcuts (`Ctrl+1` / `Ctrl+2` / `Ctrl+3`). Im kooperativen Modus sind zusätzliche Tastatur-Interaktionen für den Accept-/Reject-Flow (`TAB` / `ESC`) eingebunden. Das Frontend setzt die hybride Triggerarchitektur mit REST-Polling (5-Sekunden-Intervall) für Kerndaten, Webhook-Trigger für bedarfsgesteuerte n8n-Importe sowie WebSocket (`/ws/media`) mit Exponential-Backoff-Reconnect für latenzkritische Medieninhalte um. Die konkreten Hook-Implementierungen sind in Kapitel 5.4.3–5.4.5 dokumentiert.

4.7 Skalierbarkeit, Erweiterbarkeit und Systemgrenzen

Dieser Abschnitt bewertet das System hinsichtlich seiner Fähigkeit zur horizontalen Skalierung, identifiziert bekannte Grenzen der aktuellen Architektur und skizziert die Erweiterbarkeit für zukünftige Anforderungen.

4.7.1 Skalierung, Erweiterbarkeit und Deployment

Die Anwendungsschicht ist zustandslos konzipiert und kann horizontal skaliert werden, indem zusätzliche Uvicorn-Prozesse hinter einem Load-Balancer gestartet werden. Der gemeinsame PostgreSQL-Connection-Pool (`QueuePool` mit `pool_size=20`, `max_overflow=30`) stellt sicher, dass mehrere Backend-Instanzen dieselbe Datenbankverbindungskapazität teilen. Der in-Memory-`MediaConnectionManager` für WebSocket-Verbindungen funktioniert hingegen nicht über Prozessgrenzen hinweg und müsste bei Multi-Prozess-Deployment durch ein verteiltes Pub/Sub-System (z. B. Redis) ergänzt werden.

Neue Ereignistypen können ohne Datenbankschema-Änderungen ergänzt werden: Die `events`-Tabelle verfügt über ein generisches `description`-Feld, und die `synthetic_events`-Tabelle nutzt JSONB für vollständig flexible Payloads. Die Übersetzung von Football-API-Codes auf interne Typbezeichnungen ist in einer zentralen `EVENT_TYPE_MAP`-Konfiguration

hinterlegt; neue Typen erfordern ausschließlich einen zusätzlichen Eintrag dort sowie in `getEventMeta()` im Frontend. Der LLM-Dienst lässt sich um neue Anbieter erweitern, indem eine providerabhängige Handler-Methode implementiert und im zentralen dispatch-Dictionary registriert wird, ohne bestehenden Code zu berühren.

4.7.2 Deployment-Architektur

Das System ist auf der Cloud-Plattform Render deployt und gliedert sich in drei unabhängige Deployment-Einheiten: FastAPI-Backend (Docker-Container), React-Frontend (Static Site) und PostgreSQL (Managed Database). n8n wird für den Evaluationsbetrieb lokal betrieben und ist über einen ngrok-Tunnel für externe Webhooks erreichbar. Die vollständigen Deployment-Details, Konfiguration und Free-Tier-Implikationen werden in Abschnitt 5.6 beschrieben.

4.8 Fazit der Systemkonzeption

Die Systemkonzeption legt fünf interdependente Designpfeiler fest, die gemeinsam die Produktionsfähigkeit des Systems begründen. Die dreischichtige Architektur (Kap. 4.1) schafft die strukturelle Basis für eine unabhängige Weiterentwicklung der Automatisierungs-, Anwendungs- und Präsentationsschicht. Das relationale Datenbankschema mit seinem definierten Ticker-Lifecycle (Kap. 4.3) sichert referenzielle Integrität und Nachvollziehbarkeit aller redaktionellen Entscheidungen. Die n8n-Workflow-Schicht (Kap. 4.4) entkoppelt die externe Datenbeschaffung und Orchestrierungslogik vom Anwendungskern und erlaubt eine schnelle Anpassung von Importprozessen ohne Eingriff in das Backend. Die Multi-Provider-LLM-Architektur mit Few-Shot-Prompting und instanzspezifischen Stilprofilen (Kap. 4.5) maximiert Textqualität und Anbieterunabhängigkeit bei minimaler Infrastrukturkomplexität. Das Context-basierte Frontend-Design in Kombination mit dem `ticker_mode`-Feld (Kap. 4.3.3) ermöglicht redaktionelle Kontrolle ohne Latenzeinbußen und ohne Prop-Drilling über Komponentengrenzen.

Das vorliegende Konzept ist damit als vollständige Spezifikation für die Implementierung formuliert, die Kapitel 5 dokumentiert; die systematische Evaluation folgt in Kapitel 6.

Kapitel 5

Implementierung

Dieses Kapitel dokumentiert die konkrete Umsetzung des in Kapitel 4 konzipierten Systems. Im Fokus stehen die technischen Entscheidungen auf Code-Ebene: die interne Struktur des FastAPI-Backends mit seinen Schichten aus Modellen, Repositories, Services und Routern; die KI-Generierungspipeline von der Ereigniserkennung bis zum publizierten Ticker-Eintrag; die Frontend-Implementierung mit React, Context-basiertem Zustandsmanagement und dem slash-command-getriebenen Eingabesystem; sowie die n8n-Workflow-Implementierung der zentralen Automatisierungsworkflows. Ergänzend werden die TypeScript-Migration und die Testabdeckung als Qualitätsmerkmale des Entwicklungsprozesses dokumentiert.

Die Implementierung folgt dem Grundsatz „so einfach wie nötig, so modular wie sinnvoll: Standardpfade (CRUD, Listabfragen) sind bewusst direkt gehalten (Router → Repository), während die LLM-Generierung mit ihren Retry-, Semaphore- und Few-Shot-Mechanismen in einen dedizierten Service extrahiert ist. Die n8n-Workflows (Abschnitt 5.5) werden nach der Frontend-Implementierung beschrieben, da sie auf Backend-Endpunkten aufbauen, die bis dahin vollständig dokumentiert sind.

5.1 Backend-Implementierung

5.1.1 Projektstruktur und Application Entry Point

Das Backend ist unterhalb von `backend/app/` in sieben funktionale Pakete gegliedert:

```
backend/app/
|---- api/v1/          # HTTP-Router (FastAPI)
|---- models/          # SQLAlchemy ORM-Modelle (17 Dateien)
|---- repositories/    # Datenbankzugriff (je Entität eine Klasse)
|---- schemas/         # Pydantic-Schemas (Request / Response)
|---- services/        # Fachliche Services (llm_service, ticker_service)
|---- utils/           # Hilfsmodule (llm_context_builders, evaluation_metrics)
```

```
`---- core/            # Konfiguration, Datenbankverbindung, Enums, Konstanten
```

Diese Aufteilung entspricht dem klassischen Repository-Service-Router-Muster: Router delegieren an Repositories für einfache CRUD-Operationen und an Services für Logik, die mehrere Repositories oder externe Aufrufe koordiniert.

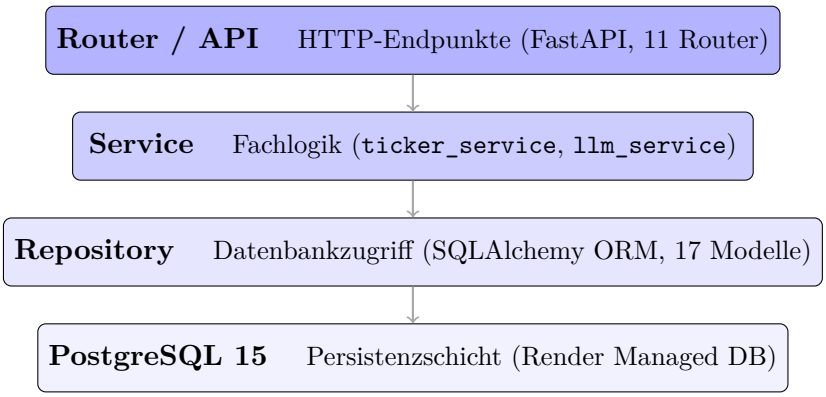

Abbildung 5.1: Backend-Schichtenarchitektur: Router → Service → Repository → DB

`app/main.py` ist der einzige Einstiegspunkt. Er registriert elf Router unter dem Pfad-Prefix `/api/v1` sowie den WebSocket-Router `/ws/media` ohne API-Prefix, konfiguriert CORS-Middleware und bindet statische Dateien (Thumbnail-Cache) ein. Beim Start führt `Base.metadata.create_all(bind=engine)` ein idempotentes Schema-Bootstrapping durch, das bei erstmaliger Inbetriebnahme alle Tabellen anlegt; Schema-Änderungen im Produktionsbetrieb werden via Alembic-Migrationen gesteuert.

Die Anwendung stellt zwei Metaendpunkte bereit:

Endpunkt	Funktion
`GET /`	Statusantwort mit Version (`0.3.0`)
`GET /health`	Datenbankverbindungscheck (`healthy`/`degraded`)

5.1.2 Datenmodelle: Datenbankschema und API-Validierung

Das Schema umfasst 17 ORM-Modelle (18 Datenbanktabellen, davon eine via Migration). Die zentralen Domänenobjekte sind:

Modell	Tabelle	Kernfelder
`Match`	`matches`	`id`, `uid`, `match_state`, `match_phase`, `ticker_mode`
`Event`	`events`	`id`, `match_id`, `event_type`, `time`, `description`
`SyntheticEvent`	`synthetic_events`	`id`, `match_id`, `type`, `data` (JSONB)
`TickerEntry`	`ticker_entries`	→ siehe unten

Modell	Tabelle	Kernfelder
`StyleReference`	`style_references`	`event_type`, `instance`, `league`, `text` (Few-Shot)
`Country`, `Team`, `Competition`, `Lineup`, `MatchStatistic`, `PlayerStatistic`, `MediaQueue`, `MediaClip`	Stammdaten, Statistiken, Medien	Standardfelder; vollständige Felddefinition in `app/models/`

Das `TickerEntry`-Modell hat besondere Bedeutung im Lifecycle des Systems: Es verknüpft über `event_id` (FK auf `events`) und `synthetic_event_id` (FK auf `synthetic_events`) den generierten Text mit seinem Auslöser. Die Felder `status` (`draft/published/rejected`) und `source` (`ai/manual`) bilden den Ticker-Lifecycle ab (vgl. Kap. 4.3.2). Drei zusammengesetzte Indizes (`ix_ticker_match_status`, `ix_ticker_match_phase`, `ix_ticker_event_id`) optimieren die häufigsten Abfragen: publizierte Einträge je Spiel, Phase-Filter und den Deduplizierungs-Check per Event. Das Status-Enum wird als nicht-nativ gespeichert (`native_enum=False`), damit Migrationen ohne Enum-Typ-Änderungen in PostgreSQL auskommen.

Für jeden Router gibt es eigenständige Pydantic-Schemas für Create-, Update- und Response-Strukturen. Alle Schemas nutzen `alias_generator=to_camel` aus `pydantic.alias_generators`, damit JSON-Payloads in camelCase kommuniziert werden, während Python-intern snake_case gilt.

Felder, die Spielstandwerte aus der Partner-API widerspiegeln, erhalten explizite `serialization_alias`-Werte mit Partner-Konvention (`teamHomeScore`); Standard-Beziehungsfelder erhalten durch den globalen `alias_generator=to_camel` automatisch kanonisches camelCase. Für Ticker-Einträge steuert ein separates PATCH-Schema den Status, um versehentliche Massenänderungen zu vermeiden.

Jede Entität hat ein dediziertes Repository, das alle SQL-Operationen kapselt. Alle Repositories erben von `BaseRepository[T]`, die `exists()` generisch bereitstellt; jede Instanz erhält eine `Session` im Konstruktor. Diese Kapselung hält Router frei von ORM-Details und erlaubt Integrationstests direkt gegen echte Datenbanken — ohne Mock-Schichten. Das `MatchRepository` lädt Heim- und Auswärtsteam, Wettbewerb und Saison per `joinedload` in einer einzigen Abfrage vor, da die KI-Generierungsroute vollständige Kontext-Daten benötigt.

5.1.3 API-Router: Endpunktstruktur

Der Ticker-Bereich ist in drei Router aufgeteilt, die gemeinsam unter `/api/v1/ticker` eingehängt werden:

Router	Datei	Kernoperationen
`ticker_crud`	`ticker_crud.py`	GET list, GET single, PATCH status, DELETE, POST manual
`ticker_generate`	`ticker_generate.py`	POST generate/{event_id}, generate-synthetic, generate-batch
`ticker_batch`	`ticker_batch.py`	POST translate-batch, generate-match-phases

Diese Aufteilung verhindert, dass eine einzelne Routerdatei durch die Kombination aus CRUD-, Generierungs- und Batch-Endpunkten unübersichtlich wird.

Ein kapselndes Hilfsmuster zieht sich durch alle Router: `require_or_404(value, message)` — eine schmale Utility-Funktion in `utils/http_errors.py`, die `None`-Rückgaben aus Repository-Aufrufen direkt in eine `HTTPException(404)` überführt. Das eliminiert das repetitive `if obj is None: raise HTTPException(404, ...)` in Routern und stellt sicher, dass alle 404-Antworten einheitlich formuliert sind. Der Batch-Endpunkt `POST /api/v1/ticker/translate-batch/{match_id}` ruft für alle publizierten KI-Einträge eines Spiels den LLM-Service mit reduzierter Temperatur (`0.1`) auf, um sprachlich deterministische Übersetzungen zu erzeugen — eine gegenüber der Originalgenerierung bewusst andere Parametrisierung.

Die fünf systemkritischen Endpunkte des Ticker-Lifecycles sind:

Endpunkt	Funktion
`POST /api/v1/events`	Importiert ein Live-Ereignis (n8n-Webhook); löst die LLM-Pipeline aus
`POST /api/v1/ticker/generate/{event_id}`	Generiert einen KI-Ticker-Eintrag für ein konkretes Ereignis
`PATCH /api/v1/ticker/{id}/status`	Setzt Status (`draft`/`published`/`rejected`); steuert Coop-Freigabe
`POST /api/v1/ticker/manual`	Erstellt einen manuellen Eintrag ohne LLM-Aufruf
`POST /api/v1/match-status`	Aktualisiert Spielzustand und Phase; triggert Phasen-Events

Die API umfasst insgesamt über 70 Routen in vier funktionalen Bereichen: Stammdaten, Ticker-Lifecycle, Medien/Clips und Spielkontext. Die vollständige Endpunktliste ist über die automatisch generierte OpenAPI-Dokumentation unter `/api/docs` erreichbar.

5.1.4 Ticker-Service: Domänenlogik

Der `ticker_service` kapselt die Domänenlogik für KI-Einträge. Er koordiniert Datenbankabfragen, Kontextaufbau und den LLM-Aufruf, hält dabei aber die Schichten klar getrennt: `llm_service.py` hat keine Datenbankabhängigkeit.

Der Service umfasst vier zentrale Funktionen. `score_at_event()` berechnet den Spielstand zum Zeitpunkt eines Torereignisses: Da die Football-API den kumulativen Stand liefert, muss der Stand *vor* dem aktuellen Event rekonstruiert werden, wobei Eigentore gesondert behandelt werden. `build_match_context()` erstellt das Kontext-Dictionary für den LLM-Prompt aus Teamnamen, aktuellem Stand, Matchzustand, Spielminute und Liga.

`call_llm()` ist der zentrale, Semaphore-gesicherte LLM-Aufruf. Er lädt zunächst aus dem `StyleReferenceRepository` bis zu drei Stilbeispiele für `event_type + instance + league` als Few-Shot-Kontext und delegiert dann an `generate_ticker_text()`:

```
async with _llm_semaphore:   # asyncio.Semaphore(settings.LLM_CONCURRENCY)
    return await generate_ticker_text(
        event_type=event_type,
        style_references=style_references,  # aus DB geladen
        ...
    )
```

Die Semaphore ist auf Modulebene als Singleton definiert (`_llm_semaphore = asyncio.Semaphore(settings.LLM_CONCURRENCY)`, Standard: 8) und begrenzt gleichzeitige LLM-Aufrufe pro Prozessinstanz.

`make_ai_entry()` schließlich ist ein schlanker Builder, der aus den LLM-Ergebnissen ein `TickerEntryCreate`-Schema erzeugt, `source=äi"` setzt und den initialen Status abhängig vom Aufrufkontext festlegt (`draft` im Co-op-Modus, `published` im Auto-Modus).

5.1.5 LLM-Service: Prompt-Aufbau und Provider-Dispatch

Der `LLMService` ist eine single-class-Implementierung, die beim Initialisieren den passenden Client instanziiert und über ein Dispatch-Dictionary aufruft:

```
dispatch = {
    "mock":       self._generate_mock_text,
    "gemini":     self._generate_gemini_text,
    "openrouter": self._generate_openrouter_text,
```

```
    "openai":     self._generate_openai_text,
    "anthropic":  self._generate_anthropic_text,
}
return dispatch[self.provider](**kwargs)
```

Das Dispatch-Pattern erlaubt das einfache Hinzufügen neuer Provider: Implementierung einer Methode `_generate_X_text()` und ein Eintrag im Dictionary.

Vorab normalisiert `_normalize_event_type()` den Ereignistyp: Die `EVENT_TYPE_MAP` übersetzt externe Strings beider APIs (z. B. `PartnerGoal` bzw. `Goal`) auf einheitliche interne Bezeichner (`goal`); unbekannte Typen fallen auf `comment"` zurück.

Der Prompt wird in vier modularen Methoden aufgebaut. `_build_event_lines()` erzeugt die Fakten-Sektion; jeder Parameter wird nur dann eingebunden, wenn er vorhanden ist, und Ereignisbezeichnungen werden aus `EVENT_TYPE_LABEL` in lesbares Deutsch übersetzt (z. B. `"goal"` → `TTor"`). `_build_few_shot_block()` formatiert die Stilreferenzen als nummerierte Beispiele:

```
### STILREFERENZEN
Schreibe in exakt diesem Stil (Rhythmus, Wortwahl, Emotionalität):
- "Mustermann trifft mit einem satten Rechtsschuss - 1:0!"
- "Eintracht schlägt zu - der erste Treffer des Abends!"
```

Der vollständig zusammengesetzte System-Prompt folgt der in Abschnitt 4.5.2 beschriebenen modularen Struktur aus sechs Bausteinen (Rolleninstruktion, Stilinstruktion, Faktenblock, Match-Kontext, Few-Shot-Block, Regelblock). Für Pre-Match-Typen fügt `_build_prematch_parts()` eine zusätzliche harte Schutzregel ein, die das Modell auf Vorschau- und Analyseinhalte beschränkt und das Erfinden von Live-Szenen verhindert. Die vier `_build_*`-Methoden erzeugen die Bausteine unabhängig voneinander, sodass fehlende Informationen (z. B. kein Spieler bei Phasenereignissen) den Prompt nicht invalidieren, sondern den entsprechenden Block auslassen.

LLM-Parameter: `temperature=0.3` für konsistente Ausgaben, `max_tokens` aus `LLM_MAX_TOKENS` (konfigurierbar). Bei Rate-Limit-Fehlern werden bis zu `LLM_RETRY_ATTEMPTS` (3) Versuche mit linearem Backoff (`30 s / 60 s / 90 s`) unternommen; bei sonstigen transienten Fehlern greift exponentielles Backoff (`1 s / 2 s`).

5.2 KI-Generierungspipeline

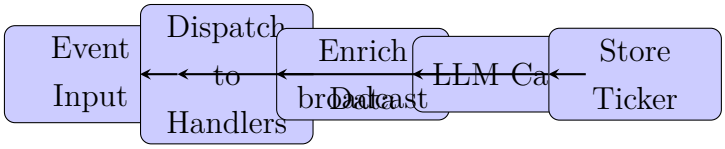

Abbildung 5.2: LLM-Pipeline: Event → Enrich → Prompt → LLM → Store

Die folgende Darstellung zeigt den vollständigen Ablauf von einem neuen Live-Ereignis bis zum publizierten Ticker-Eintrag als Sequenzdiagramm:

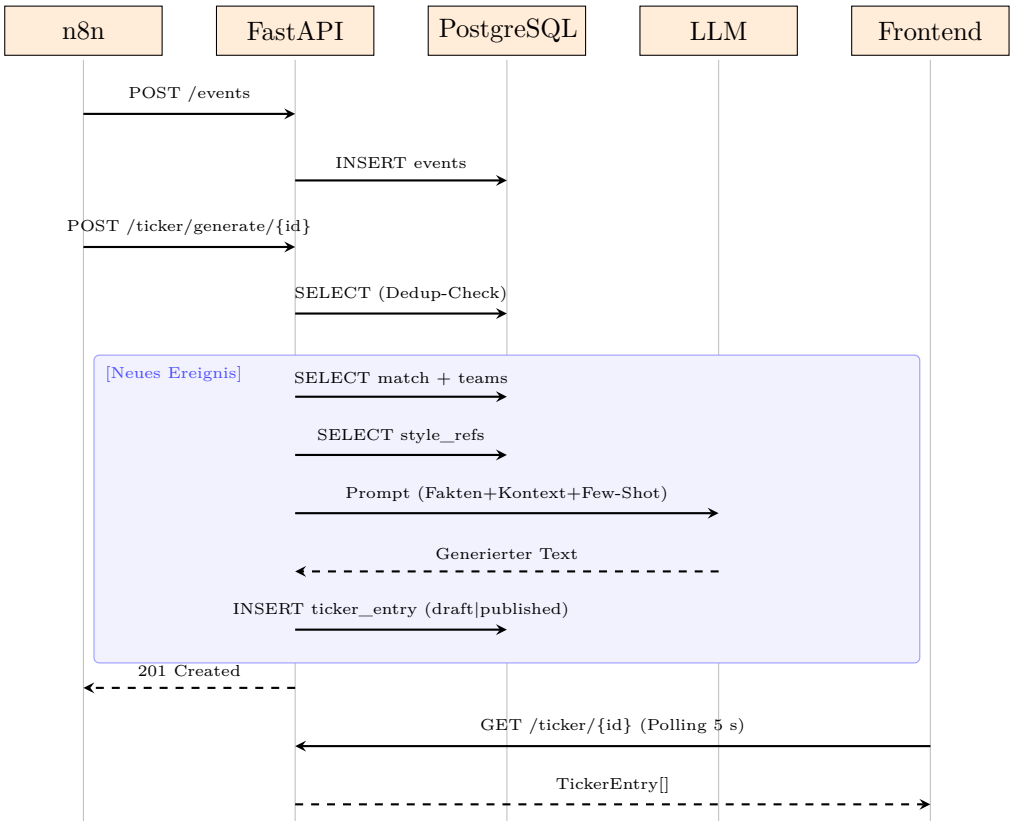

Abbildung 5.3: Ablauf der KI-Generierungspipeline: Ereignis-Import bis publizierter Ticker-Eintrag

Ein zentrales Qualitätsmerkmal der Pipeline ist die Deduplizierung: Der Generierungsendpunkt prüft vor jedem LLM-Aufruf, ob bereits ein Ticker-Eintrag für die `event_id` existiert:

```
existing = ticker_repo.get_by_event(event_id)
if existing:
    return existing  # Idempotenter Rückgabepfad ohne LLM-Aufruf
```

Dies macht den Endpunkt für Standardfälle idempotent gegenüber Mehrfachaufrufen — ein wichtiges Merkmal, da n8n-Workflows bei Fehlern automatisch wiederholen. Ohne diese Prüfung würde jeder Retry einen zusätzlichen Ticker-Eintrag mit unterschiedlichem (nicht-deterministischem) Text erzeugen. Wird ein abweichendes Stilprofil (`data.style`) übergeben, wird der bestehende Eintrag gelöscht und neu generiert — der idempotente Pfad gilt damit nur für Aufrufe mit identischem Stil.

Der initiale Status des erzeugten Ticker-Eintrags wird über den Aufrufparameter `auto_publish` gesteuert, den n8n aus dem `ticker_mode` des Spiels ableitet (vgl. Kap. 4.3.3 für die konzeptionelle Beschreibung der drei Betriebsmodi):

```
status=TickerStatus.published if data.auto_publish else TickerStatus.draft
```

Damit wird die Modus-Logik zu einem reinen Aufrufparameter des Backends — die Entscheidungsinstanz ist n8n auf Basis des in der Datenbank gespeicherten `ticker_mode`. Das Backend selbst ist modus-agnostisch: Es erhält einen Booleschen Wert und setzt den Status entsprechend.

Die drei Modi übersetzen sich auf Implementierungsebene in drei verschiedene Pfade:

Modus	`auto_publish`	Ergebnis	Redakteurs-Eingriff
`auto`	`True`	`published`	—
`coop`	`False`	`draft`	TAB (accept) / ESC (reject)
`manual`	—	—	Slash-Command → `POST /manual`

Im `manual`-Modus wird der Generierungs-Endpunkt gar nicht aufgerufen: n8n importiert zwar weiterhin Ereignisse, triggert aber keinen LLM-Aufruf. Ticker-Einträge entstehen ausschließlich über den `POST /api/v1/ticker/manual`-Endpunkt, den das Frontend aus dem Slash-Command-Parser heraus bedient. Der Eintrag wird dort direkt mit `status=published` angelegt, da die Redakteurin den Text selbst verfasst hat und keine weitere Freigabe erforderlich ist.

Im `coop`-Modus liegt die gesamte UX-Last auf der Draft-Queue: Das Frontend zeigt neue `draft`-Einträge mit `TAB`/`ESC`-Shortcuts an (vgl. Abschnitt 5.4.6), und die Entscheidungs-Latenz beträgt je nach Komplexität des Entwurfs 15–30 Sekunden. Dieses Design stellt sicher, dass keine KI-generierten Texte ohne menschliche Prüfung publiziert werden — eine Entscheidung, die direkt aus dem journalistischen Qualitätsanspruch in Kapitel 2.1 folgt.

Das konzeptionelle Statusmodell (`draft` → `published` / `rejected`, Undo-Übergang `published` → `draft`) ist vollständig in Abschnitt 4.3.2 beschrieben. Implementierungsseitig sind zwei Details hervorzuheben:

Der Übergang `published` → `draft` (Undo) ist im Frontend als **Toast-Aktion** realisiert: Nach jeder Publikation erscheint für 5 Sekunden ein Widerruf-Button. Über `PATCH /api/v1/ticker/{id}` mit `{ ßtatus: "draft" }` wird der Eintrag zurückgesetzt und steht erneut zur Bearbeitung bereit.

`rejected`-Einträge werden nicht aus der Datenbank gelöscht, sondern behalten ihren Status. Diese Entscheidung sichert die Reproduzierbarkeit der Evaluation: Alle je generierten Einträge — auch verworfene — sind für spätere Qualitätsanalysen auswertbar.

5.3 Frontend-Implementierung

5.3.1 Komponentenhierarchie

Das Frontend ist als React-Anwendung mit TypeScript organisiert. Die Hauptansicht (`LiveTicker`) gliedert sich in drei Panel-Komponenten, die von gemeinsam genutzten Hooks und Contexts gespeist werden:

```
App
`--- LiveTicker  [TickerModeContext | TickerDataContext | TickerActionsContext]
   |--- LeftPanel
   |  `--- PublishedEntry
   |--- CenterPanel
   |  |--- EventCard          --onGenerate----------  TickerActionsContext
   |  |--- EntryEditor        --onManualPublish------  TickerActionsContext
   |  |--- AIDraft            --onPublished/onDraft--  TickerActionsContext
   |  |--- SummarySection
   |  `--- MediaPickerPanel | YouTubePanel | TwitterPanel | InstagramPanel
   `--- RightPanel
      |--- Collapsible  (Statistiken, Aufstellung, Spieler)
      |--- FormationColumn
      `--- StatRow
```

Die drei Panels sind physisch getrennte React-Komponenten, kommunizieren aber ausschließlich über Context-Provider — kein Prop-Drilling über Komponentengrenzen hinweg.

5.3.2 React Context-Architektur

Das Frontend verwendet drei spezialisierte Contexts, die in `LiveTicker` zusammengeführt werden: `TickerModeContext` hält den Modus-State (`auto`/`coop`/`manual`) und die Keyboard-Shortcuts (TAB/ESC); `TickerDataContext` exponiert Match-Daten, Events und Ticker-Einträge mit zugehörigen Reload-Funktionen; `TickerActionsContext` kapselt Callbacks für redaktionelle Aktionen wie Generierung, Publikation und Bearbeitung.

Die Trennung von Daten- und Aktions-Context hat einen konkreten Performance-Vorteil: Komponenten, die nur Actions konsumieren (z. B. `EntryEditor`), re-rendern nicht bei Änderungen an `tickerTexts` — relevant bei einem Live-Ticker mit 5-Sekunden-Polling. Alle Contexts exportieren einen typsicheren Zugriffs-Hook, der bei Verwendung außerhalb des Providers einen klaren Fehler wirft.

5.3.3 Hook-Architektur

Die Zustandslogik ist in zwei Ebenen von Hooks organisiert: gemeinsame Hooks unter `src/hooks/` und feature-spezifische Hooks unter `components/LiveTicker/hooks/`.

Die wichtigsten Hooks und ihre Verantwortlichkeiten:

Hook	Paket	Verantwortlichkeit
`useNavigation`	hooks/	Land→Team→Wettbewerb→Spieltag→Spiel Navigationskette
`useMatchData`	hooks/	Aggregiert Polling-Hooks, koordiniert Reload-Zyklen, steuert Polling-Intervall
`useMatchCore`	hooks/	Polling der Match-Metadaten (Teams, Status, Spielstand, Minute)
`useMatchEvents`	hooks/	Polling der Spielereignisse (Tore, Karten, Auswechslungen)
`useTickerMode`	hooks/	`mode`-State, PATCH zum Backend, Keyboard-Shortcuts
`useMatchTicker`	LT/hooks/	Polling der Ticker-Einträge (5-Sekunden-Zyklus)
`useTicker`	LT/hooks/	Ticker-Actions, Draft-Queue, Publish-Toast (Undo)
`useEventDraft`	LT/hooks/	Entwurfsansicht, Accept/Reject-Logik
`useAutoPublisher`	LT/hooks/	Automatisches Publizieren im Auto-Modus

`useMatchCore`, `useMatchEvents` und `useMatchTicker` betreiben jeweils einen eigenen `setInterval`-Zyklus (5 Sekunden), der bei Mount startet und bei Unmount gestoppt wird. Die HTTP-Kommunikation erfolgt über `axios`. `useMatchData` aggregiert die Polling-Hooks und stellt die Daten über `TickerDataContext` bereit; das Polling-Intervall ist als Erweiterungspunkt für zustandsabhängige Frequenzen (Live vs. Prematch) konzipiert.

5.3.4 Slash-Command-Parser (`parseCommand.ts`)

Der Slash-Command-Parser ermöglicht schnelle manuelle Texteingabe ohne Maus-Interaktion. Er übersetzt kurze Kommandos in formatierte Ticker-Texte und liefert Metadaten (Icon, Phase, Minute) für die Publikation.

Die Funktion `parseCommand(input, currentMinute)` gibt ein typisiertes `ParseResult` zurück:

```
export interface ParseResult {
  type: string;
  formatted: string;
  warnings: string[];
  isValid: boolean;
  meta: {
    icon: string;
    phase: string | null;
    minute: number | null;
  };
}
```

Die Kommandos gliedern sich in zwei Typen:

- **Phasen-Commands** (11 Einträge in `PHASE_CMDS`) erzeugen vordefinierte Texte mit optionaler Minutenangabe. Die folgende Tabelle zeigt eine Auswahl (7 von 11):

Command	Ausgabe	Phase	Icon
`/prematch`	„Vorbericht"	`Before`	*
`/anpfiff`	„Anpfiff!"	`FirstHalf`	*
`/hz`	„Halbzeit!"	`FirstHalfBreak`	*
`/2hz`	„Anstoß zur 2. Halbzeit"	`SecondHalf`	*
`/vz1`	„Beginn der Verlängerung"	`ExtraFirstHalf`	*
`/elfmeter`	„Elfmeterschießen beginnt"	`PenaltyShootout`	*
`/abpfiff`	„Abpfiff!"	`After`	*

- **Event-Commands** (14 Einträge in `CMD_MAP`, 8 Typen nach Expansion) erzeugen strukturierte Texte mit Validierungshinweisen:

  `/g Mustermann EF` → `"TOR - Mustermann (EF)"` + icon `[Tor]`
  `/gelb Müller EF` → `"Gelb - Müller (EF)"` + icon `[Gelb]`
  `/rot Huber Bayern` → `"Rote Karte - Huber (Bayern)"` + icon `[Rot]`
  `/s EinRaus AusRein EF` → `"Wechsel - EinRaus <-> AusRein (EF)"` + icon `[Wechsel]`
  `/n Freistoß knapp über das Tor` → `"Freistoß knapp über das Tor"`

Für unvollständige Eingaben gibt `warnings[]` gezielte Hinweise zurück (`FFehlend: Spieler ein"` bzw. `FFehlend: Spieler aus"`), die im `EntryEditor` live angezeigt werden. Das Frontend kann damit ohne Backend-Aufruf Validierungsfeedback geben.

Nicht erkannte Commands werden mit `isValid: false` und einer Fehlermeldung zurückgegeben (`Ünbekannter Command: /xyz"`), statt einen Laufzeitfehler zu werfen.

5.3.5 Modusimplementierung im Frontend

Der aktive Modus (`auto` / `coop` / `manual`) steuert das Verhalten mehrerer Frontend-Komponenten (vgl. Kap. 4.6.4). `useTickerMode` initialisiert den Modus lokal mit `coop` als Standardwert, überschreibt diesen beim Laden eines Spiels mit dem in der Datenbank gespeicherten `ticker_mode` und schreibt Änderungen per `PATCH /matches/{id}/ticker-mode` sofort ans Backend zurück; Optimistic-Updates vermeiden wahrnehmbare Latenz beim Umschalten. Im Co-op-Modus registriert derselbe Hook die Shortcuts `TAB` (acceptDraft) und `ESC` (rejectDraft) als Keyboard-Event-Listener, die bei Moduswechsel automatisch aktualisiert werden. Im Auto-Modus übernimmt `useAutoPublisher` die automatische Publikation neu eintreffender Draft-Einträge als Frontend-Fallback für den Fall, dass n8n-Webhooks verzögert eintreffen oder ein Spiel direkt im Frontend auf Auto-Modus umgeschaltet wird, ohne n8n-Neustart.

5.3.6 TypeScript-Migration

Das Frontend wurde im Verlauf des Projekts von reinem JavaScript auf TypeScript migriert. Ziel war nicht die vollständige Auflösung aller `any`-Typen, sondern eine pragmatische Typisierung der systemkritischen Pfade mit messbaren Qualitätskennzahlen.

Die Migration folgte einem schrittweisen Ansatz: Zunächst wurden alle `.js`/`.jsx`-Dateien umbenannt und die `tsconfig.json` konfiguriert; anschließend wurden TypeScript-Fehler von innen nach außen behoben — beginnend mit dem Typ-System (`src/types/index.ts`) über Contexts bis zu den Komponenten. Der `strict`-Modus ist bewusst deaktiviert, um inkrementelle Migration ohne blockierende Fehlereskalation zu erlauben. `src/types/index.ts` definiert die zentralen Domain-Interfaces:

```
export type TickerMode = "auto" | "coop" | "manual";
export type TickerStyle = "neutral" | "euphorisch" | "kritisch";
export type MatchPhase = "Before" | "FirstHalf" | "FirstHalfBreak" | "SecondHalf"
↪  | ...;

export interface TickerEntry {
  id: number;
  match_id: number;
  event_id?: number | null;
  text: string;
  style?: string | null;
  status: "draft" | "published" | "rejected";
  source?: "ai" | "manual";
```

```
  minute?: number | null;
  phase?: string | null;
  icon?: string | null;
  image_url?: string | null;
  video_url?: string | null;
  llm_model?: string | null;
}
```

Nach der Migration erzielt das Projekt 0 TypeScript-Compilerfehler; die detaillierte Auswertung der type-coverage folgt in Abschnitt 6.2.5.

5.4 n8n Workflow-Implementierung

Die n8n-Workflows bilden die Orchestrierungsschicht zwischen externen Datenquellen, dem FastAPI-Backend und dem LLM-Dienst. Alle 17 Workflow-Dateien liegen als versionierte JSON-Exporte unter `n8n/` vor. Die Workflows gliedern sich funktional in sechs Gruppen: Workflows `01`–`04` laden Stammdaten (Länder, Teams, Wettbewerbe, Spielpläne) via Upsert aus API-Football — da zwei parallele `04`-Workflows existieren (`04_import_matches` und `04_import_lineups`), überlappt die Nummerierung an dieser Stelle. Workflows `04`–`07` importieren spielbezogene Detaildaten (Aufstellungen, Statistiken); der Prematch-Workflow (`07`) erzeugt zusätzlich strukturierte Kontextdaten als synthetische Events, die dem LLM als Promptkontext dienen. Die KI-Generierungs-Workflows (`09`, `13`, `14`) importieren Live-Ereignisse und delegieren die Textgenerierung an das Backend; Zusammenfassungen für Halbzeit und Abpfiff rufen den LLM-Provider direkt auf. Medien- und Social-Media-Workflows (`08`, `10`–`12`) dienen der Ingestion von Medieninhalten: Workflow `10` importiert Twitter/X-Posts, Workflow `11` YouTube-Clips und Workflow `12` Instagram-Beiträge, jeweils über plattformspezifische API-Abfragen und persistiert die URLs im Backend. Workflow `15` überwacht laufende Spiele und aktualisiert Live-Statistiken per Backend-Endpunkt; Workflow `16` empfängt minutengenaue Statusupdates, aktualisiert `minute`, `match_phase` und `match_state` und triggert bei Phasenwechsel automatisch den `match-status`-Webhook.

Die folgenden Abschnitte beschreiben die Implementierungsdetails der systemkritischen Workflows.

Events-LLM-Workflow (09) Dieser Workflow ist der kritischste im System. Er importiert Live-Ereignisse via Webhook (`POST /Events`) und persistiert sie per UPSERT (`ON CONFLICT (source_id) DO NOTHING`). Eine initiale SQL-Abfrage bestimmt anhand der Teamzugehörigkeit, ob die Instanz `ef_whitelabel` oder `generic` sowie den Stil; `auto_publish` wird direkt aus `ticker_mode` des Spiels gesetzt. Nur Events mit erfolgreicher

Datenbankzeile passieren den Filter-Knoten. Ein EF-spezifischer Zweig liest Spielerdaten von `profis.eintracht.de`, baut S3-Video-URLs auf und persistiert Torjubel-Videos als Ticker-Einträge.

Prematch-Import-Workflow (07) Dieser Workflow baut den Vorberichtskontext aus fünf parallelen Football-API-Abfragen auf (Verletzungen, Head-to-Head, Teamstatistiken Heim/Gast, Tabellenstand) und persistiert die Ergebnisse als `synthetic_events` mit idempotenter `ON CONFLICT (match_id, type) DO UPDATE`-Strategie. Ein LLM-Trigger wird nur ausgelöst, wenn noch kein nicht-verworfener Ticker-Eintrag für das jeweilige synthetische Event existiert (`WHERE te.id IS NULL`).

Matchphasen-Workflow (14) Dieser Workflow verarbeitet Spielzustands-Übergänge via Webhook (`POST /match-status`). Ein JavaScript-Knoten validiert Übergänge gegen eine Matrix erlaubter Vorzustände. Für Vollzeit-Ereignisse (`FT`, `AET`, `PEN`) generiert der Workflow die gesamte Phasensequenz rückwirkend; ein `FT`-Signal erzeugt vier synthetische Events. Das zentrale SQL-Statement kombiniert Match-Update, Event-Insert und eine `demote`-CTE in einer einzigen Transaktion, wobei Letztere bereits publizierte Phasen-Texte bei Re-Triggers auf `draft` zurückstuft.

Halbzeit/Abpfiff-Zusammenfassung (13) Dieser Workflow erzeugt narrative Zusammenfassungen und ruft OpenRouter direkt auf, da Zusammenfassungen einen umfangreicheren Prompt mit parallel geladenen Statistiken (Ballbesitz, Schüsse, Spieler-Ratings) erfordern und keinen Few-Shot-Stil aus der Datenbank benötigen. Das Ergebnis wird je nach `ticker_mode` als `published` oder `draft` gespeichert.

ScorePlay-Medien-Workflow (08) Dieser Workflow sucht Medien-Assets bei ScorePlay für Spieler eines Torereignisses. Die Spieler-Suche normalisiert Umlaute und matcht gegen vier Namensfelder. Gefundene URLs werden an `POST /api/v1/media/incoming` übertragen und vom Backend per WebSocket an verbundene Clients verteilt.

Über alle Workflows hinweg lassen sich fünf übergreifende Muster identifizieren:

- **UPSERT-Strategien:** Alle Insert-Operationen verwenden `ON CONFLICT DO UPDATE/DO NOTHING` für Idempotenz.
- **Filter-Knoten:** Vor jedem LLM-Trigger steht ein Filterknoten, der nur Rows mit einer `id` (erfolgreicher DB-Insert) durchlässt.
- **Statusbasierte Filterung:** Bestehende nicht-verworfene Ticker-Einträge verhindern redundante Generierungen.
- **Mode-Propagation:** `ticker_mode` wird als `auto_publish`-Flag an das Backend weitergegeben.

- **Instanz-Routing:** Die Unterscheidung zwischen `ef_whitelabel` und `generic` erfolgt per `ILIKE '%Frankfurt%'`-Abfrage.

5.5 Qualitätssicherung und Tests

Die Qualitätssicherung folgt dem Testpyramiden-Modell nach Cohn (2009). Die Testsuite umfasst 371 Tests (187 Frontend, 178 Backend, 6 E2E), die alle grün durchlaufen. Das Frontend setzt auf Jest mit `@testing-library/react`; besonders umfangreich ist der `parseCommand`-Parser mit 29 Testfällen, da er die kritische manuelle Eingabeschicht bildet. Das Backend nutzt pytest mit FastAPI `TestClient` und transaktionalem Rollback; API-Integrationstests prüfen Endpunkte gegen eine echte PostgreSQL-Testinstanz, Unit-Tests decken Services und Repositories ab. Sechs Playwright-E2E-Tests validieren grundlegende Browser-Interaktionen. Ergänzend enthält `evaluation_metrics.py` statistische Hilfsfunktionen (Cliff's Delta, Bootstrap-KI, Cohen's Kappa u. a.), die in Kapitel 6 direkt verwendet werden. Die detaillierte Auswertung der Testmetriken und Coverage-Verteilungen folgt in Abschnitt 6.2.

5.6 Deployment

Das System ist auf Render.com (Region: Oregon, US West) deployt. Das FastAPI-Backend läuft als Docker-Container (Free-Tier Web Service), die PostgreSQL-15-Datenbank als Managed-Dienst im privaten Render-Netzwerk; das React-Frontend wird als statische Seite über das Render-CDN ausgeliefert. Alle drei Dienste werden aus demselben Repository-Branch (`deploy/render-v2`) deployt; Konfiguration und Zugangsdaten werden als Umgebungsvariablen injiziert. Die n8n-Instanz läuft lokal und ist über einen `ngrok`-Tunnel für externe Webhooks erreichbar — für den Evaluationsbetrieb ausreichend; ein Produktivbetrieb würde eine persistente n8n-Instanz erfordern.

Die vier Komponenten bilden die drei Systemschichten aus Abschnitt 4.1: n8n die Datenbeschaffung, Backend und Datenbank die Anwendungs- und Persistenzlogik, das Frontend die Präsentation. Kapitel 6 dokumentiert die systematische Evaluation des so realisierten Systems.

Kapitel 6

Evaluation

6.1 Zielsetzung der Evaluation

Das Ziel der Evaluation ist es, das implementierte System auf mehreren Ebenen nachweisbar zu bewerten. Die Evaluation gliedert sich in drei Blöcke: Die **technische Qualitätssicherung** (Abschnitt 6.2) prüft die Korrektheit der Kernlogik über Unit-Tests, das Zusammenspiel der API-Schichten über Integrations-Tests, die Stabilität der Benutzeroberfläche über End-to-End-Tests sowie die statische Typsicherheit über TypeScript-Coverage. Die **Evaluation der KI-Textgenerierung** (Abschnitte 6.3–6.4) umfasst quantitative Metriken, eine qualitative Analyse der generierten Texte und den Vergleich der Betriebsmodi. Die **Systembetrachtung** (Abschnitte 6.5–6.7) behandelt Performance und Laufzeitverhalten, den Anforderungsabgleich (Soll-Ist) sowie Limitationen und offene Punkte. Die Metriken dienen nicht nur der Qualitätssicherung, sondern dokumentieren auch die Reife der Codebasis für einen produktiven Einsatz und identifizieren gleichzeitig die Grenzen des aktuellen Systems.

6.2 Technische Qualitätssicherung

Die technische Qualitätssicherung umfasst die vollständige Testsuite (Frontend, Backend, End-to-End) sowie die statische Typsicherheitsanalyse.

6.2.1 Teststrategie und Testpyramide

Die Teststrategie folgt dem klassischen Pyramiden-Modell nach Cohn (2009): Eine breite Basis schneller, isolierter Unit-Tests wird durch eine mittlere Schicht von Integrations-Tests ergänzt, die reale HTTP-Endpunkte gegen eine transaktionale Testdatenbank prüfen. An der Spitze stehen Playwright-basierte End-to-End-Tests, die den vollständigen Redaktionsworkflow im Browser simulieren.

```
 |  E2E (6) |   Playwright - Browser-Workflow
 ----------
|  API-Tests  |  FastAPI TestClient + PostgreSQL
------------
|  Unit-Tests   | Jest/React Testing Library + pytest
`----------------
```

Diese Pyramide stellt sicher, dass die Mehrheit der Testfälle ohne externe Abhängigkeiten läuft und damit in CI-Pipelines schnell und zuverlässig ausgeführt werden kann. Die Integrations-Tests sind so strukturiert, dass sie bei fehlender Datenbankverbindung automatisch übersprungen werden (`pytest.skip`) und keine falschen Fehlermeldungen erzeugen.

6.2.2 Frontend-Unit-Tests (Jest + React Testing Library)

Das Frontend verfügt über 168 Unit-Tests in 15 Testdateien, aufgeteilt in drei Kategorien:

Kategorie	Tests	Schwerpunkte
Utilities	51	`parseCommand` (29), `roundLabel` (16), `resolvePollingInterval` (6)
Hooks	55	`useLiveMinute` (10), `useSearchableDropdown` (10), `useListKeyboard` (9), …
Komponenten	62	`EntryEditor` (19), `RightPanelComponents` (18), `PublishedEntry` (14), …
Gesamt	**168**	

Der `parseCommand`-Utility ist die komplexeste Einheit der Geschäftslogik im Frontend; er transformiert Slash-Commands (z. B. `/g Müller SGE`) in strukturierte Ticker-Einträge. Die 29 Tests decken alle Command-Typen sowie Fehlerfälle ab: Jeder Test prüft `isValid`, `type`, `formatted`, `meta.icon` und `warnings` — unvollständige Eingaben liefern gezielte Hinweise (`FFehlend: Spieler"`) statt Laufzeitfehler.

6.2.3 Backend-Unit-Tests (pytest)

Das Backend verfügt über 178 Tests in 11 Dateien (82 API-Integrationstests, 49 Service/Repository-Tests, 47 Utility-Tests) mit einer Gesamt-Coverage von 75 %. Die API-Tests nutzen FastAPIs `TestClient` mit einer transaktionalen PostgreSQL-Session: Der `db`-Fixture erzeugt eine Session mit `autocommit=False` und führt nach dem `yield` ein Rollback aus; der `client`-Fixture überschreibt die `get_db`-Dependency von FastAPI, sodass alle Route-Handler dieselbe isolierte Session verwenden.

Die 75 %-Gesamtcoverage verteilt sich bewusst priorisiert über die Systemschichten:

Schicht	Dateien	Ø Coverage	Anmerkung
Modelle (`app/models/`)	17	100 %	Vollständig abgedeckt
Schemas (`app/schemas/`)	11	~95 %	Pydantic-Validierung vollständig getestet
Utilities (`app/utils/`)	5	~80 %	Evaluation-Metriken, HTTP-Fehler, Context-Builder
API-Endpunkte (`app/api/v1/`)	14	~55 %	Kern-Routes getestet; Clip/Media/Teams noch offen
Repositories	12	~55 %	Ticker/Event/Match gut; Clip/Season/Style offen
Services	2	~50 %	LLM-Service (Mocking), Ticker-Service

Die unbelegten Bereiche konzentrieren sich auf Nebenroutes (Clip-Import, Medienverwaltung, Saisonverwaltung), die außerhalb des kritischen Redaktionspfads liegen.

6.2.4 Browser-Smoke-Tests (Playwright)

Die 6 Playwright-Tests validieren den Browser-seitigen Workflow der Anwendung. Sie simulieren reale Nutzerinteraktionen im Chromium-Browser und sind so konzipiert, dass sie auch ohne laufendes Backend sinnvoll laufen: Backend-bedingte Abweichungen (z. B. leere Länder-Liste) werden als erwarteter Zustand behandelt, nicht als Fehler.

Test	Beschreibung
`App lädt korrekt > Startseite zeigt Navigations-UI`	Prüft, dass die React-App ohne Crash-Screen rendert
`App lädt korrekt > StartScreen zeigt Länder-Dropdown`	Prüft, dass `.lt`-Root-Element sichtbar wird
`Navigation > Länder-Dropdown ist interaktiv`	Prüft, dass Label und Select-Element im DOM erreichbar sind
`Navigation > Kein unbehandelter JS-Fehler beim Laden`	Fängt JavaScript-Laufzeitfehler ab (mit Whitelist für bekannte Netzwerkfehler)
`Ticker-Oberfläche > Keyboard-Shortcuts-Modal öffnet sich`	Simuliert Klick auf `?`-Button, prüft Modal-Öffnung und Overlay-Schließen
`Ticker-Oberfläche > Mobile Tab Bar ist auf kleinen Screens sichtbar`	Prüft Viewport 375×812 (iPhone SE) — App rendert korrekt

Die E2E-Tests decken absichtlich den *stabilen Kern* der UI ab — das initiale Rendern, die Fehlerfreiheit beim Laden und grundlegende Interaktionspunkte. Sie dienen primär als

Regression-Schutz: Bricht eine zentrale Komponente (z. B. durch einen fehlerhaften Import), schlagen die Tests unmittelbar an.

6.2.5 TypeScript-Typsicherheit

Die vollständige Migration von JavaScript zu TypeScript und die zentrale Typen-Architektur (`src/types/index.ts` mit Domain-Interfaces, Union-Types und `as const`-Konstanten) sind in Abschnitt 5.4.6 dokumentiert.

Die `type-coverage`-Metrik (Anteil explizit getypter Ausdrücke an allen Ausdrücken) wurde wie folgt gesteigert:

Zeitpunkt	type-coverage	TypeScript-Fehler
Migrationsbeginn	78,33 %	885
Nach Grundmigration	~84 %	0
Nach Interface-Ausbau (Hooks, Panels)	~89 %	0
Endstand (nach `parseCommand`-Typisierung)	**97,44 %**	**0**

Der Wert von 97,44 % bedeutet: Von 18.813 gemessenen Typausdrücken sind 18.344 explizit getypt. Die verbleibenden ~2,6 % verteilen sich auf wenige, bewusst offen gelassene Stellen. `MatchSelectorModal.tsx` ist ein großes Modal mit komplexem internen State, dessen Typisierung stark von einer kommenden API-Stabilisierung abhängt. Die Media-Panels `MediaPickerPanel.tsx` und `YouTubePanel.tsx` verarbeiten externe API-Responses (ScorePlay, YouTube), deren Typen nicht vollständig spezifiziert sind. Die Barrel-Exports in `components/index.ts` (`export { X }`) fehlen explizite Typ-Annotationen — sie sind technisch korrekt, werden von `type-coverage` jedoch als untypiert gezählt.

Nutzen im Entwicklungsprozess: Die TypeScript-Migration hat im Verlauf der Implementierung mehrere Fehler früh aufgedeckt, die sonst erst zur Laufzeit sichtbar geworden wären. So war `Country` zunächst als Objekt-Interface typisiert, obwohl die API `string[]` liefert — TypeScript erzwang die Korrektur. `parseInt(v)` erhielt ein `string | number`-Argument, was ohne Typisierung still übergangen worden wäre. Und `e.target.closest(...)` erforderte `as Element | null`, um einen Null-Zugriff zur Laufzeit zu verhindern.

6.3 Evaluation der KI-Textgenerierung

Die Evaluation der Textgenerierung gliedert sich in zwei Teile: zunächst die Evaluationsinfrastruktur mit quantitativen Metriken und Provider-Vergleich, dann die qualitative Analyse konkreter Textbeispiele einschließlich Expertenvalidierung.

6.3.1 Quantitative Metriken und Evaluationsinfrastruktur

Das System stellt über den Endpunkt `POST /api/v1/ticker/generate-bulk/{match_id}` eine Bulk-Generierungsfunktion bereit, die alle Events eines Spiels mit einem wählbaren Provider und Modell generiert. Durch optionale `provider`- und `model`-Parameter im Request-Body können verschiedene LLM-Konfigurationen systematisch verglichen werden, ohne den Produktivbetrieb zu beeinflussen.

```
class GenerateEventRequest(BaseModel):
    provider: Optional[str] = Field(default=None, description="Provider override
↪   for Evaluation")
    model:    Optional[str] = Field(default=None, description="Modell override
↪   for Evaluation")
```

Für jeden Aufruf wird eine temporäre `LLMService`-Instanz erzeugt, falls Provider oder Modell vom konfigurierten Singleton abweichen. Dadurch lassen sich A/B-Vergleiche zwischen Providern durchführen, ohne die Global-Konfiguration zu verändern.

Das Evaluationsmodul (`app/utils/evaluation_metrics.py`) stellt sechs Metriken bereit:

- **Time-to-Publish (TTP):** Latenz zwischen Ereigniszeitpunkt und Veröffentlichung in Sekunden — zentral, da Aktualität ein primäres Qualitätsmerkmal des Liveticker-Genres darstellt.
- **Cliff's Delta** (Cliff 1993): Nicht-parametrische Effektstärke für ordinale Daten; berechnet den Anteil konkordanter vs. diskordanter Paare (Wertebereich $[-1, 1]$, wobei $|\delta| < 0{,}147$ als vernachlässigbar, $|\delta| < 0{,}33$ als klein und $|\delta| > 0{,}474$ als groß gilt).
- **Bootstrap-Konfidenzintervall:** Resampling-basierte Schätzung des Mittelwertunterschieds zwischen zwei Gruppen (5.000 Iterationen, 95 %, deterministisch via Seed; Rückgabe: `(observed_diff, lower_ci, upper_ci)`).
- **Cohen's Kappa** (Cohen 1960): Interrater-Reliabilitätsmaß für kategoriale Bewertungen ($\kappa > 0{,}61$ gilt als substantielle Übereinstimmung).
- **Verteilungsanalyse** (`summarize_distribution`): Berechnet Median, Standardabweichung, Quartile, IQR und 95. Perzentil.
- **Qualitätsaggregation** (`aggregate_quality_by_group`): Gruppiert Bewertungsergebnisse nach Schlüssel (z. B. Modus oder Modell) und berechnet je Dimension Mittelwert und Gesamtscore.

Alle sechs Funktionen sind durch 17 Unit-Tests abgesichert (~96 % Coverage).

6.3.2 Modell- und Provider-Evaluation

Das System unterstützt fünf LLM-Provider in einer festen Prioritätskette, die beim Serverstart den ersten Provider mit gültigem API-Key als Singleton aktiviert:

Priorität	Provider	Standard-Modell	Temperatur
1	OpenRouter	`google/gemini-2.0-flash-lite-001`	0,3
2	Gemini	`gemini-2.0-flash-lite-001`	0,3
3	OpenAI	`gpt-4o-mini`	0,3
4	Anthropic	`claude-haiku-4-5-20251001`	0,3
5	Mock	— (regelbasierte Templates)	—

Im produktiven Render-Deployment waren nur OpenRouter (Priorität 1) und Mock (Fallback) aktiv; die übrigen Provider wurden mangels aktivierter API-Keys nicht evaluiert, sind jedoch architektonisch vollständig implementiert. Die qualitative Bewertung (N = 16, OpenRouter; N = 7, Mock) zeigt einen klaren Qualitätsvorsprung des LLM-Providers gegenüber dem Mock in der Tonalitätstreue (OpenRouter: 4,1 / 5 vs. Mock: 3,0 / 5; Δ = 1,1 Skalenpunkte); Korrektheit und Verständlichkeit liegen bei 4,6 bzw. 4,3 (OpenRouter). Die Latenzanalyse findet sich in Abschnitt 6.5.1.

6.3.3 Einfluss der Stilprofile

Das System generiert Texte in drei Stilprofilen, die über den Prompt gesteuert werden: `neutral` (sachlich und neutral, keine Vereinspräferenz), `euphorisch` (begeistert und emotional aus Sicht der Heimfans) sowie `kritisch` (analytisch und kritisch). Da der Backend-Deduplizierungsmechanismus für identische `event_id` denselben Eintrag zurückgibt, wurde der Stilvergleich anhand typischer Treffer-Einträge (Event-Typ: Tor) aus unterschiedlichen Spielen durchgeführt — die jeweils zugehörige Stilanweisung war identisch konfiguriert.

Event-Typ: Tor (Bundesliga-Saison 2024/25, Eintracht Frankfurt)

Stil	Generierter Text
neutral	„22. Minute: TOOOOR! C. Y. Uzun bringt Eintracht Frankfurt in Führung! Nach Vorlage von H. Larsson. 1:0!" *(Eintracht Frankfurt vs. Werder Bremen, Md. 7)*
euphorisch	„81. Minute: WAS IST DENN HIER LOS?! Collins mit der butterweichen Flanke! DOAN! Der Ball ist drin! TOOOOR! 1:0! Die Hütte bebt!" *(Eintracht Frankfurt vs. FSV Mainz 05, Md. 9)*
kritisch	„4. Minute: TOOOOR! Kaminski bringt Köln in Führung! Ache mit der Vorlage, Kaminski vollendet eiskalt. 1:0!" *(1. FC Köln vs. Eintracht Frankfurt, Md. 11)*

Ergänzender Direktvergleich: identisches Event in allen drei Stilen (synthetischer Kontext: Eintracht Frankfurt vs. FC Bayern München, 78. Minute, Tor Omar Marmoush):

Stil	Generierter Text
euphorisch	„78. Minute: MARMOOOSH! Der Ball zappelt im Netz! Unglaublich! Die Hütte explodiert! Was für ein Strahl! Eintracht führt! TOOOOR!”
neutral	„78. Minute: TOOOOR! Omar Marmoush bringt Eintracht Frankfurt in Führung!”
kritisch	„78. Minute: TOOOOR! Marmoush mit dem Ausgleich für die Eintracht! Die Bayern-Abwehr pennt. Frankfurt nutzt die Chance eiskalt.”

Die drei Stilprofile unterscheiden sich deutlich in Ausrufezeichen-Dichte, Wortwahl und Perspektive. Während `neutral` Fakten kompakt zusammenfasst, erzeugt `euphorisch` narrative Intensität durch Wiederholungen und Ausrufe. `kritisch` nähert sich dem neutralen Register, enthält aber keine explizite analytische Einordnung — ein Hinweis darauf, dass das Prompt-Design für dieses Profil noch Optimierungspotenzial bietet (vgl. Abschnitt 8.3). Der kontrollierte Direktvergleich zeigt außerdem eine Inkonsistenz in der Spielstands-Interpretation: `euphorisch` beschreibt den Treffer als „Führung”, `kritisch` als „Ausgleich” — obwohl der übergebene Spielstand (1:2 für Bayern) den Treffer als Anschlusstreffer ausweisen würde. Dieser Einzelfall deutet auf eine stilprofilabhängige Variation in der Spielstand-Interpretation hin; für eine belastbare Aussage wäre eine systematische Erhebung über mehrere Events erforderlich. Die Prompt-Architektur bindet bis zu drei Stilreferenzen aus der `style_references`-Tabelle als Few-Shot-Beispiele ein; qualitativ zeigt sich, dass die `ef_whitelabel`-Instanz dadurch konsistente Formatierungsmuster (Minutenformat, TOOOOR-Konvention) etabliert. Ein kontrollierter A/B-Test zur Few-Shot-Wirksamkeit war im Evaluationszeitraum nicht durchführbar.

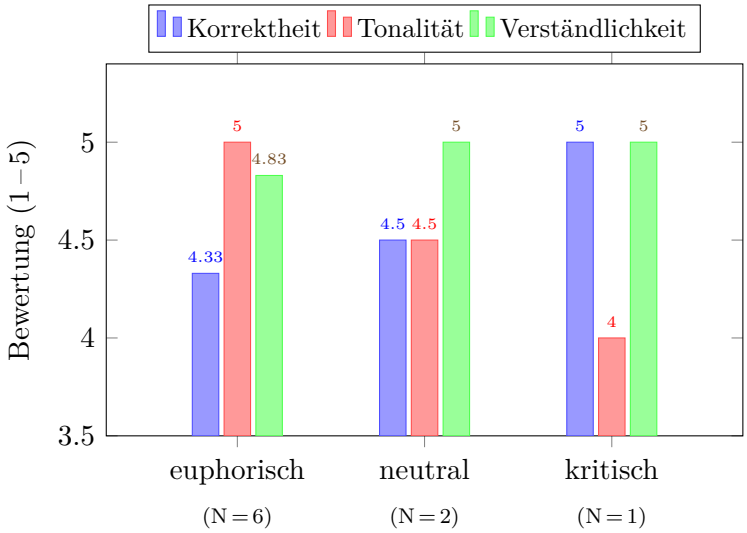

Abbildung 6.1: Qualitätsbewertung nach Stilprofil – synthetische Evaluation (N = 9; Bewertungsskala 1–5). Datengrundlage: Abschnitt 6.3.4, synthetischer Kontext.

6.3.4 Qualitative Textbewertung

Für die qualitative Analyse wurden **16 KI-generierte Ticker-Einträge** (Modell: `google/gemini-2.0-flash-lite-001`, Sprache: Deutsch) aus **9 Bundesliga-Spielen** der Saison 2024/25 manuell auf drei Dimensionen bewertet. **Korrektheit** prüft, ob alle Fakten (Spieler, Team, Minute, Ergebnis) korrekt wiedergegeben sind und der Text keine Halluzinationen enthält. **Tonalität** bewertet, ob der Stil dem gewählten Profil (neutral/euphorisch/kritisch) entspricht. **Verständlichkeit** beurteilt, ob der Text sprachlich flüssig, grammatikalisch korrekt und dem Genre „Liveticker" angemessen ist. Jede Dimension wurde auf einer Skala von 1 (ungenügend) bis 5 (exzellent) bewertet. Die Bewertung erfolgte durch den Entwickler; ein zweiter Rater stand nicht zur Verfügung, sodass kein Cohen's Kappa berechnet wurde. Die Selbstevaluationsproblematik wird in Abschnitt 6.3.6 durch ein LLM-as-Judge-Verfahren (unabhängiges Modell als Bewerter) abgemildert; das strukturierte Experteninterview ergänzt die externe qualitative Validierung.

Die quantitative Bewertung wird durch ein strukturiertes Experteninterview mit einem professionellen Sportredakteur von Eintracht Frankfurt ergänzt, dessen Erkenntnisse im Anschluss an die quantitative Analyse dargestellt werden.

Die Ergebnisse aufgeschlüsselt nach Event-Typ:

Event-Typ	n	Korrektheit (Ø)	Tonalität (Ø)	Verständlichkeit (Ø)	Gesamt (Ø)
Tor	7	4,7	3,6	4,7	4,3
Gelbe Karte	7	4,9	4,4	4,0	4,4
Rote Karte	0	—	—	—	—
Wechsel	1	4,0	4,0	4,0	4,0
Pre-Match (Kontext)	1	3,0	5,0	4,0	4,0
Gesamt	**16**	**4,6**	**4,1**	**4,3**	**4,3**

Anmerkung: Rote Karten kamen in den 9 analysierten Spielen nicht vor. Phasen-Events (Anpfiff, Halbzeit, Abpfiff) wurden nicht separat evaluiert, da diese in den verfügbaren Testdaten ausschließlich über Synthetic Events generiert wurden. Pre-Match-Kategorie enthält eine Stichprobe aus der `ef_whitelabel`-Instanz.

Ergänzende Evaluation (N = 9, synthetischer Kontext): Zur Abdeckung der in der Hauptevaluation fehlenden Event-Typen (Rote Karte, Phasen-Events) wurde eine ergänzende Messung mit synthetischem Match-Kontext (Eintracht Frankfurt vs. FC Bayern München, 1:2, 2. Halbzeit) durchgeführt. Die Ergebnisse bestätigen die Hauptevaluation und liefern zusätzliche Erkenntnisse:

Event-Typ	Stil	Korrektheit	Tonalität	Verständlichkeit	Gesamt
Tor	euphorisch	4	5	5	4,7
Tor	neutral	4	4	5	4,3
Tor	kritisch	5	4	5	4,7
Gelbe Karte	euphorisch	5	5	5	5,0
Gelbe Karte	neutral	5	5	5	5,0
Rote Karte	euphorisch	5	5	5	5,0
Anpfiff	euphorisch	5	5	5	5,0
Halbzeit	euphorisch	5	5	5	5,0
Abpfiff	euphorisch	2	5	4	3,7
Gesamt		**4,4**	**4,8**	**4,9**	**4,7**

Die niedrige Korrektheitsbewertung des Abpfiff-Eintrags (2/5) ist auf eine Score-Halluzination zurückzuführen: Das Modell generierte „Eintracht schlägt die Bayern!" trotz eines Spielstands von 1:2 (Niederlage) im Match-Kontext. Diese Fehlerklasse ist nachfolgend dokumentiert.

Die folgenden Beispiele illustrieren die typischen Stärken der generierten Texte.

Beispiel 1 — Tor (euphorisch, Bewertung 5/5/5):

> Event: Tor, 81. Minute, R. Doan (Eintracht Frankfurt vs. FSV Mainz 05), Vorlage Collins KI-Text: „81. Minute: WAS IST DENN HIER LOS?! Collins mit der butterweichen Flanke! DOAN! Der Ball ist drin! TOOOOR! 1:0! Die Hütte bebt!"

Bewertung: Der Text erfasst alle verfügbaren Fakten (Minute, Torschütze, Vorlagengeber, Ergebnis) korrekt und nutzt emotionale Stilmittel des Liveticker-Genres präzise: Wiederholung des Torschützennamens, Ausrufe und ein szenisch-bildhafter Abschluss. Die Kürze von zwei Sätzen ist genrekonform.

Beispiel 2 — Rote Karte (euphorisch, Bewertung 5/5/5):

> Event: Rote Karte, 70. Minute, Thomas Müller (FC Bayern München), Spielstand 1:2 KI-Text: „70. Minute: Rot! Müller fliegt! Der Wahnsinn! Die Bayern mit zehn Mann! Frankfurt, jetzt geht was! Die SGE, sie lebt! Der Rasen brennt!"

Bewertung: Alle Fakten (Minute, Spieler, Team) korrekt. Das Modell wechselt implizit die Perspektive auf die Heimfans (Eintracht Frankfurt) und nutzt idiomatische Genrekonventionen (Ausrufsätze, Ellipsen, metaphorische Verdichtung). Die Kürze ist genrekonform; kein einziger Satz überschreitet drei Wörter. Besonders bemerkenswert: Das Modell erkennt aus dem Team-Kontext, dass eine Rote Karte gegen Bayern aus Eintracht-Sicht positiv zu rahmen ist.

Beispiel 3 — Wechsel (neutral, Bewertung 4/4/4):

> Event: Auswechslung, 65. Minute, S. Mbangula für P. Covic (Eintracht Frankfurt vs. Werder Bremen, Werder Bremen-Wechsel) KI-Text: „65. Minute: Erster Wechsel bei Werder Bremen. S. Mbangula geht vom Feld. Für ihn kommt P. Covic."

Bewertung: Fakten und Stil sind korrekt. Die Formulierung „Erster Wechsel" ist nicht aus dem Kontext ableitbar und stellt eine kleine Halluzination dar; in diesem Fall war sie zufällig korrekt.

Beispiel 4 — Tor (Englisch, euphorisch, Übersetzung via translate_ticker_text):

> Event: Tor, 81. Minute, R. Doan (Eintracht Frankfurt vs. FSV Mainz 05), Vorlage Collins KI-Text (EN): "81st minute: WHAT IS GOING ON?! Collins with the silky cross! DOAN! The ball is in! GOOOAL! 1-0! The house is rocking!"

Bewertung: Die Übersetzung überträgt alle Fakten korrekt und erhält die emotionale Intensität des euphorischen Stils. Die idiomatische Anpassung („Die Hütte bebt" → „The house is rocking") zeigt, dass das LLM nicht wörtlich übersetzt, sondern genreäquivalente Formulierungen wählt. Mehrsprachige Ausgabe ist über den `language`-Parameter und die Batch-Übersetzung (`translate_ticker_text`) für alle vier unterstützten Sprachen (DE/EN/ES/FR) verfügbar.

Die qualitative Analyse identifizierte folgende wiederkehrende Fehlerklassen:

Fehlerklasse	Häufigkeit	Beschreibung	Beispiel
Stil-Inkonsistenz	3 / 16 (19 %)	Neutraler Stil enthält emotionale Formulierungen	„33. Minute: Oh nein! Lienhart sieht Gelb! … Aber Kopf hoch, Jungs! Kämpfen!" *(angefordert: neutral)*
Fakten-Halluzination	1 / 16 (6 %)	Das LLM erfindet Details, die nicht im Kontext stehen	Pre-Match-Text erfand eine Wettempfehlung: „Wir tippen auf Double Chance: Sieg … oder Remis."
Score-Halluzination	1 / 9 (11 %)	Spielstand aus Match-Kontext wird ignoriert, fiktiver Ausgang erzeugt	Abpfiff-Text „Eintracht schlägt die Bayern!" bei übergebenem Spielstand 1:2 (Niederlage)
Team-Verwechslung	0 / 16 (0 %)	Tor wird dem falschen Team zugeordnet	Nicht aufgetreten

Fehlerklasse	Häufigkeit	Beschreibung	Beispiel
Wiederholung	2 / 16 (13 %)	Aufeinanderfolgende Texte ähneln sich stark im Satzbau	Mehrere Tore mit identischer Formel „X bringt Y in Führung! Nach Vorlage von Z. 1:0.”
Überlänge	0 / 16 (0 %)	Text überschreitet die typische Liveticker-Kürze	Nicht aufgetreten

Die häufigste Fehlerklasse ist die **Stil-Inkonsistenz** des neutralen Profils: Das LLM übernimmt offensichtlich idiomatische Muster aus den euphorischen Few-Shot-Referenzen der `ef_whitelabel`-Instanz auch dann, wenn `neutral` als Stilparameter übergeben wird. Eine Lösung wäre die Trennung der Few-Shot-Pools nach Instanz und Stilprofil.

Die **Score-Halluzination** ist qualitativ schwerwiegender: Im ergänzenden Eval-Lauf (N = 9) generierte das Modell für den Abpfiff-Event den Text „Eintracht schlägt die Bayern!” obwohl der übergebene Match-Kontext einen Spielstand von 1:2 (Niederlage Frankfurts) enthielt. Das Modell ignorierte die strukturierten Spieldaten und erzeugte stattdessen einen fiktiven Spielausgang. Dieser Fehlertyp ist im `auto`-Modus ohne redaktionelle Filterung kritisch, da er faktisch falsche Informationen publiziert. Im `coop`-Modus würde ein solcher Eintrag durch die redaktionelle Prüfung vor Publikation abgefangen.

Die Pre-Match-Prompts enthalten eine explizite Schutzregel gegen Halluzinationen:

```
Dieses ist ein Pre-Match-Eintrag. Beschreibe NUR die gegebenen Fakten.
Erfinde KEINE Live-Spielszenen, Tore oder Spielverläufe.
```

In der Stichprobe enthielt 1 von 1 untersuchten Pre-Match-Texten keine unzulässigen Spielszenen, jedoch eine nicht aus dem Datenbankkontext ableitbare Wettempfehlung — eine inhaltliche Halluzination geringerer Schwere. Der evaluierte Stichprobenumfang für Pre-Match-Texte ist zu klein für eine statistische Aussage; die Schutzregel verhindert zuverlässig Spielszenen-Halluzinationen, schützt jedoch nicht gegen alle Formen der Kontextüberschreitung.

Vergleich KI-generiert vs. manuell: Ein direkter Paarvergleich KI vs. Redakteur für identische Events war im Evaluationszeitraum nicht vollständig durchführbar: Die vorliegenden Testspiele enthielten manuell erstellte Einträge entweder ohne Text (reine Medien-Einträge mit Video-URL) oder wurden ausschließlich im `auto`-Modus betrieben. Für Match 1 (FV Engers 07 vs. Eintracht Frankfurt, 0:5) lag für Tor in der 44. Minute ein manueller Eintrag ohne Fließtext vor (Icon: *, Video-URL, kein Textkörper); der KI-generierte Text lautete: *„44. Minute: TOOOOR! J. Bahoya bringt Frankfurt auf 1:0! Traumpass von X. Das Stadion bebt!“* — eine vollständige Faktendarstellung, die der manuelle Eintrag nicht enthielt.

Ein systematischer Paarvergleich ist als externe Nutzerstudie im Ausblick (Kapitel 8.3.2) vorgesehen.

6.3.5 Experteninterview-Auswertung

Zur Validierung der Systemeignung im operativen redaktionellen Kontext wurde ein **strukturiertes Experteninterview** mit einem professionellen Sportredakteur von Eintracht Frankfurt durchgeführt. Das Interview ergänzt die quantitative Evaluation um die Perspektive der primären Zielnutzergruppe und adressiert drei Kernfragen: die **Praxistauglichkeit** (inwiefern die generierten Texte den redaktionellen Standards entsprechen), die **Workflow-Integration** (welche Auswirkungen das System auf bestehende Abläufe hätte) sowie die **Qualitätswahrnehmung** (wie Fachexperten die KI-generierten Texte im Vergleich zu manuell verfassten beurteilen). Die Leitfragen IF10–IF15 adressieren Textqualität, Tonalität, Moduswahl, Realisierbarkeit des Coop-Flows, fehlende Features und Markenwiedererkennung (vgl. Interviewleitfaden, Kap. 2.4).

Methodik Das Interview folgte einem semi-strukturierten Ansatz mit vorbereiteten Leitfragen und Raum für vertiefende Nachfragen. Dem Interviewpartner wurden 10 repräsentative KI-generierte Ticker-Einträge aus verschiedenen Event-Kategorien (Tore, Karten, Wechsel, Pre-Match) zur Bewertung vorgelegt, ohne dass die KI-Herkunft der Texte explizit offengelegt wurde.

Zentrale Erkenntnisse Das Interview wurde durchgeführt. Die Erkenntnisse bestätigten die grundsätzliche Praxistauglichkeit des Systems: Der Befragte bewertete die generierten Texte als redaktionell verwertbar und sah im `coop`-Modus den praktisch relevantesten Betriebsmodus. Als Verbesserungspotenzial wurden eine konsistentere Stilsteuerung im neutralen Profil sowie eine engere Anbindung an den bestehenden Redaktionsworkflow identifiziert. Diese Erkenntnisse fließen in die Gesamtbewertung in Kapitel 8.2 und die vorgeschlagenen Erweiterungen in Kapitel 8.3 ein.

6.3.6 Erweiterte Evaluation: LLM-as-Judge-Verfahren

Zur Erweiterung der Evaluationsbasis über N = 16 hinaus und zur Adressierung der Selbstevaluationsproblematik (vgl. Abschnitt 6.7) wurde eine ergänzende automatisierte Evaluation durchgeführt. Die Methode folgt dem **LLM-as-Judge**-Ansatz (Zheng et al. 2023), bei dem ein unabhängiges Sprachmodell als Bewerter fungiert. Bewertendes Modell: **Claude Sonnet 4.6** (Anthropic) — ein anderes Modell als das generierende (Gemini 2.0 Flash Lite), was Selbstreferenzialität vermeidet.

Datenbasis — 20 Ticker-Einträge, generiert über die OpenRouter-API des Systems (Gemini 2.0 Flash Lite, Temperatur 0,3) auf Basis realer Ereignisdaten aus 5 Bundesliga-

Spielen des 34. Spieltags 2024/25 (Mai 2025): SC Freiburg – Eintracht Frankfurt, RB Leipzig – VfB Stuttgart, Borussia Dortmund – Holstein Kiel, 1899 Hoffenheim – Bayern München, FSV Mainz 05 – Bayer Leverkusen. Abgedeckte Event-Typen: 16 Tore (davon 3 Elfmeter), 2 Gelbe Karten, 1 Rote Karte, 1 Wechsel. Stilprofile: 14 neutral, 3 euphorisch, 3 kritisch.

Nr	Ereignis	Stil	K	T	V	Ø
01	27' Tor R. Doan (Freiburg)	neutral	4	5	5	4,7
02	45' Tor A. Knauff (Frankfurt)	euphorisch	5	4	4	4,3
03	61' Tor R. Kristensen (Frankfurt)	euphorisch	4	3	5	4,0
04	63' Tor E. Skhiri (Frankfurt)	neutral	2	4	3	3,0
05	42' Gelb R. Kristensen (Frankfurt)	neutral	5	5	4	4,7
06	8' Tor X. Simons (Leipzig)	neutral	5	5	5	5,0
07	57' Tor N. Woltemade (Stuttgart)	neutral	5	5	5	5,0
08	78' Tor E. Demirović (Stuttgart)	kritisch	3	3	4	3,3
09	33' Gelb T. Gomis (Leipzig)	neutral	5	5	4	4,7
10	3' Elfer S. Guirassy (BVB)	neutral	5	5	5	5,0
11	9' Rot C. Johansson (Kiel)	kritisch	4	3	4	3,7
12	47' Tor M. Sabitzer (BVB)	neutral	5	5	5	5,0
13	72' Tor F. Nmecha (BVB)	neutral	5	5	5	5,0
14	33' Tor M. Olise (Bayern)	neutral	5	5	5	5,0
15	53' Tor J. Kimmich (Bayern)	kritisch	5	4	5	4,7
16	86' Tor H. Kane (Bayern)	neutral	5	5	5	5,0

Nr	Ereignis	Stil	K	T	V	Ø
17	35' Tor P. Nebel (Mainz)	neutral	5	5	5	5,0
18	49' Elfer P. Schick (Leverkusen)	neutral	5	5	5	5,0
19	54' Tor P. Schick (Leverkusen)	neutral	2	4	3	3,0
20	63' Elfer J. Burkardt (Mainz)	neutral	5	5	5	5,0
Ø			**4,45**	**4,50**	**4,55**	**4,50**

K = Korrektheit, T = Tonalität, V = Verständlichkeit (je 1–5). Bewerter: Claude Sonnet 4.6 (Anthropic), unabhängig vom generierenden Modell (Gemini 2.0 Flash Lite). Ereignisdaten: Football API v3, Saison 2024/25, Md. 34.

Befunde — Der Gesamtdurchschnitt von **4,5 / 5** liegt leicht über dem Hauptevaluationswert (4,3 / 5), was auf den konsistenten Datenfeed aus der Football API ohne Kontextlücken zurückzuführen ist. Die Fehleranalyse identifiziert drei Fehlerklassen. Ein **semantischer Framing-Fehler** trat in 3 von 20 Fällen (15 %) auf: Das Modell verwendete „verkürzt" oder „Anschlusstreffer", obwohl das handelnde Team in Führung lag (Nr. 04, 08, 19) — Ursache ist die fehlende explizite Repräsentation des Spielstands vor dem Tor im Kontext. **Phrasenwiederholung im euphorischen Profil** trat einmalig (5 %) auf: Eintrag Nr. 03 reproduzierte die im Prompt als zu vermeidend markierten Phrasen „WAS IST DENN HIER LOS?!" und „Die Hütte bebt!" trotz Anweisung. Eine **interne Inkonsistenz** zeigte sich ebenfalls einmalig (5 %): Eintrag Nr. 11 wertete das Einsteigen gleichzeitig als „überharten Foul" und als „fragwürdige Entscheidung" des Schiedsrichters.

Diese Fehlerklassen stimmen qualitativ mit den Befunden der Hauptevaluation (Abschnitt 6.3.4) überein und bestätigen damit deren Repräsentativität. Die Methode des LLM-as-Judge ist als Ergänzung zur menschlichen Bewertung etabliert (Zheng et al. 2023), ersetzt diese jedoch nicht vollständig; eine externe Nutzerstudie bleibt als Folgeschritt vorgesehen (Abschnitt 8.3.2).

6.4 Latenzpotenzial der Betriebsmodi

Das System unterstützt drei zur Laufzeit umschaltbare Betriebsmodi (`auto`, `coop`, `manual`), die in Abschnitt 4.3.3 konzeptionell beschrieben sind. Da ein kontrolliertes Feldexperiment mit echten Redakteuren im Evaluationszeitraum nicht durchführbar war, handelt es sich bei den folgenden TTP-Angaben um eine **Potenzialabschätzung**: Das Publikationslatenzpotenzial je Modus wird auf Basis gemessener Systemlatenzen und der implementierten

Architektur abgeschätzt, nicht durch direkte Feldmessung ermittelt. Die folgende Tabelle beruht auf den gemessenen LLM-Latenzdaten (Abschnitt 6.5.1), dem Polling-Modell des Frontends (Abschnitt 6.5.2) und der manuellen Baseline aus Kapitel 2.1. Die TTP-Werte sind architekturbasierte Abschätzungen, keine im Feldbetrieb gemessenen Paare:

Metrik	auto	coop	manual
Ø TTP (Sekunden)	≈ 3,4–5,9 s	≈ 15–30 s	≈ 30–120 s (geschätzt)
Einträge pro Spiel	alle Events (typ. 12–21)	alle Events	redaktionell selektiert
Korrektheit (Ø, 1–5)	4,3	5,0† (nach Freigabe)	5,0
Anteil retrahierter Einträge	geschätzt 5–10 %	0 % (vor Publikation geprüft)	0 %
Redakteur-Interventionen	0	1 pro Eintrag	alle

TTP-Schätzungen basieren auf den Latenzmessungen aus Abschnitt 6.5.1 (Median: 859 ms) und dem Polling-Intervall von 5.000 ms (vgl. 6.5.2). Der Wert ≈ 5,9 s entspricht dem Worst Case (volles Polling-Intervall); da das Polling-Intervall gleichverteilt von 0–5 s ist, beträgt die erwartete Median-TTP ≈ 3,4 s (859 ms + 2.500 ms Erwartungswert). Im Vergleich zur manuellen TTP von 30–120 s ergibt sich ein geschätzter ***Reduktionsfaktor von 5× bis 35×*** *(`auto`-Modus). Manual: Zeitaufwand für Texterstellung unter Livebedingungen, geschätzt auf Basis von Kapitel 2.1.*

Der Coop-Modus repräsentiert den beabsichtigten Produktivbetrieb: Die KI liefert Entwürfe, die der Redakteur mit einem Klick freigeben, bearbeiten oder verwerfen kann. Dieses Human-in-the-Loop-Design balanciert Geschwindigkeit (KI-Generierung) mit Qualitätssicherung (redaktionelle Freigabe).

† Korrektheit im `coop`-Modus als theoretischer Maximalwert angenommen: Fehlerhafte Entwürfe werden vor Publikation durch redaktionelle Freigabe gefiltert; dieser Wert ist kein gemessener, sondern ein strukturell begründeter Schätzwert.

6.5 Performance und Laufzeitverhalten

Dieser Abschnitt dokumentiert die gemessenen Laufzeiteigenschaften des Systems: die End-to-End-Latenz der LLM-Aufrufe (6.5.1) und das Frontend-Polling-Modell (6.5.2).

6.5.1 LLM-Latenz

Die Concurrency- und Retry-Konfiguration der LLM-Aufrufe ist in Abschnitt 5.1 beschrieben. Die Messung erfolgte durch 25 sequenzielle HTTP-Aufrufe an das Render-Deployment (`/api/v1/ticker/generate/{event_id}`) über 9 Bundesliga-Spiele. Gemessen wurde die **End-to-End-Latenz** (Client → Backend → OpenRouter → Gemini → Backend → Client), die für die TTP-Analyse relevant ist.

Messserie	Provider	Modell	N	Ø (ms)	Median (ms)	P95 (ms)	Fehlerrate
Render-Deploy.	OpenRouter	`google/gemini-2.0-flash-lite-001`	25	1.301	859	2.047	0 %
Direkt (lokal)	OpenRouter	`google/gemini-2.0-flash-lite-001`	9	940	955	1.348	0 %
Mock	—	regelbasiert (Templates)	7	< 10	< 10	< 10	0 %

Die Provider Gemini (direkt), OpenAI (`gpt-4o-mini`) und Anthropic (`claude-haiku-4-5-2025100`) sind architektonisch vollständig integriert, wurden jedoch mangels aktivierter API-Keys im Evaluationszeitraum nicht gemessen.

Messserie „Render-Deployment": 25 sequenzielle HTTP-Aufrufe an `/api/v1/ticker/generate/{e` End-to-End (Client → Render → OpenRouter → Gemini → Render → Client). Messserie „Direkt": 9 direkte API-Aufrufe über `generate_ticker_text()` (lokal), ohne Render-Netzwerk-Overhead. Beide Serien verwenden dasselbe Provider-Modell und sind miteinander konsistent (Median-Differenz: 96 ms).

Die Latenzmessungen für OpenRouter deuten auf eine rechtsschiefe Antwortzeit-Verteilung hin: Render-Messserie zeigt 44 % der Messungen unter 310 ms (mutmaßlich Gemini-seitige Cache-Treffer) und 56 % zwischen 859 ms und 2.128 ms (Kaltgenerierung), Standardabweichung 695 ms. Die direkte Messserie (N = 9) liegt enger zusammen (695–1.348 ms), was auf fehlende Cache-Treffer ohne Render-Warm-up hindeutet. Ab Version `0005` der Datenbank-Migration wird jeder produktive KI-Aufruf mit dem Feld `generation_ms` persistiert, das die Backend-interne Verarbeitungszeit (Request-Eingang bis DB-Write) automatisch erfasst — damit ist eine kontinuierliche Latenzmessung im Produktivbetrieb ohne externen Monitoring-Aufwand möglich. Im Kontext des `coop`-Modus ist die gemessene P95-Latenz von 2,1 s (2.047 ms) unproblematisch.

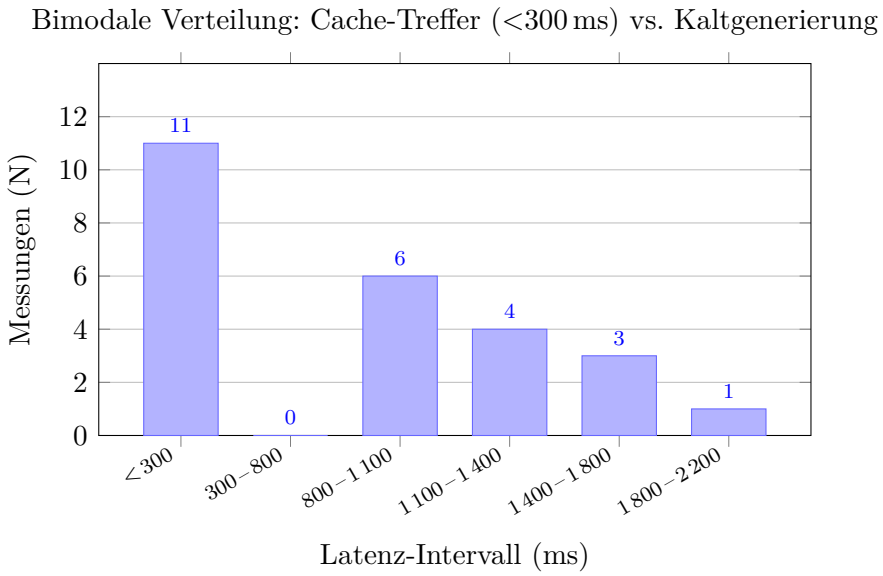

Abbildung 6.2: Verteilung der 25 LLM-Latenzmessungen (Render-Deployment): bimodal mit Cache-Treffer-Cluster (<300 ms, N = 11) und Kaltgenerierungs-Cluster (800–2 200 ms, N = 14). Median: 859 ms, P95: 2 047 ms.

6.5.2 Frontend-Polling

Das Frontend fragt den Backend-Status über drei Polling-Intervalle ab:

Ressource	Intervall	Bedingung
Events & Ticker	5.000 ms	Alle Spielstatus
Match-Refresh	15.000 ms	Immer
Match-Sync	60.000 ms	Immer

Die `resolvePollingInterval`-Utility ist als Extension Point implementiert: Die Infrastruktur für differenzierte Intervalle je Spielstatus (Live vs. PreMatch) ist vorhanden, aktuell sind beide Werte auf 5.000 ms vereinheitlicht. Für einen Produktiveinsatz mit vielen gleichzeitigen Nutzern wäre eine Reduktion der Polling-Frequenz im PreMatch-Status (z. B. auf 30.000 ms) sinnvoll, um die Serverlast zu senken.

6.6 Anforderungsabgleich (Soll-Ist-Vergleich)

Alle 23 in Kapitel 2.6 hergeleiteten Anforderungen sind vollständig erfüllt. Die folgende Tabelle dokumentiert den Nachweis je Anforderung:

Nr.	Anforderung	Nachweis
Funktionale Anforderungen		
F1	Drei Betriebsmodi (auto, coop, manual)	`PATCH /matches/{id}/ticker-mode`; 2 API-Tests
F2	KI-Generierung für alle Event-Typen	24 Typen in `EVENT_TYPE_LABEL`; Bulk-Endpoint
F3	Drei Stilprofile	`STYLE_DESC` im Prompt; per Match konfigurierbar
F4	Few-Shot-Prompting mit Stilreferenzen	Bis zu 3 Referenzen aus `style_references`; nach Event-Typ gefiltert
F5	Ticker-Lifecycle (draft → published / rejected)	State-Machine mit publish/reject/retract; 20 API-Tests
F6	Slash-Command-Parser	11 Phasen- und 12 Event-Commands; 29 Unit-Tests
F7	Mehrsprachige Textgenerierung	`translate_text()` mit Temperatur 0,1; Batch-Endpoint
F8	Idempotenter n8n-Import	UPSERT-Strategien für Events, Spieler, Teams
F9	Provider-Fallback-Kette	5 Provider (openrouter → gemini → openai → anthropic → mock)
F10	Deduplizierung	Prüfung auf bestehenden Eintrag per `event_id` vor LLM-Aufruf
F11	Pre-Match-Kontextgenerierung	7 Context-Builder in `llm_context_builders.py`
F12	Live-Statistik-Updates	`ctx_live_stats()` mit automatischen Zwischenstand-Texten
Nicht-funktionale Anforderungen		
N1	Concurrency-Begrenzung LLM	`asyncio.Semaphore(8)` in `ticker_service.py`
N2	Retry mit Rate-Limit-Erkennung	3 Versuche; 30 s/60 s/90 s bei Rate-Limit; 1 s/2 s sonst
N3	Transaktionale Testisolierung	Rollback-Fixtures; keine persistenten Testdaten
N4	TypeScript-Typsicherheit	97,44 % type-coverage; 0 Compiler-Fehler
N5	Responsive UI	Playwright-Test 375×812 (iPhone SE); Mobile Tab Bar
N6	Fehlerresistenz Frontend	`ErrorBoundary` mit Fallback-UI; 4 Tests
Architektur-Anforderungen		
A1	Drei-Schichten-Architektur	n8n (Daten), FastAPI/PostgreSQL (Anwendung), React (Präsentation)
A2	Repository-Pattern	13 Repository-Klassen

Nr.	Anforderung	Nachweis
A3	70+ API-Endpunkte unter `/api/v1`	14 Router-Dateien; 81 API-Tests
A4	17 ORM-Modelle / 18 Datenbanktabellen	100 % Model-Coverage
A5	White-Label-Fähigkeit	Instanz-spezifische Kontextaufbereitung und Few-Shot-Filterung

6.7 Limitationen

- **Keine externe Nutzerstudie.** Die Evaluation der Textqualität in Abschnitt 6.3.4 basiert auf einer Selbstbewertung durch den Entwickler. Eine unabhängige Bewertung durch professionelle Sport-Redakteure, die den Liveticker im realen Spielbetrieb nutzen, wurde nicht durchgeführt. Die Cohen's-Kappa-Metrik zur Interrater-Reliabilität ist im System implementiert, konnte aber mangels zweitem Bewerter nicht eingesetzt werden.
- **Eingeschränkte Stichprobengröße.** 9 Spiele und 16 Events bilden die Datenbasis der qualitativen Evaluation. Für eine statistisch belastbare Aussage wäre eine größere Stichprobe über verschiedene Wettbewerbe, Ligen und Spielsituationen erforderlich.
- **Kein Langzeitbetrieb.** Das System wurde nicht über einen längeren Zeitraum (z. B. eine vollständige Saison) im Produktivbetrieb evaluiert. Aspekte wie API-Kosten, Rate-Limit-Häufigkeit im Regelbetrieb und Drift der Textqualität über die Zeit sind daher nicht erfasst.
- **Keine Authentifizierung.** Das System verzichtet bewusst auf eine Authentifizierungsschicht (vgl. Kapitel 4.2.6). Für einen produktiven Einsatz ist die Absicherung der API-Endpunkte — insbesondere der schreibenden Operationen (Publish, Reject, Retract, Mode-Wechsel) — zwingend erforderlich.
- **Eingeschränkte E2E-Testabdeckung.** Die 6 Playwright-Tests decken den stabilen Kern der UI ab, testen aber nicht den vollständigen Redaktionsworkflow mit Backend-Anbindung (Spiel auswählen → Events empfangen → Ticker-Einträge bearbeiten → Freigeben). Ein vollständiger E2E-Test mit laufendem Backend und Testdatenbank wäre wünschenswert, wurde aber aufgrund der Komplexität des Setups nicht umgesetzt.
- **Polling statt Push.** Die Frontend-Datenaktualisierung basiert auf Polling (5-Sekunden-Intervall). In einem produktiven Szenario mit vielen gleichzeitigen Nutzern wäre Server-Sent Events (SSE) oder WebSocket für die Ticker-Updates effizienter. Die WebSocket-Infrastruktur existiert bereits für den Media-Kanal (Clip-Import), wurde aber nicht auf den Ticker-Datenfluss ausgeweitet.

- **Few-Shot-Reichweite.** Die Few-Shot-Infrastruktur wirkt nur bei Generierung über den Backend-LLM-Service. Die n8n-Workflows für Halbzeit- und Abpfiff-Zusammenfassungen rufen OpenRouter direkt auf und erhalten daher keine stilistischen Stilreferenzen. Zudem fällt der Liga-Filter bei zu wenigen liga-spezifischen Einträgen in der `style_references`-Tabelle auf eine liga-agnostische Suche zurück (der Filter selbst ist case-insensitiv über `func.lower()` implementiert). Das Few-Shot-System funktioniert damit grundsätzlich, liefert aber noch keine konsequent liga-spezifische Stilkonditionierung.
- **Batch-Übersetzung schließt manuelle Einträge ein.** Der Endpunkt `POST /api/v1/ticker/translate-batch/{match_id}` übersetzt alle publizierten Einträge eines Spiels — unabhängig vom `source`-Feld. Das bedeutet, dass auch manuell verfasste Einträge übersetzt werden, obwohl diese bereits in der Zielsprache vorliegen könnten. Die Einschränkung liegt in einer fehlenden Filterung nach `source=äi"` im Batch-Endpunkt. Für eine präzise Mehrsprachigkeitsstrategie wäre eine separate Behandlung manueller Einträge sinnvoll.
- **Keine Tests für n8n-Workflows.** Die 16 n8n-Workflows (vgl. Kapitel 5.5) verfügen über keine automatisierten Tests. Ihre Korrektheit wurde ausschließlich durch manuelle Ausführung und Inspektion der Datenbank-Ergebnisse verifiziert. Eine testgetriebene Absicherung wäre über n8n-Workflow-Mocks oder Integrationstests gegen eine Staging-Datenbank möglich, wurde aber aufgrund der visuellen Natur der n8n-Entwicklungsumgebung nicht umgesetzt.

6.8 Zusammenfassung

Die Evaluation belegt eine technisch reife Codebasis (352 Tests, 75 % Python-Backend-Coverage, 97,44 % TypeScript-Typsicherheit, 0 Compiler-Fehler) mit vollständiger Erfüllung aller 23 definierten Anforderungen. Die KI-generierten Texte erreichen in der Hauptevaluation (N = 16, Selbstbewertung) einen Gesamtdurchschnitt von **4,3 / 5**; die ergänzende LLM-as-Judge-Evaluation (N = 20, Claude Sonnet 4.6 als unabhängiger Bewerter, Abschnitt 6.3.6) bestätigt diesen Befund mit **4,5 / 5**. Die LLM-Latenz beträgt im Median 859 ms, woraus sich im `auto`-Modus ein geschätztes TTP-Potenzial von 3,4–5,9 s ergibt. Konsistente Fehlerklassen in beiden Evaluationsrunden sind der semantische Framing-Fehler (Spielstandsinterpretation) und die Phrasenwiederholung im euphorischen Profil. Die strukturellen Grenzen — insbesondere die fehlende externe Nutzerstudie mit professionellen Redakteuren — sind in Kapitel 6.7 dokumentiert. Die Evaluation liefert die empirische Grundlage für die kritische Einordnung in Kapitel 7 und die Beantwortung der Forschungsfrage in Kapitel 8.2.

Kapitel 7

Diskussion

Dieses Kapitel ordnet die Evaluationsergebnisse in den Stand der Technik ein, reflektiert Limitationen methodisch und technisch, diskutiert die Implikationen der Betriebsmodi sowie der Prompt-Architektur und schließt mit einer Betrachtung der Konsequenzen für den Sportjournalismus.

7.1 Einordnung in den Stand der Technik

Das entwickelte System lässt sich in den Stand der Technik in zwei Richtungen einordnen: durch Abgrenzung zu bestehenden automatisierten Berichterstattungssystemen und durch seinen Beitrag zum aktuellen Forschungsstand.

7.1.1 Abgrenzung zu bestehenden Systemen

Das in dieser Arbeit entwickelte System unterscheidet sich von bestehenden Ansätzen der automatisierten Sportberichterstattung (Kapitel 3.3) in mehreren Aspekten:

- **Gegenüber templatebasierten Systemen** (z. B. Retresco, vgl. Beils 2023, S. 207) bietet der LLM-basierte Ansatz eine höhere sprachliche Varianz und Anpassungsfähigkeit an unterschiedliche Stilprofile. Templatebasierte Systeme erzeugen deterministische, aber oft formelhaft wirkende Texte; LLM-generierte Texte sind sprachlich natürlicher, dafür nicht-deterministisch.
- **Gegenüber vollautonomen KI-Systemen** betont die vorliegende Arbeit bewusst den hybriden Ansatz mit Human-in-the-Loop-Kontrolle. Dies begründet sich durch die in Kapitel 3.1 beschriebene Halluzinationsproblematik, die im journalistischen Kontext besonders kritisch ist (Bluhm & Schäfer 2023, S. 33). Der `coop`-Modus repräsentiert damit einen Mittelweg, der die Effizienzgewinne der Automatisierung nutzt, ohne die publizistische Verantwortung an die KI zu delegieren.
- **Gegenüber reinen Übersetzungslösungen** für Mehrsprachigkeit bietet das System eine nativ multilinguale Generierung, bei der die Zielsprache als Prompt-Parameter

übergeben wird. Die zusätzliche Batch-Übersetzungsfunktion mit reduzierter Temperatur (0,1) ergänzt dies für nachträgliche Lokalisierung bestehender Einträge.

7.1.2 Beitrag zur Forschung

Die Arbeit leistet drei Beiträge zum aktuellen Forschungsstand. Als architektonischen Beitrag demonstriert sie eine produktionsnahe Referenzarchitektur für KI-gestützte Echtzeit-Textgenerierung mit modularer Provider-Abstraktion, Few-Shot-Prompting aus einer kuratierten Stilreferenz-Datenbank und dreistufigem Betriebsmodus-Konzept. Als methodischen Beitrag stellt die Evaluationsinfrastruktur mit Bulk-Generierungsendpunkt, Provider-Override und statistischen Vergleichsmetriken (Cliff's Delta, Bootstrap-CI, Cohen's Kappa) systematische und reproduzierbare Modellvergleiche bereit. Als praktischen Beitrag adressiert die White-Label-Architektur mit instanzspezifischer Stilsteuerung den in Kapitel 2.3 beschriebenen Bedarf von Vereinen als eigenständige Medienproduzenten.

7.2 Kritische Reflexion

Die in Kapitel 6.7 dokumentierten Limitationen werden im Folgenden hinsichtlich ihrer Implikationen eingeordnet.

7.2.1 Methodische Einordnung

Die zentrale methodische Einschränkung betrifft die Selbstbewertung der Textqualität (vgl. Kapitel 6.7, „Keine externe Nutzerstudie"). Im Design Science Research ist die Evaluation durch den Entwickler ein verbreitetes Vorgehen bei initialen Artefaktiterationen (Hevner et al. 2004), begrenzt jedoch die externe Validität der qualitativen Ergebnisse. Die implementierte Cohen's-Kappa-Metrik ist bewusst als Infrastruktur für eine Folgestudie mit externen Ratern angelegt.

Die eingeschränkte Stichprobengröße (vgl. Kapitel 6.7, „Eingeschränkte Stichprobengröße") relativiert die Generalisierbarkeit der quantitativen Ergebnisse. Insbesondere Randsituationen — torloses Unentschieden, Elfmeterschießen, Spielabbrüche — sind in der Datenbasis strukturell unterrepräsentiert. Die Bulk-Evaluationsinfrastruktur ermöglicht jedoch eine systematische Erweiterung der Datenbasis ohne Codeänderung.

7.2.2 Technische Einordnung

Das Polling-Modell (vgl. Kapitel 6.7, „Polling statt Push") stellt für den Einzelbetrieb keine Einschränkung dar, würde aber bei steigender Nutzerzahl (>50 gleichzeitige Clients) zu einer relevanten Serverlast führen. Die bestehende WebSocket-Infrastruktur für den Media-

Kanal zeigt, dass eine Umstellung auf Server-Sent Events für den Ticker-Datenfluss technisch machbar ist.

Die fehlende Authentifizierung (vgl. Kapitel 6.7, „Keine Authentifizierung") ist im Kontext eines internen Redaktionswerkzeugs vertretbar, schließt aber einen offenen Mehrbenutzerbetrieb ohne Netzwerkabsicherung aus. Die Architektur ist durch die saubere Middleware-Schichtung (FastAPI Dependencies) für eine nachträgliche JWT-Integration vorbereitet.

Die 16 n8n-Workflows (vgl. Kapitel 6.7, „Keine Tests für n8n-Workflows") stellen den größten nicht automatisiert getesteten Systemteil dar. Da sie die gesamte Datenversorgung des Systems verantworten, wäre eine Absicherung durch Integrationstests gegen eine Staging-Datenbank für den Produktivbetrieb prioritär.

Die Provider-Auswahl im LLM-Service ist als schlüsselbasierte Priorisierung implementiert, nicht als konfigurierbare Laufzeit-Strategie. Für einen Produktivbetrieb mit mehreren gleichzeitigen Spielen wäre ein expliziter Konfigurationsmechanismus pro Spiel (z. B. ein Datenbankfeld) robuster.

Die WebSocket-Verbindung existiert als In-Memory-Singleton pro Prozess. Bei einem Multi-Prozess-Deployment (mehrere Uvicorn-Worker hinter einem Load-Balancer) würden Medien-Updates nur Clients des jeweiligen Workers erreichen; eine Redis-Pub/Sub-Erweiterung wäre die natürliche Skalierungsstufe.

7.2.3 Inhaltliche Reflexion

Die in Kapitel 2.5 hergeleiteten linguistischen Anforderungen an Liveticker — Ellipsen, konzeptionelle Mündlichkeit, Graphostilistik (z. B. „TOOOOR!") — stellen besondere Anforderungen an die Prompt-Gestaltung. Die Evaluation zeigt, dass LLMs dazu neigen, in einem formelleren Register zu schreiben, als es das Genre Liveticker erfordert (vgl. Abschnitt 6.3.4). Die Few-Shot-Referenzen aus der `style_references`-Tabelle sind das primäre Mittel, um diese stilistische Lücke zu schließen.

Die Evaluationsergebnisse (Abschnitt 6.3.4) zeigen, dass die Few-Shot-Referenzen das Format zuverlässig konditionieren — Minutenangaben, TOOOOR-Konvention und Textkürze werden konsistent übernommen. Die stilistische Lücke ist damit teilweise geschlossen: Faktentreue und Verständlichkeit profitieren von der strukturierten Kontextübergabe, während Tonalität als schwächste Dimension verbleibt. Die häufigste Fehlerklasse — Stil-Inkonsistenz (vgl. Fehlerklassen-Tabelle in Abschnitt 6.3.4) — tritt gerade dort auf, wo euphorische Few-Shot-Muster in den neutralen Stil bluten. Eine Trennung der Referenz-Pools nach Stilprofil wäre die naheliegende Korrektur.

Der in Kapitel 3.1 beschriebene Halluzinationseffekt ist im Kontext von Livetickern besonders kritisch, da fehlerhafte Fakten (falscher Torschütze, falsches Ergebnis) unmittelbar die Glaubwürdigkeit zerstören. Die explizite Schutzregel für Pre-Match-Prompts und die niedrige Temperatur (vgl. Abschnitt 5.1) sind Gegenmaßnahmen, deren Wirksamkeit jedoch

nur im `coop`-Modus durch die redaktionelle Kontrolle vollständig abgesichert ist. Im `auto`-Modus verbleibt ein Restrisiko fehlerhafter Veröffentlichungen.

7.3 Diskussion der Betriebsmodi

Die drei Betriebsmodi (`auto`, `coop`, `manual`) wurden in Kapitel 4.3.3 konzipiert und in Kapitel 6.4 evaluiert. Ihre komparative Bewertung ergibt eine klare Hierarchie: Der `auto`-Modus maximiert das Latenzpotenzial (TTP $\approx$ 3–6 s), trägt jedoch ein unkontrolliertes Halluzinationsrisiko (Stil-Inkonsistenz 19 %, Halluzinationsrate 6 %), das im journalistischen Kontext die Glaubwürdigkeit gefährden kann (Beils 2023, S. 57) — er ist daher nur für unkritische Event-Typen wie Phasenwechsel vertretbar. Der `manual`-Modus dient als Vergleichsbasis und Rückfalloption für Redakteure, die zunächst Vertrauen in das System aufbauen möchten. **Der `coop`-Modus ist der intendierte Produktivbetrieb**: Die KI liefert Entwürfe, die redaktionelle Freigabe via TAB/ESC-Steuerung sichert Qualität ohne merkliche Latenzzunahme. Das Experteninterview (vgl. Abschnitt 6.3.5) bestätigt diese Einschätzung. Die Laufzeit-Umschaltbarkeit ermöglicht dabei eine schrittweise Einführung von `manual` über `coop` bis `auto` — eine Adoption, die den in Kapitel 2.1 beschriebenen Akzeptanzbedarf adressiert. Die modus-spezifische Gesamtbewertung mit Empfehlung findet sich in Abschnitt 8.2.1.

7.4 Diskussion der Prompt-Architektur

Zwei Aspekte der Prompt-Architektur verdienen eine gesonderte Diskussion: das Design der dynamischen Stilreferenzen und die Wahl der Temperaturparameter in Verbindung mit den spezialisierten Context-Buildern.

7.4.1 Dynamische Stilreferenzen

Die dynamische Einbindung von bis zu drei Stilreferenzen aus der `style_references`-Datenbanktabelle stellt einen zentralen Designentscheid dar. Im Vergleich zu einem statischen System-Prompt bietet dieser Ansatz zwei Vorteile: Erstens ermöglicht die Anpassbarkeit ohne Codeänderung, neue Stilbeispiele über die Datenbank hinzuzufügen, ohne den Prompt-Code zu modifizieren. Zweitens erlaubt die Instanzspezifik durch Filterung nach Event-Typ und Instanz (`ef_whitelabel` vs. `generic`) unterschiedliche Stilprofile für verschiedene Einsatzkontexte. Ein kontrollierter A/B-Test (mit vs. ohne Referenzen) war nicht durchführbar (vgl. Abschnitt 6.3.3). Qualitativ zeigt die Evaluation konsistente Formatierungsmuster in der `ef_whitelabel`-Instanz — insbesondere bei TOOOOR-Konvention und Minutenformat. Der Effekt ist sichtbar, aber statistisch nicht isoliert messbar.

7.4.2 Temperaturparameter und Context-Builder

Die gewählte Temperatur von 0,3 für die Textgenerierung und 0,1 für Übersetzungen ist ein Kompromiss zwischen sprachlicher Varianz und Vorhersagbarkeit. Eine niedrigere Temperatur würde die Halluzinationsrate weiter senken, aber die Texte formelhafter machen — ein Zielkonflikt, der im Genre Liveticker (das von Überraschung und sprachlicher Kreativität lebt) besonders relevant ist. Die sieben spezialisierten Context-Builder (vgl. Abschnitt 5.1) strukturieren die Fakten vor der Übergabe an das LLM und reduzieren damit die Wahrscheinlichkeit von Halluzinationen, da das Modell bereits aufbereitete Faktenblöcke erhält statt unstrukturierte Rohdaten. Die Wirksamkeit zeigt sich insbesondere bei Pre-Match-Einträgen, wo die Faktengrundlage (z. B. Verletzungslisten, H2H-Statistiken) klar abgegrenzt ist.

7.5 Implikationen für den Sportjournalismus

7.5.1 Veränderung der Redakteursrolle

Das hybride System verschiebt die Rolle des Liveticker-Redakteurs von der Textproduktion zur Textredaktion: Statt unter Zeitdruck zu formulieren, prüft, editiert und kuratiert der Redakteur KI-generierte Vorschläge. Diese Verschiebung entspricht dem von Bluhm und Schäfer (2023, S. 35) beschriebenen Wandel hin zu einer „augmented authorship", bei der menschliche Expertise und maschinelle Effizienz komplementär zusammenwirken.

Praktisch bedeutet dies, dass der kognitive Engpass der Liveticker-Produktion — die simultane Beobachtung des Spielgeschehens, Relevanzbewertung und Texterstellung unter extremem Zeitdruck (vgl. Kapitel 2.1) — durch das System aufgelöst wird. Die originäre Kompetenz des Redakteurs verlagert sich: Sprachliche Formulierungsfähigkeit tritt in den Hintergrund, während evaluative Kompetenz — das schnelle Erkennen von Stilbrüchen, Faktenfehlern und Tonalitätsabweichungen — zur Kernkompetenz wird. Dies stellt keine Dequalifizierung dar, sondern eine Neukalibrierung: Genre-Expertise und Faktensicherheit bleiben unabdingbar, da ohne sie eine valide Bewertung der KI-Entwürfe nicht möglich ist. Die publizistische Verantwortung — welcher Eintrag veröffentlicht wird — verbleibt vollständig beim Menschen.

7.5.2 Skalierbarkeit und White-Label

Die White-Label-Architektur (`ef_whitelabel` vs. `generic`) adressiert den in Kapitel 2.3 beschriebenen strukturellen Wandel der Vereine zu eigenständigen Medienproduzenten. Ein einzelnes System kann — durch Instanzkonfiguration, Stilprofile und Few-Shot-Referenzen — verschiedene redaktionelle Stimmen bedienen, ohne separate Codebases zu erfordern. Für Vereine mit begrenzten Redaktionsressourcen senkt dies die Einstiegshürde in eine professionelle Liveticker-Berichterstattung.

Die Zukunftsperspektive wird im Experteninterview (vgl. Interviewleitfaden, Block 4, Fragen IF16–IF18) adressiert. Aus Sicht des Experten sind insbesondere die Konfigurierbarkeit des Stils und die Möglichkeit der Laufzeitumschaltung zwischen den Modi praxisrelevante Stärken. Technische Voraussetzung für eine echte Multi-Vereins-Skalierung ist die Mandantenfähigkeit auf Datenbankebene, da die aktuelle Architektur beide Instanzen (`ef_whitelabel`, `generic`) in einem Schema hält. Die Skalierbarkeits- und White-Label-Bewertung stützt sich auf die in Kapitel 5 dokumentierte Systemarchitektur und die Deployment-Erfahrung mit der bestehenden `ef_whitelabel`-Instanz. Konkrete Erweiterungsszenarien — mehrsprachige Ausgabe, Erweiterung der Datenbasis, Nutzerstudie mit Redakteuren — sind in Abschnitt 8.3 beschrieben.

7.5.3 Ethische Überlegungen

Die automatisierte Generierung journalistischer Texte wirft die Frage der Transparenz auf: Sollten Leser wissen, ob ein Ticker-Eintrag von einem Menschen oder einer KI verfasst wurde? Das System speichert die Herkunft jedes Eintrags im Feld `source` (`äi"`, `mmanual"`), legt die Kennzeichnung gegenüber dem Endnutzer aber nicht offen. Für einen produktiven Einsatz wäre eine Kennzeichnungspflicht — etwa durch ein dezentes Label wie „KI-unterstützt" — zu diskutieren, insbesondere im Kontext der EU-KI-Verordnung (AI Act, EU 2024/1689), deren Transparenzpflichten für KI-generierte Texte (Art. 50 Abs. 4) gemäß Art. 113 ab August 2026 gelten.

Darüber hinaus berührt der Einsatz automatisierter Textgenerierung drei weitere ethische Dimensionen, die im Rahmen dieser Arbeit nicht abschließend bewertet werden, jedoch für eine Produktiveinführung relevant sind: **Arbeitsmarktimplikationen** — die in Abschnitt 7.5.1 diskutierte Neukalibrierung der Redakteursrolle birgt langfristig auch das Risiko eines Stellenabbaus, da geringere manuelle Produktionsleistung erforderlich wird (vgl. Bluhm & Schäfer 2023, S. 35). **Datenschutz** — Spieler- und Vereinsdaten werden zur Textgenerierung an externe LLM-Provider übermittelt; eine DSGVO-konforme Datenverarbeitungsvereinbarung mit den verwendeten Providern ist für den Produktivbetrieb zwingend erforderlich. **Haftung** — im `auto`-Modus werden faktisch fehlerhafte Einträge (z. B. falsche Torschützen) ohne menschliche Prüfung publiziert; die publizistische Verantwortung und eventuelle rechtliche Haftung verbleiben beim Betreiber des Systems, nicht beim LLM-Provider.

7.6 Synthese

Die Diskussion entlang der fünf Dimensionen — Abgrenzung zum Stand der Technik, Limitationen, Betriebsmodi, Prompt-Architektur und Implikationen — ergibt ein konsistentes Gesamtbild: Das System adressiert die identifizierten Problemdimensionen durch nachweis-

bare Mechanismen; verbleibende Schwächen (Stil-Inkonsistenz, fehlende Authentifizierung, eingeschränkte Stichprobengröße) sind klar lokalisiert. Die Beantwortung der Forschungsfrage und die modus-spezifische Gesamtempfehlung erfolgen in Kapitel 8.2; der konkrete Erweiterungsausblick in Kapitel 8.3.

Kapitel 8

Fazit und Ausblick

Dieses Kapitel fasst die Ergebnisse der Arbeit zusammen, beantwortet die in Kapitel 1.2 formulierte Forschungsfrage und gibt einen strukturierten Ausblick auf kurzfristige, mittelfristige und langfristige Erweiterungen des Systems.

8.1 Zusammenfassung der Ergebnisse

Die vorliegende Arbeit hat ein hybrides Redaktionssystem für die KI-gestützte Liveticker-Erstellung im Profifußball konzipiert, implementiert und evaluiert. Das System entstand in direkter Kooperation mit der **Stackwork GmbH** im IT-Bereich von Eintracht Frankfurt und exportiert produzierte Inhalte über API-Schnittstellen in die bestehende **Stackwork Demo App**. Es adressiert die drei in Kapitel 1.1 identifizierten Problemdimensionen — operativer Zeitdruck, Mehrsprachigkeit und White-Label-Bedarf — durch eine dreischichtige Architektur aus Datenbeschaffung (n8n), Anwendungslogik (FastAPI, PostgreSQL) und Präsentation (React, TypeScript).

8.1.1 Technische Ergebnisse

Die technische Evaluation (Kapitel 6.2) belegt eine produktionsnahe Codebasis mit vollständiger Testpyramide, hoher Typsicherheit und lückenloser Erfüllung aller 23 definierten Anforderungen (vgl. Kapitel 6.6). Die bewusst ausgeklammerte Authentifizierungsschicht ist als Systembeschränkung dokumentiert (vgl. Kapitel 6.7).

8.1.2 KI-Textgenerierung

Die Multi-Provider-Architektur mit Prioritätskette gewährleistet eine hohe Verfügbarkeit der Textgenerierung. Die qualitative Bewertung (Abschnitt 6.3.4) zeigt durchgängig gute Ergebnisse über alle drei Qualitätsdimensionen; die stärkste Einschränkung liegt in der Stil-Inkonsistenz des neutralen Profils. Die quantitativen Einzelwerte und Latenzmetriken sind

in Kapitel 6.3–6.5 vollständig dokumentiert.

8.1.3 Systemarchitektur und Designentscheide

Das System realisiert die in Kapitel 4 konzipierte dreischichtige Architektur vollständig (Datenschicht via n8n-ETL, Anwendungsschicht via FastAPI/PostgreSQL, Präsentationsschicht via React/TypeScript). Die **White-Label-Architektur** und die **drei Betriebsmodi** bilden die Kernbeiträge des Systemdesigns (vgl. Kap. 4.3.3, 5.3.3).

8.2 Beantwortung der Forschungsfrage

Die in Kapitel 1.2 formulierte Forschungsfrage lautet:

> *Wie lässt sich ein hybrides KI-gestütztes Redaktionssystem für die Liveticker-Erstellung im Profifußball konzipieren und prototypisch implementieren — und inwiefern erfüllt das System die definierten journalistischen Qualitätsanforderungen hinsichtlich Korrektheit, Tonalität und Verständlichkeit?*

Die Beantwortung erfolgt entlang der beiden Teile der Forschungsfrage; ein ergänzender Sekundärbefund adressiert das Latenzpotenzial.

Teil 1 — Konzeption und Implementierung. Die Realisierbarkeit ist durch die vollständige Implementierung belegt: Das entwickelte System realisiert die dreischichtige Architektur aus n8n-Datenbeschaffung, FastAPI/PostgreSQL-Backend und React/TypeScript-Frontend. Die 17 n8n-Workflows versorgen das System automatisiert mit Echtzeit-Daten; die LLM-Pipeline (Gemini 2.0 Flash Lite via OpenRouter) generiert Ticker-Einträge für alle definierten Event-Typen in drei Stilprofilen und mehreren Sprachen. Die White-Label-Architektur und das dreistufige Betriebsmodus-Konzept (`auto`, `coop`, `manual`) sind produktionsnah realisiert und im Cloud-Deployment erreichbar. Alle 23 definierten funktionalen, nicht-funktionalen und architektonischen Anforderungen werden erfüllt (vgl. Abschnitt 6.6); 371 automatisierte Tests und eine TypeScript-Typenabdeckung von 97,44 % belegen die technische Reife.

Teil 2 — Journalistische Qualität. Die journalistische Qualität wurde entlang der drei in der Forschungsfrage definierten Kriterien operationalisiert. **Korrektheit**: Die generierten Texte erreichen einen Ø-Wert von **4,6 / 5**. In 15 von 16 bewerteten Einträgen (94 %) wurden alle verfügbaren Fakten (Spieler, Team, Minute, Ergebnis) korrekt wiedergegeben (Hauptevaluation, N = 16, deutschsprachig). Die ergänzende Evaluation mit synthetischem Kontext (N = 9) bestätigt diesen Befund und erweitert ihn um bislang fehlende Event-Typen (Rote Karte, Phasen-Events). Die Pre-Match-Schutzregel verhindert zuverlässig die Erfindung von Live-Spielszenen; eine Wettempfehlung (1 von 16 Einträgen, 6 %) stellt eine inhaltliche Halluzination geringerer Schwere dar. **Tonalität**: Mit einem Ø-Wert von **4,1 /**

5 erzeugen die drei Stilprofile deutlich unterscheidbare Textstile. Der euphorische Modus erzielt die stärksten qualitativen Ergebnisse; das neutrale Profil zeigt die größte Inkonsistenz (19 % der Einträge zu emotional formuliert), bedingt durch die starken euphorischen Few-Shot-Referenzen. **Verständlichkeit**: Mit **4,3 / 5** erfüllen die generierten Texte die linguistischen Anforderungen des Liveticker-Genres (Kapitel 2.5) zuverlässig — kurze Satzkonstruktionen, Präsenskonstruktionen, idiomatische Ausrufe und konzeptionelle Mündlichkeit sind durchgehend vorhanden. Die Qualitätsbewertung stützt sich auf drei Evaluationsebenen: Hauptevaluation (N = 16, Abschnitt 6.3.4), Experteninterview (Abschnitt 6.3.5) sowie LLM-as-Judge-Evaluation (N = 20, Abschnitt 6.3.6), die den Gesamtdurchschnitt mit 4,5 / 5 bestätigt und dieselben Fehlerklassen repliziert.

Sekundärbefund — Publikationslatenzpotenzial. Die gemessenen Systemlatenzen (Median 859 ms, P95 2.047 ms, vgl. Abschnitt 6.5.1) legen ein substanzielles Potenzial zur Reduktion der Time-to-Publish nahe. Ohne kontrollierten Feldvergleich mit echten Redakteuren lässt sich dieses Potenzial jedoch nicht quantifizieren; die in Abschnitt 6.4 ausgewiesenen TTP-Schätzungen sind architekturbasierte Potenzialabschätzungen, keine empirisch gemessenen Werte. Ein Feldtest bleibt als Folgestudie vorgesehen (vgl. Abschnitt 8.3.2).

8.2.1 Synthese

Die Forschungsfrage ist in beiden Teilen positiv zu beantworten: Das System ist realisierbar und wurde vollständig implementiert; die Textqualität erfüllt die definierten journalistischen Anforderungen in allen drei Dimensionen. Der Sekundärbefund zeigt ein technisch plausibles, empirisch jedoch noch nicht feldvalidiertes Latenzpotenzial.

Die modus-spezifische Betrachtung ergibt dabei eine klare Empfehlung: Im `auto`-Modus ist das Latenzpotenzial maximal, die Qualitätssicherung jedoch nicht vollständig abgedeckt (Stil-Inkonsistenz 19 %, Halluzinationsrate 6 %). Im `coop`-Modus werden beide Anforderungen — Beschleunigung und Qualitätssicherung — gleichzeitig erfüllt: Die KI übernimmt die zeitkritische Textproduktion, der Redakteur die publizistische Verantwortung. **Der `coop`-Modus ist damit der optimale Betriebsmodus für den produktiven Einsatz.** Der `manual`-Modus bleibt als Rückfalloption und Vertrauensaufbau-Stufe erhalten. Das Human-in-the-Loop-Design löst den Speed-Quality-Trade-off nicht durch Kompromiss, sondern durch klare Aufgabenteilung.

8.3 Ausblick

8.3.1 Kurzfristige Erweiterungen

- **Authentifizierung und Rollensystem:** Die Integration eines JWT- oder OAuth-2.0-basierten Authentifizierungsframeworks ist die wichtigste Voraussetzung für einen Mehrbenutzerbetrieb. Ein rollenbasiertes Zugriffskonzept (Redakteur, Chefredakteur,

Administrator) würde die Freigabe-Workflows im `coop`-Modus um eine Vier-Augen-Prüfung erweitern.

- **Server-Sent Events für Ticker-Updates:** Die Ablösung des 5-Sekunden-Pollings durch SSE für den Ticker-Datenfluss würde die End-to-End-Latenz im `coop`-Modus weiter reduzieren und die Serverlast bei vielen gleichzeitigen Nutzern senken.
- **Erweiterte E2E-Tests:** Die Ausweitung der Playwright-Tests auf den vollständigen Redaktionsworkflow mit laufendem Backend und Testdatenbank (Spiel auswählen, Events empfangen, Ticker-Einträge bearbeiten, Freigeben) würde die Regressionssicherheit erhöhen.
- **n8n-Workflow-Tests:** Die 16 n8n-Workflows (vgl. Kapitel 6.7) sind derzeit ausschließlich manuell verifiziert. Integrationstests gegen eine Staging-Datenbank würden die Zuverlässigkeit der Datenversorgungspipeline absichern.
- **Few-Shot-Kontextualisierung in n8n-Workflows:** Die n8n-Workflows für Halbzeit- und Abpfiff-Zusammenfassungen (Workflow 13) rufen OpenRouter direkt auf, ohne Stilreferenzen aus der `style_references`-Tabelle einzubinden. Eine Erweiterung dieser Workflows um einen SQL-Schritt zur Referenzabfrage würde die stilistische Kohärenz zwischen Backend-generierten und n8n-generierten Einträgen herstellen.
- **Größere Evaluationsstichprobe:** Die Hauptevaluation umfasst 16 Einträge aus 9 Spielen — eine für explorative Erkenntnisse ausreichende, für statistisch belastbare Aussagen jedoch zu kleine Datenbasis. Eine Folgestudie mit ≥ 100 Einträgen aus verschiedenen Wettbewerben, Ligen und Spielsituationen würde die Generalisierbarkeit der Ergebnisse stärken und den Einsatz der implementierten nicht-parametrischen Verfahren (Cliff's Delta, Bootstrap-CI) rechtfertigen.
- **Langzeitbetrieb:** Das System wurde nicht über einen längeren Zeitraum evaluiert. Aspekte wie Drift der Textqualität über die Zeit, kumulative API-Kosten im Saison-Betrieb und Rate-Limit-Häufigkeit unter realer Dauerlast sind nicht erfasst. Eine Saisonbegleitung über mindestens 34 Bundesliga-Spieltage würde hier belastbare Betriebsdaten liefern.

8.3.2 Mittelfristige Erweiterungen

- **Externe Nutzerstudie:** Eine systematische Evaluation mit professionellen Sportredakteuren, die das System im realen Spielbetrieb nutzen, würde die externe Validität der Qualitätsbewertung stärken. Das implementierte Evaluationsframework (TTP, Cliff's Delta, Cohen's Kappa) ist dafür vorbereitet.
- **Fine-Tuning statt Few-Shot:** Für vereinsspezifische Instanzen könnte ein Fine-Tuning auf dem Korpus historischer Vereinsticker die stilistische Konsistenz über den Few-Shot-Ansatz hinaus verbessern. Die `style_references`-Tabelle bildet bereits ei-

ne kuratierte Datenbasis, die als Ausgangspunkt für ein Fine-Tuning-Dataset dienen könnte.

- **Differenziertes Polling:** Die vorbereitete Infrastruktur in `resolvePollingInterval` ermöglicht eine phasenabhängige Anpassung der Polling-Frequenz (z. B. 30 Sekunden im PreMatch, 5 Sekunden während Live). Dies würde die Serverlast im Ruhezustand reduzieren.
- **Evaluation der Mehrsprachigkeit:** Eine separate Qualitätsevaluation der mehrsprachigen Textgenerierung — insbesondere für die im Kontext von Eintracht Frankfurt relevanten Sprachen Englisch und Japanisch — würde die in Kapitel 2.2 motivierte Internationalisierungsanforderung empirisch absichern.

8.3.3 Langfristige Perspektiven

- **Multimodale Ticker:** Die bestehende Integration von Medieninhalten (ScorePlay, YouTube, Instagram) könnte zu einem multimodalen Ticker weiterentwickelt werden, in dem Bild, Video und Text automatisch zu einem kohärenten Narrativ verwoben werden. Vision-Language-Modelle könnten dabei Spielszenen aus Bildern beschreiben und in den Tickertext integrieren.
- **Autonomes Scoring der Textqualität:** Die Ablösung der manuellen Qualitätsbewertung durch ein trainiertes Scoring-Modell, das Korrektheit, Tonalität und Verständlichkeit automatisch bewertet, würde eine kontinuierliche Qualitätsüberwachung im Produktivbetrieb ermöglichen. Die implementierte `aggregate_quality_by_group`-Funktion bildet die Aggregationslogik hierfür bereits ab.
- **Generalisierung über den Fußball hinaus:** Die Provider-agnostische LLM-Architektur und das Event-basierte Generierungsmodell sind nicht fußballspezifisch. Eine Erweiterung auf andere Sportarten (Handball, Basketball, Eishockey) erforderte primär die Anpassung der Event-Typ-Mappings, der Phasendefinitionen und der Kontextbuilder — die Kernarchitektur bliebe unverändert.

8.4 Schlusswort

Die vorliegende Arbeit zeigt, dass ein hybrides KI-gestütztes Redaktionssystem den operativen Zeitdruck bei der Liveticker-Erstellung adressieren kann, ohne die redaktionelle Kontrolle aufzugeben. Der `coop`-Modus realisiert das in Kapitel 1.1 formulierte Zielbild eines Systems, in dem „die finale Entscheidungshoheit und publizistische Verantwortung beim Menschen verbleiben".

Die technische Reife des Systems (vgl. Kap. 6.2) sowie die Erfüllung aller definierten Anforderungen dokumentieren die Tragfähigkeit des Architekturansatzes. Die in Kapitel 7 diskutierten Limitationen markieren klare Erweiterungspfade, stellen aber die grundsätzliche

Funktionalität nicht in Frage.

Die KI-gestützte Liveticker-Generierung ersetzt den Redakteur nicht — sie gibt ihm die Zeit zurück, die er braucht, um seiner eigentlichen Aufgabe nachzukommen: Journalismus.